Rapport de projet

Travail de maturité

01.11.2026

ABRAXAS

anonyme-sjf-26

TABLE DES MATIÈRES

1 Introduction 5
Source du projet 5
Rappel de la problématique 5
Forme du projet 6
Démarche générale 6
Choix techniques principaux 6
Critères de réussite 6
2 Notes 7
2.1 Usage de l'IA 7
2.2 Sources 7
Liens 7
Photographies 7
3 Lexique et théorie vélique 9
3.1 Unités 9
3.2 Référentiels 9
Référentiel Terre 9
Référentiel Bateau 9
3.3 Convention 9
3.4 Déplacement au-dessus du fond 10
SOG 10
COG 10
3.5 Vent réel 10
3.6 Vent vitesse 11
3.7 Vent apparent 12
3.8 Calcul du vent réel 12
3.9 Stratégie de gain 13
Gain au vent 13
Gain sous le vent 13
3.10 Calcul de la VMG 14
3.11 Pitch et roll 14
4 Intentions 15
4.1 Identification des besoins 15
4.2 Solutions existantes 15
4.3 Exigences de la solution 17
4.4 Licence MIT 18
4.5 Nom du dispositif 19
4.6 Logo du dispositif 20
5 Premier prototype 21
5.1 Matériel 21
Commande 1 21
Commande 2 21
Commande 3 22
Commande 4 22
5.2 Montage 23

5.3 Installation de l'OS ... 24
5.4 SSH par clés publiques ... 24
Chiffrement asymétrique ... 25
NAT-Loopback et Port mapping ... 25
Changement de clé publique du Raspberry ... 26
5.5 Transfert de fichiers ... 27
Dans la pratique ... 28
5.6 Paquets et librairies ... 28
5.7 Test des GPIO ... 29
5.8 Réglage de Serial Port ... 30
5.9 Usage de l'IMU ... 32
5.10 Usage du GPS ... 33
Installation de Blinka ... 33
Précaution particulière ... 35
Installations de la bibliothèque relative au GPS ... 36
Test du GPS ... 36
5.11 Usage de la caméra ... 38
5.12 Usage de l'anémomètre ... 40
Choix de l'anémomètre ... 40
Raccordement au prototype ... 41
5.13 Base de données SQL ... 42
6 Software du premier prototype ... 45
6.1 Fichier de classes — classes.py ... 45
Bibliothèque pour le logging ... 46
6.2 Surveillance du bouton — interface.py ... 46
Exécution automatique ... 46
6.3 Gestion de la session — master.py ... 47
6.4 Gestion de l'IMU et du GPS — imu.py & gps.py ... 48
6.5 Enregistrement de la vidéo — vid.py ... 49
6.6 Finalisation de l'enregistrement de la vidéo — grp.py ... 50
6.7 Installation automatique ... 51
6.8 Fichiers de configuration ... 51
progress_bar.sh ... 51
config.sh ... 52
auto_install.exp ... 53
config.sql ... 53
6.9 Passage sur GitHub ... 53
7 Deuxième prototype ... 55
7.1 Alimentation autonome ... 55
Batterie et autonomie ... 55
PowerBoost ... 55
ON/OFF SHIM ... 55
7.2 Carte mère ... 56
Schéma électrique ... 56
7.3 Empreinte du PCB ... 61
8 Travail 3D ... 63

8.1 Modélisation ... 63
Boîte ... 63
Couvercle ... 66
Entretoises amovibles ... 66
Fixations compatibles GoPro ... 66
Support pour l'anémomètre ... 66
8.2 Impression ... 67
8.3 Montage ... 67
Éléments supplémentaires pour le montage ... 67
Soudures ... 68
Plan de câblage ... 69
Assemblage ... 69
9 Software du deuxième prototype ... 71
9.1 Usage ... 71
Mise en place du dispositif ... 71
Allumer et éteindre le dispositif ... 71
Témoins ... 71
Lancer et arrêter un enregistrement ... 72
Affichage des données en temps réel ... 72
Accès au debrief ... 73
9.2 Installation automatique ... 74
config.sh ... 75
config.sql ... 75
progress_bar.sh ... 75
auto_install.exp ... 75
9.3 Système d'enregistrement ... 75
MQTT ... 75
db_recording.py ... 76
anemo.py, imu.py, gps.py ... 76
interface.py ... 76
master.py ... 76
vid.py ... 76
grp.py ... 76
classes.py ... 76
gpio_daemon.py ... 76
9.4 Hotspot ... 77
server.js ... 77
client.html ... 77
client.css ... 77
horizon.png ... 78
client.js ... 78
export.html ... 78
export.css ... 78
export.js ... 78
9.5 Serveur distant ... 78
Système du serveur ... 79

Fonctionnement et objectif du serveur 79
Debrief IA 80
Rôle du serveur au sein du projet 81
Code du serveur 82
9.6 Abraxas OS 82
10 Essai 83
10.1 Installation du dispositif 83
10.2 Déroulement de l'entraînement 83
10.3 Fonctionnement technique du dispositif 85
10.4 Qualité de l'analyse et de la visualisation 86
10.5 Inconvénients de l'usage du dispositif pour l'équipage 86
10.6 Bénéfices de l'usage du dispositif pour l'équipage 86
11 Conclusion 91
11.1 Synthèse 91
Sur le plan technique 92
Sur le plan du bénéfice pour l'équipe 92
11.2 Améliorations et perspectives 92
11.3 Ouverture 93
12 Remerciements 95
13 Bibliographie 97
13.1 Ouvrages 97
13.2 Entretiens et sources orales 97
13.3 Systèmes existants 97
13.4 Documentation technique et dépôts de code 97
13.5 Articles et pages web 98
13.6 Résultats de régates 98
14 Annexes 99
14.1 Photos du prototype 99
14.2 Accès au PDF du rapport et aux dépôts GitHub 113
14.3 Accès au modèle CAD 114
14.4 Accès au serveur 114
14.5 Accès au hotspot 115
14.6 Sites des solutions existantes 115
14.7 Matériel 115
14.8 Documentation technique 117
14.9 Articles et pages web 118
14.10 Accès aux codes 118

1 INTRODUCTION

Source du projet

Au printemps 2025, j'ai pris part avec mon équipe au Critérium Longtze, une étape du circuit européen de Longtze, un voilier monocoque performant de compétition.

Nous nous sommes classés douzièmes sur quatorze[1]. Ce classement a été le résultat d'une saison complète d'entraînements assidus. De toute évidence, quelque chose dans la façon dont nous nous entraînions devait être revu.

Dans notre club, nous nous entraînons à trois équipages sur trois bateaux. Lors des séances, nous naviguons ensemble en comparant nos vitesses, nos manœuvres et nos stratégies. La clef de voûte d'une bonne performance était et est toujours pour nous la reproductibilité et la constance de nos performances. En effet, l'équipage qui gagne une régate n'est pas celui qui est le meilleur dans ses actions ou dans ses décisions, mais celui qui commet le moins d'erreurs.

À ce titre, lorsque les régates d'entraînement ont donné des résultats satisfaisants, à savoir que les différents équipages naviguaient à un niveau équivalent et sans laisser voir de faiblesse majeure dans leur technique, nous sommes passés au perfectionnement d'éléments de moindre importance.

Cependant, lors du critérium, nous nous sommes rendu compte que les bases que nous croyions acquises étaient à revoir en profondeur. Bord à bord, nos concurrent·e·s avaient systématiquement une vitesse supérieure à la nôtre et un cap favorable. Si nous nous en étions rendu compte durant l'entraînement, nous aurions pu travailler ces points et ainsi rester compétitifs lors du critérium.

Le projet Abraxas est donc né d'une frustration, et du désir de ne pas rester bloqué dessus, mais au contraire d'avancer et de tout mettre en œuvre pour éviter qu'une telle situation ne se reproduise.

C'est tout naturellement que mon choix de sujet de travail de maturité s'est porté vers le développement d'une solution à ce problème.

Rappel de la problématique

La problématique formulée en début d'année pour mon travail est la suivante :

> Comment serait-il possible, à l'aide d'un dispositif informatique, de collecter des données durant les entraînements d'une équipe de compétition en voile inshore[2] en vue de fournir de la matière supplémentaire pour les debriefings et, à terme, de donner à l'équipe l'opportunité d'améliorer ses performances ?

[1] « Classement du Criterium Longtze YCG 2025 », *Manage2Sail*, 04.05.2025, https://www.manage2sail.com/fr/event/Longtze2025 (consulté le 21.10.2026).

[2] Inshore désigne la pratique de la voile en compétition sur des plans d'eau abrités, tels que les lacs ou les zones côtières. Les régates disputées sont courtes, techniques et dynamiques. Les bateaux employés sont généralement légers, rapides et particulièrement exigeants.

> Après avoir répondu à cette question, je tâcherai de réaliser un tel dispositif, et ce tout en gardant à l'esprit que de tels dispositifs existent déjà, mais sont réservés à l'élite de par leur prix et leur complexité. Ce projet a donc également pour but de permettre la démocratisation de tels systèmes.

Cette problématique n'a pas évolué et s'applique encore au projet.

Forme du projet

Le projet se nomme Abraxas, en référence au roman *Demian*, de Hermann Hesse[3]. Une explication du choix de ce nom est donnée au Chapitre 4.5. À terme, le résultat du projet se compose d'un boîtier pouvant être attaché à la base du mât, d'un anémomètre relié par un câble au boîtier, et d'une antenne GPS externe. Le boîtier est le centre du dispositif, c'est depuis là qu'il est contrôlé.

Les mesures effectuées concernent le vent, la vitesse, la position du bateau et son orientation dans l'espace. Une vidéo est également enregistrée. Le dispositif ne dispose pas lui-même d'un écran, mais on peut utiliser n'importe quel appareil pouvant se connecter à un WiFi pour interagir avec lui.

Démarche générale

La première étape dans la réalisation de ce projet a été de définir un cahier des charges précis en cernant les intentions qui le motivent. J'ai ensuite entamé la conception puis l'assemblage d'un premier prototype. L'objectif n'était pas de pouvoir l'utiliser en situation réelle, mais de prouver le concept en réalisant un montage fonctionnel de tous les composants entre eux. Une fois ce premier prototype réalisé, j'ai pu passer à la construction d'un deuxième qui, cette fois-ci, allait pouvoir être embarqué sur un bateau. Cela a amené un nouveau lot de difficultés, liées notamment aux contraintes de robustesse et d'étanchéité. Ce n'est qu'après avoir pu constituer un jeu de données exploitables avec ce prototype que je me suis lancé dans la création de deux systèmes de visualisation des données. L'un affiche les données en temps réel, l'autre est conçu pour être utilisé durant le debrief.

Choix techniques principaux

Le dispositif est construit autour d'un Raspberry Pi Zero. Fonctionnant sur un Linux complet (sans interface graphique), il offre toute la puissance nécessaire pour le traitement massif de données et pour l'enregistrement vidéo. Le boîtier est imprimé en trois dimensions et contient une carte mère sur laquelle sont fixés tous les composants.

Critères de réussite

La condition principale pour considérer mon projet comme abouti est d'avoir un dispositif en mesure d'enregistrer, de traiter et de présenter des données à propos d'un entraînement de voile inshore respectant les principes d'abordabilité, d'adaptabilité aux clubs, d'open-source, de no-nonsense et de prise en compte multifactorielle. Ces principes sont développés au Chapitre 4.3. Le dispositif devrait à terme être inclus dans les entraînements et contribuer à leur amélioration afin d'éviter de se retrouver à nouveau dans la situation du critérium 2025.

[3] HESSE Hermann, *Demian. Die Geschichte von Emil Sinclairs Jugend*, Berlin, Suhrkamp, 2012.

2 NOTES

2.1 Usage de l'IA

Comme on le verra, l'usage de l'IA au sens large fait intrinsèquement partie de mon travail, notamment dans l'analyse des données. L'usage des grands modèles de langage (LLMs) est un cas différent. C'est pour moi, même en dehors du cadre de ce travail, un questionnement difficile. Je pense être pleinement conscient de leurs limites et des dangers inhérents à leur usage, je ne suis néanmoins pas aveugle aux avantages qu'ils ont.[4] Je n'ai pas décidé de totalement les laisser de côté, mais plutôt d'en faire un usage réfléchi. Dans cette démarche, j'ai eu la chance de recevoir les lumières d'une philosophe spécialiste dans les questions relatives aux usages de l'IA.

Dans le cadre de ce projet, aucun texte n'a été écrit ni reformulé par IA. Je n'ai pas utilisé d'IA pour organiser ou développer mes idées, ni faire de relecture de fond. C'est uniquement dans la partie informatique que j'ai eu recours à des LLMs. Le contenu généré par IA présent dans le debrief des entraînements est marqué comme tel. J'ai notamment utilisé des processus IA pour :

- Obtenir un avis sur mes idées d'architecture de code.
- Identifier et résoudre des erreurs.
- Effectuer des complétions.
- Découvrir, apprendre et comprendre de nouveaux langages et de nouvelles syntaxes.
- Recevoir des suggestions sur des cas complexes, comme l'échange de données en temps réel entre deux langages différents.
- Générer automatiquement des textes de commit et des notes de version sur GitHub.

J'ai également, dans certains cas, délégué l'écriture d'un code à un LLM. Lorsque c'est le cas :

- Je ne considère pas le code comme faisant partie intégrante de mon travail, mais comme un outil que j'ai choisi et que j'utilise, au même titre que j'utiliserais une bibliothèque que j'aurais importée.
- Je le précise explicitement dans mon rapport.
- J'en explique la raison et donne une justification.

2.2 Sources

Liens

Tous les liens et fichiers mentionnés dans ce document sont présents sous forme de QR-codes dans les annexes (Chapitre 14), y compris la version PDF du présent rapport.

Photographies

Sauf mention contraire, toutes les photographies ont été prises par l'auteur en 2026.

[4] Philosophe spécialiste des questions relatives à l'IA, discussion du 09.06.2026.



Fig. 1. – Photo prise lors du SailGP 2026, à Genève. À l'avant plan, on voit le F50 allemand. Derrière lui, on aperçoit le F50 états-uniens.

3 LEXIQUE ET THÉORIE VÉLIQUE

3.1 Unités

Les unités utilisées en voile sont les suivantes[5] :

Grandeur	Unité	Équivalence
Distances en miles nautiques	NM	1 NM = 1852 m
Vitesse en nœuds	knt	1 knt = 1,852 km/h
Angles en degrés, minutes et secondes	00°00'00''	1° = 60' = 3600''

3.2 Référentiels

On définit deux référentiels, le référentiel Bateau et le référentiel Terre. Le décalage existant entre la direction des axes de ces référentiels est le cap vrai du bateau (COG). Pour passer une direction du référentiel Bateau au référentiel Terre, on additionne le COG.

Référentiel Terre

On oriente ce référentiel avec l'axe x pointant vers le Nord vrai et l'axe y vers l'Est. L'axe z pointe vers le bas, le sens de rotation positif est donc horaire.

Référentiel Bateau

On oriente ce référentiel avec l'axe x selon l'axe longitudinal (de l'arrière vers l'avant) et l'axe y selon l'axe transversal du bateau (de bâbord vers tribord). L'axe z pointe également vers le bas, le sens de rotation positif est donc également horaire.

On néglige la dérive et les courants : le cap vrai du bateau (HDG), soit l'orientation de l'axe x du bateau par rapport au Nord vrai, est égal à l'angle de sa trajectoire par rapport au Nord vrai (COG).

3.3 Convention

Au sens strict du terme, les vecteurs vent ont pour sens le sens dans lequel le vent souffle. Cependant, l'affichage des données de l'angle du vent se fait par convention dans le sens inverse. Au près, le vent souffle à environ 220° de l'axe x du bateau. On dit alors que le vent est à 40°. Par abus de langage, on dit que le vent apparent est à 40°, il est en réalité à 220°.

Par souci de clarté, on considère dans ce document que le sens d'un vecteur vent est la direction vers laquelle il souffle. Seul l'affichage final sera conforme à la convention. On y soustraira simplement 180°.

[5] Entraîneur de l'équipe de compétition du Yacht Club de Genève, *Cours théoriques, briefing et debriefing*, connaissances orales non publiées.

3.4 Déplacement au-dessus du fond

Le déplacement au-dessus du fond est le vecteur de déplacement du référentiel Bateau par rapport au référentiel Terre. Il est noté $\vec{B}$ (de Bewegung, mouvement en allemand).

Dans le référentiel Terre :

$$\vec{B} = \begin{pmatrix} \mathrm{SOG} \cdot \cos(\mathrm{COG}) \\ \mathrm{SOG} \cdot \sin(\mathrm{COG}) \end{pmatrix}$$

Dans le référentiel Bateau :

$$\vec{B} = \begin{pmatrix} \mathrm{SOG} \\ 0 \end{pmatrix}$$

SOG

La vitesse au-dessus du fond est appelée Speed Over Ground (SOG). Elle est calculée à partir des mesures de position d'un GPS : il s'agit de la norme de la dérivée de la trajectoire.

Il peut arriver qu'elle diffère de la vitesse sur la surface de l'eau, dans le cas de courants. Dans le lac, ce phénomène est négligeable ; il peut cependant devenir primordial de le prendre en compte dans des régions à fort courant telles que la Bretagne nord, où les courants peuvent atteindre 3knt[6].

COG

Le cap au-dessus du sol est appelé Course Over Ground (COG). Il est calculé à partir des mesures de position d'un GPS : il s'agit de l'angle par rapport au nord vrai de la dérivée de la trajectoire.

Il peut différer du cap sur la surface à cause des courants. Il diffère également du cap par rapport au nord magnétique (déclinaison magnétique), cependant ce phénomène est négligeable à nos latitudes (quelques minutes d'angle). Il devient significatif en montant au nord : le delta peut atteindre 15° en Laponie[7].

3.5 Vent réel

Le vent réel est le vecteur de déplacement de la masse d'air par rapport au référentiel Terre en un point. C'est celui qui sera ressenti par un·e observateur·trice immobile sur la côte. Il est noté $\overrightarrow{V_R}$.

On appelle True Wind Speed (**TWS**) la norme de ce vecteur et True Wind Angle (**TWA**) l'angle par rapport à l'axe du bateau.

[6] Id.
[7] Id.

Dans le référentiel Terre :

$$\overrightarrow{V_R} = \begin{pmatrix} \mathrm{TWS} \cdot \cos(\mathrm{COG} + \mathrm{TWA}) \\ \mathrm{TWS} \cdot \sin(\mathrm{COG} + \mathrm{TWA}) \end{pmatrix}$$

Dans le référentiel Bateau :

$$\overrightarrow{V_R} = \begin{pmatrix} \mathrm{TWS} \cdot \cos(\mathrm{TWA}) \\ \mathrm{TWS} \cdot \sin(\mathrm{TWA}) \end{pmatrix}$$

Le TWA est différent du True Wind Direction (**TWD**) météorologique : le TWD indique la direction depuis laquelle le vent souffle par rapport au nord vrai, alors que le TWA indique la direction depuis laquelle le vent souffle par rapport à l'axe du bateau. En d'autres termes, le TWD n'est autre que le TWA exprimé dans le référentiel Terre.

Par exemple : La bise est un vent soufflant du nord. Son TWD est 0°. Un bateau remontant à 30° du vent réel observera un TWA de 30°.

3.6 Vent vitesse

Le vent vitesse est le vecteur de déplacement du référentiel Terre par rapport au référentiel Bateau. C'est le vent qui est ressenti par un·e observateur·trice immobile sur le bateau en déplacement si le vent réel est égal à $\vec{0}$. Il est noté $\overrightarrow{V_V}$.

On appelle Induced Wind Speed (**IWS**) la norme de ce vecteur et Induced Wind Angle (**IWA**) l'angle par rapport à l'axe du bateau.

Étant donné que le vent vitesse est créé par le déplacement du bateau, l'IWA est toujours égal à 180°, sauf si le bateau dérive ou si le courant le déplace latéralement. On considère ces effets comme négligeables sur le lac.

Une fois cette approximation faite, le vent vitesse a la propriété de valoir l'opposé du vecteur du déplacement au-dessus du fond $\vec{B}$.

Dans le référentiel Terre :

$$\overrightarrow{V_V} = \begin{pmatrix} \mathrm{IWS} \cdot \cos(\mathrm{COG} + \mathrm{IWA}) \\ \mathrm{IWS} \cdot \sin(\mathrm{COG} + \mathrm{IWA}) \end{pmatrix} = -\vec{B} = \begin{pmatrix} -\mathrm{SOG} \cdot \cos(\mathrm{COG}) \\ -\mathrm{SOG} \cdot \sin(\mathrm{COG}) \end{pmatrix}$$

Dans le référentiel Bateau :

$$\overrightarrow{V_V} = \begin{pmatrix} \mathrm{IWS} \cdot \cos(\mathrm{IWA}) \\ \mathrm{IWS} \cdot \sin(\mathrm{IWA}) \end{pmatrix} = -\vec{B} = \begin{pmatrix} -\mathrm{SOG} \\ 0 \end{pmatrix}$$

avec $\mathrm{IWS} = \mathrm{SOG}$ et $\mathrm{IWA} = 180°$ (le vent vitesse souffle vers l'arrière).

3.7 Vent apparent

Le vent apparent est le vecteur de déplacement de la masse d'air par rapport au référentiel Bateau. Il est aussi la résultante du vent réel et du vent vitesse. C'est le vent qui est observé par un·e observateur·trice immobile sur un bateau en mouvement. Il est noté $\overrightarrow{V_A}$.

On appelle Apparent Wind Speed (**AWS**) la norme de ce vecteur et Apparent Wind Angle (**AWA**) l'angle par rapport à l'axe du bateau.

Dans le référentiel Terre :

$$\overrightarrow{V_A} = \begin{pmatrix} \mathrm{AWS} \cdot \cos(\mathrm{COG} + \mathrm{AWA}) \\ \mathrm{AWS} \cdot \sin(\mathrm{COG} + \mathrm{AWA}) \end{pmatrix}$$

Dans le référentiel Bateau :

$$\overrightarrow{V_A} = \begin{pmatrix} \mathrm{AWS} \cdot \cos(\mathrm{AWA}) \\ \mathrm{AWS} \cdot \sin(\mathrm{AWA}) \end{pmatrix}$$

3.8 Calcul du vent réel

On mesure facilement le vent apparent $\overrightarrow{V_A}$ grâce à un anémomètre, on calcule le vent vitesse $\overrightarrow{V_V}$ à partir de $\vec{B}$, mesuré avec le GPS. À partir de là, on veut obtenir le vent réel $\overrightarrow{V_R}$.

Le calcul est effectué dans le référentiel Terre.

$$\overrightarrow{V_A} = \overrightarrow{V_R} + \overrightarrow{V_V} \Rightarrow \overrightarrow{V_R} = \overrightarrow{V_A} - \overrightarrow{V_V} = \overrightarrow{V_A} + \vec{B}$$

$$\overrightarrow{V_R} = \begin{pmatrix} \mathrm{AWS} \cdot \cos(\mathrm{COG} + \mathrm{AWA}) \\ \mathrm{AWS} \cdot \sin(\mathrm{COG} + \mathrm{AWA}) \end{pmatrix} + \begin{pmatrix} \mathrm{SOG} \cdot \cos(\mathrm{COG}) \\ \mathrm{SOG} \cdot \sin(\mathrm{COG}) \end{pmatrix}$$

$$\overrightarrow{V_R} = \begin{pmatrix} \mathrm{AWS} \cdot \cos(\mathrm{COG} + \mathrm{AWA}) + \mathrm{SOG} \cdot \cos(\mathrm{COG}) \\ \mathrm{AWS} \cdot \sin(\mathrm{COG} + \mathrm{AWA}) + \mathrm{SOG} \cdot \sin(\mathrm{COG}) \end{pmatrix} = \begin{pmatrix} V_{R_x} \\ V_{R_y} \end{pmatrix}$$

$$\mathrm{TWS} = \sqrt{V_{R_x}^2 + V_{R_y}^2}$$

$$\mathrm{TWD} = \mathrm{atan2}\big(V_{R_y}, V_{R_x}\big)$$

On passe dans le référentiel Bateau :

$$\mathrm{TWA} = \mathrm{TWD} - \mathrm{COG}$$

Ou, selon la convention consistant à afficher la direction d'où vient le vent :

$$\mathrm{TWA}_{\mathrm{convention}} = \mathrm{TWA} + 180°$$

Dans leurs formes développées :

$$\mathrm{TWS} = \sqrt{\begin{aligned}&(\mathrm{AWS} \cdot \cos(\mathrm{COG} + \mathrm{AWA}) + \mathrm{SOG} \cdot \cos(\mathrm{COG}))^2 \\ &+(\mathrm{AWS} \cdot \sin(\mathrm{COG} + \mathrm{AWA}) + \mathrm{SOG} \cdot \sin(\mathrm{COG}))^2\end{aligned}}$$

$$\begin{aligned}\mathrm{TWA}_{\mathrm{convention}} = \operatorname{atan2}(&\mathrm{AWS} \cdot \sin(\mathrm{COG} + \mathrm{AWA}) + \mathrm{SOG} \cdot \sin(\mathrm{COG}), \\ &\mathrm{AWS} \cdot \cos(\mathrm{COG} + \mathrm{AWA}) + \mathrm{SOG} \cdot \cos(\mathrm{COG}) + 180)\end{aligned}$$

Ce sont ces valeurs de vent réel qui sont affichées dans le dashboard, elles sont issues des calculs ci-dessus. La seule différence est que l'on travaille en radians dans le code.

3.9 Stratégie de gain

Gain au vent

Il est impossible avec un voilier de se diriger dans la direction d'où vient le vent. S'il veut atteindre un objectif situé au vent, il doit naviguer avec un angle d'environ 45° par rapport à son objectif, puis virer de bord (tourner de façon à ce que le vent frappe l'autre côté du bateau en passant la proue face au vent) et continuer à naviguer sur ce bord. C'est le processus que l'on appelle louvoyage.

Il y a un angle optimal de remontée au vent : plus on lofe (se rapprocher du vent), plus on ralentit, mais plus on va dans la bonne direction ; plus on abat (s'éloigner du vent), plus on accélère, mais moins on va dans la bonne direction. L'objectif est de maximiser la composante de la vitesse du bateau selon l'axe du vent. Cette composante est nommée Velocity Made Good (**VMG**), en français vitesse capitalisée[8].

Gain sous le vent

Si l'on veut atteindre un objectif sous le vent, le plus efficace n'est pas non plus d'y aller en ligne droite. Le problème est ici que la vitesse du bateau est créée par le vent apparent, pas par le vent réel. S'il va dans la même direction que le vent, sa vitesse va se soustraire au vent réel et le vent apparent va diminuer. Afin d'éviter ce phénomène, on navigue vers un objectif sous le vent en alternant des bords à environ 135° du vent. Ainsi, le vent apparent ne risque pas de tendre vers zéro. Au contraire : si l'on construit assez de vent vitesse, le vent apparent va devenir considérablement plus grand que le vent réel, et sa direction va changer, de sorte que le AWA peut être de 45° alors que le TWA est lui de 135°. C'est ce phénomène qui permet d'atteindre les vitesses les plus élevées sur des bateaux performants.

À nouveau, il y a ici un optimum : plus on lofe, plus on accélère, mais moins on va dans la bonne direction ; plus on abat, plus on ralentit, mais plus on va dans la bonne direction. L'objectif est de minimiser la VMG (maximiser sa valeur absolue)[9].

[8] Id.
[9] Id.

3.10 Calcul de la VMG

À nouveau, au sens strict, la VMG devrait être positive si l'on va dans le sens dans lequel souffle le vent, mais elle est par convention positive quand on remonte le vent. On doit donc artificiellement en inverser le signe. Le calcul de la VMG est donc le calcul de l'opposé de la composante selon le TWA du SOG.

$$\mathrm{VMG} = -\ \mathrm{SOG} \cdot \cos(\mathrm{TWA})$$

Si l'on considère le TWA selon la convention, le signe de la VMG est cohérent :

$$\mathrm{VMG} = \mathrm{SOG} \cdot \cos(\mathrm{TWA}_{\mathrm{convention}})$$

3.11 Pitch et roll

Le pitch, tangage en français, est le mouvement de rotation effectué autour de l'axe y du bateau : c'est le mouvement qui est effectué lorsque le bateau pique du nez. Cette valeur est sur les bateaux archimédiens souvent proche de zéro et donc difficilement mesurable. C'est sur les bateaux à foil qu'elle prend tout son sens : les foils sont optimisés pour une valeur de pitch pouvant parfois aller jusqu'à −10°[10].

Le roll, roulis, ou, plus généralement, gîte, est le mouvement de rotation effectué selon l'axe x du bateau : c'est le mouvement qui est effectué quand le bateau s'incline sous l'effet du vent par exemple. Chaque bateau a, en fonction de la forme de sa coque, une valeur de roll pour laquelle ses performances sont optimisées. Le but de l'équipage est donc, dans la mesure du possible, de maintenir le bateau dans cette position[11].

[10] Id.
[11] Id.

4 INTENTIONS

4.1 Identification des besoins

Suite à l'identification du problème concernant notre méthode d'entraînement décrite dans l'introduction s'est rapidement posée la question de la résolution. Quelles sont les stratégies que nous devrions mettre en place pour remédier à ce problème ? Quels sont les moyens dont nous devrions disposer afin de pouvoir le faire ?

Le problème ne venait pas de la qualité de l'entraînement lui-même, mais du manque de données objectives auxquelles se comparer. En effet, il est possible de ne pas se rendre compte de l'existence d'une lacune majeure, simplement parce que tous nos points de comparaison au niveau local la partagent.

Nous avons donc décidé de rechercher une manière de récolter ces données afin de non seulement pouvoir détecter ce type de problème à l'avenir, mais également pour identifier les mécanismes qui y conduisent. Pour ce faire, j'ai imaginé un appareil capable de mesurer différentes variables significatives du fonctionnement d'un bateau, et un logiciel capable de les analyser et de mettre en exergue les axes d'amélioration. Parmi ces variables, nous souhaitions avoir le SOG, le COG, le TWS, le TWA, le AWS, le AWA et surtout la VMG. Nous voulions également une vidéo de l'équipage durant la navigation pour faire le lien entre nos actions et les conséquences sur les données mesurées. Pour finir, ces données ne devaient pas être analysées individuellement, mais être mises en perspective sur plusieurs entraînements afin de pouvoir identifier une réelle évolution des performances.

4.2 Solutions existantes

Notre équipe n'est bien sûr pas la première à rencontrer ce besoin, et il existe à ce titre plusieurs systèmes proposant un service semblable. Nous en avons identifié quatre :

- Montres connectées de type Garmin

Avec une fourchette de prix s'étendant d'environ 250 à 1500CHF[12], ce type de montres représente un investissement conséquent pour une personne, mais envisageable pour un club. C'est un outil que nous utilisons d'ores et déjà pour connaître notre vitesse (SOG). Il occupe une place importante dans la façon dont nous pratiquons notre sport. Néanmoins, il est plus axé sur la performance individuelle (fréquence cardiaque, nombre de calories dépensées, etc.) que sur la performance d'équipe. De plus, il n'offre ni possibilité de prise en compte du vent ni de réel suivi de performance entre les entraînements. Or, sans données de vent, les données de vitesses sont inutilisables après la fin de l'entraînement.

[12] « Wearables smartwatches », *Garmin*, s.d., https://www.garmin.com/fr-CH/c/wearables-smartwatches/ (consulté le 11.10.2026).



Fig. 2. – Photographie de notre équipage lors du Critérium Longtze 2025.
Sans titre, 04.05.2025, collection personnelle, avec l’accord de la photographe.

- Vakaros

Il s'agit d'un système américain de monitoring d'entraînements, spécialisé dans les sports véliques. À ce titre, il est parfaitement adapté au besoin d'une équipe pour optimiser la navigation pendant l'entraînement. Néanmoins, il n'offre pas non plus de possibilité de prise en compte du vent, ce qui ne permet pas un suivi efficace. De plus, il est vendu au prix prohibitif de 1085CHF[13] sans accessoires.

- Orca

Ce système norvégien est conçu pour les bateaux lestés naviguant en mer. J'ai eu l'occasion de le tester durant une traversée de plusieurs jours en mer Baltique et d'en constater la redoutable efficacité. Cependant, il est axé sur l'aide à la navigation au long cours et non sur l'optimisation des performances sur des entraînements de voile légère. Il est malgré tout en mesure de produire un rapport de performance parfaitement exploitable, et au prix de 599CHF[14], c'eût probablement été la meilleure option sur le marché. De plus, il offre la prise en compte du vent. Malheureusement, sa mise en place sur le bateau est complexe : il ne s'agit pas d'un dispositif amovible mais permanent. Cela ne convient absolument pas aux besoins d'un club. Un tel outil devrait pouvoir être utilisé sur différents bateaux ou être totalement enlevé lors de cours de voile pour débutant·e·s.

- Systèmes sur mesure réservés à l'élite

Pour des équipes professionnelles d'élite telles que celles prenant part au Sail Grand Prix (SailGP) ou à la Coupe de l'America (AC), des systèmes de monitoring extrêmement précis et directement intégrés à la conception du bateau existent. On trouve extrêmement peu d'informations à ce sujet, car ils représentent un avantage comparatif considérable. Chaque organisation garde secret le fonctionnement exact de son système. Ces dispositifs sont évidemment les meilleurs, car les plus spécialisés et axés sur la régate. Ils prennent en compte un nombre gigantesque de paramètres, jusqu'aux forces et aux moments internes à la structure du bateau ainsi que les micro-vibrations auxquelles il est soumis. De tels systèmes sont par ailleurs par nature impossibles à démonter[15].

4.3 Exigences de la solution

Après avoir identifié le problème, les besoins pour y remédier et les solutions déjà existantes, nous sommes arrivés à la conclusion qu'aucun système existant ne satisfaisait entièrement nos besoins. C'est seulement suite à cette réflexion que j'ai pris la décision de concevoir par moi-même un système correspondant à nos besoins.

[13] « Atlas 2 », *Vakaros*, s.d., https://www.vakaros.com/en-eu/collections/atlas-family/products/atlas2 (consulté le 11.10.2026).
[14] « Get Orca », *Orca*, s.d., https://getorca.com/product-configuration/ (consulté le 11.10.2026).
[15] *Cours, briefing et debriefing*, op. cit., p.9.

Les exigences fixées pour ce dispositif sont les suivantes :

- Abordabilité

Le dispositif ne doit pas être réservé à une élite ou à des équipes compétitives de haut niveau. Chaque club devrait avoir l'opportunité de proposer un moyen de qualité permettant de monter en niveau.

- Adaptabilité aux clubs

Les bateaux dans les clubs ne sont pas uniquement utilisés par l'équipe de compétition. Ils servent aussi à un éventail de cours et d'activité qui ne doit pas être restreint par ce dispositif. De plus, une équipe peut s'entraîner sur différents supports. Le dispositif doit donc être déplaçable facilement pour pouvoir être utilisé sur différents bateaux.

- Open Source

Cela a un rapport avec la volonté d'abordabilité. Le dispositif doit être un moyen de progression, pas un potentiel objet de spéculation. La possibilité de progresser devrait appartenir à l'athlète, pas à une personne tierce susceptible de la monétiser. Le fait de placer ce projet dans le domaine public est la manifestation d'une volonté de démocratisation ainsi qu'une garantie de reproductibilité et de qualité.

- No-nonsense

Le focus est placé sur des résultats clairs et exploitables pour le debrief, compréhensibles pour le coach comme pour les athlètes. La mise en œuvre elle aussi devrait être aisée. Le dispositif doit gagner du temps à l'équipe, pas en coûter.

- Prise en compte multifactorielle

Les mesures ne doivent pas se concentrer sur un aspect unique, mais au contraire prendre en compte de nombreux aspects de la navigation pour en avoir un aperçu global représentatif. On veut notamment au minimum des données sur le vent, sur la vitesse, sur la position géographique du bateau et sur sa situation dans l'espace (accélération linéaire, accélération de rotation et orientation selon les trois axes).

4.4 Licence MIT

Le projet est placé dans le domaine Open Source par l'usage de la licence MIT. Elle est disponible aux côtés de mon travail dans le dépôt GitHub où il est accessible publiquement. Elle est reproduite ci-après.

> MIT License
>
> Copyright (c) 2026 anonyme-sjf-26
>
> Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the «Software»), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:
>
> The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.
>
> THE SOFTWARE IS PROVIDED «AS IS», WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

4.5 Nom du dispositif

Dans *Demian*, de Hermann Hesse, Abraxas est un symbole de l'unification des contraires. Il ne nie pas le principe même des opposés, mais propose de dépasser les contradictions existant entre eux en cessant de les opposer systématiquement dans la réflexion pour arriver à une vision plus holistique d'une situation. En d'autres termes, les opposés ne sont pas seulement indissociables, mais profondément imbriqués. Ainsi, considérer un élément sans celui qui s'y oppose serait un non-sens, et le simple fait de proposer une délimitation entre deux concepts antagonistes relèverait de l'arbitraire[16].

Dans le domaine de la voile compétitive, j'ai fréquemment été confronté en tant que personne à une limite implicite entre la compétition «classique» et la compétition «de haut niveau», souvent partiellement professionnalisée. Si cette distinction m'a tout d'abord semblé rationnelle, je me suis par la suite aperçu qu'elle ne correspondait pas toujours à une différence de niveau, mais davantage à une différence de moyens. Afin de passer à une pratique de la compétition à plus haut niveau, il s'est souvent dans mon expérience agi davantage de relations, de savoir se vendre, et ce parfois au détriment des autres, plutôt que de prouver avoir un niveau réel. Les initiatives dans le domaine de la compétition sont souvent prises en ne considérant qu'un seul de ces groupes, alors qu'ils ne peuvent pas fonctionner l'un sans l'autre. Le haut niveau repose sur un vivier d'athlètes en préparation, et ce vivier ne peut pas s'entretenir sans objectif à atteindre.

[16] *Demian*, op. cit., p.6.

J'ai pour ma part eu la chance de pouvoir intégrer un équipage semi-professionnel (Mercedes-AMG Team Joker 2.0) grâce à l'intérêt porté par le propriétaire de l'équipe envers les jeunes. Malheureusement, ce type d'opportunité est une exception : le passage vers le haut niveau est souvent rendu difficile par la nécessité d'avoir des relations pour entrer dans un équipage. Plus qu'une question de capacités, le haut niveau est un milieu, et, sauf exception, il faut quasiment y être né·e pour en faire partie. Je ne nie pas qu'il existe deux mondes différents, mais je ne reconnais pas comme légitime la façon dont ils sont séparés. À ce titre, je ne souhaite pas réaliser un travail qui les considère de façon étanche.

C'est en référence au refus de la façon dont ces groupes sont déterminés, et donc de considérer l'un indépendamment de l'autre, de fournir une solution qui ne peut s'adresser qu'à un seul d'entre eux, que j'ai nommé mon projet Abraxas. Tant du fait de sa conception que de son prix, il est intéressant pour les compétiteur·trice·s de tous les niveaux.

Demian étant un livre qui m'a fortement marqué, je suis heureux de pouvoir lui dédié un hommage au travers de mon travail. C'est une manière pour moi de conjuguer deux de mes passions : la voile et la littérature.

4.6 Logo du dispositif

Le logo se compose principalement d'un cercle noir traversé à 45° par des capsules blanches de taille variable. Il représente à la fois des gouttes d'eau sur un écran ou un objectif, le défilement de données ou le glissement de bateaux à la surface du lac. Ce cercle est accompagné à droite du mot « Abraxas », écrit en lettres stylisées. Les lettres font exactement la hauteur du cercle. Le « X » reprend le style des capsules dans le cercle noir par sa branche interrompue. Le logo a été vectorisé avec le logiciel VectorCraft. Il se décline en deux variantes : une avec le cercle uniquement, et une avec le nom « Abraxas ».



Fig. 3. – Logo avec le cercle uniquement



Fig. 4. – Logo avec le nom « Abraxas »

5 PREMIER PROTOTYPE

5.1 Matériel

Le choix du matériel a été une étape compliquée. En effet, je n'avais quasiment aucune connaissance en matière d'électronique, et choisir les composants demande de réfléchir à la compatibilité des protocoles, des tensions et des pinouts. De plus, les boutiques en ligne proposent des centaines de composants extrêmement similaires, mais pas interchangeables pour autant. Faire la différence entre eux a parfois été un défi en soi. Tous les composants ne sont pas disponibles chez le même vendeur, ce qui m'a forcé à passer commande auprès de plusieurs entités différentes.

J'ai dû à plusieurs reprises recommander du matériel, parce que je n'avais pas pris le bon ou parce qu'il m'en manquait. Je m'attendais à ce que la principale difficulté soit dans la partie software, mais le choix du hardware s'est avéré particulièrement chronophage.

Commande 1

La première commande comprend le matériel de base, tel que le Raspberry lui-même, l'IMU, le GPS et la caméra. L'anémomètre ne sera commandé qu'ultérieurement.

Type de composant	Lien	Nom du composant
IMU	lien	ICM20948, Breakout Pimoroni
Micro-ordinateur	lien	Raspberry Pi Zero 2 WH
GPS	lien	MTK 3339, Adafruit Ultimate GPS Breakout
Caméra	lien	Original Raspberry Pi Kamera Module V2
Power boost	lien	Adafruit PowerBoost 1000C
Batterie	lien	LiPO 8000 mAh 2C

Commande 2

En passant la première commande, j'ignorais qu'il existait deux normes de câble CSI. Le câble fourni avec la caméra est un CSI 15 contacts, mais l'entrée sur le Raspberry est un mini-CSI 22 contacts. Il a donc fallu racheter une nappe pour raccorder les deux composants. On voit le détail de cette nappe sur la Fig. 12.

Type de composant	Lien	Nom du composant
Nappe CSI	lien	CSI (15 contacts) ↔ Mini-CSI (22 contacts)



Fig. 5. – Photo du matériel de la première commande avant montage.

Commande 3

La troisième commande avait pour but l'ajout de l'anémomètre. On le voit sur la Fig. 14. Cette commande s'est également axée sur la réalisation d'un deuxième prototype utilisable en conditions réelles. À ce titre, l'usage de certains composants mentionnés n'est pas décrit dans le présent document.

Type de composant	Lien	Nom du composant
Anémomètre	lien	HY-WDC2E
Prise USB encastrable	lien	micro-B ↔ C
Prise USB encastrable	lien	A ↔ C
Gestion de l'arrêt	lien	ON/OFF SHIM
USB ↔ RS485	lien	WaveShare
USB ↔ RS485	lien	SparkFun
Connecteur	lien	Mâle
Connecteur	lien	Femelle
Bouton	lien	Métal IP67
Hub USB	lien	Zero4U

Commande 4

Type de composant	Lien	Nom du composant
Antenne GPS active	lien	3-5V 28dB 5 Meter SMA
Connecteur d'antenne	lien	SMA to uFL/u.FL/IPX/IPEX RF
Câble USB	lien	20cm (câble) + B3 (mini-B) + C1 (micro-B)
Fiches micro-B	lien	Dénudées, deux contacts
Convertisseur	lien	USB ↔ RS485

5.2 Montage

Tous les schémas des montages dans cette section ont été créés par mes soins à l'aide du logiciel Fritzing.

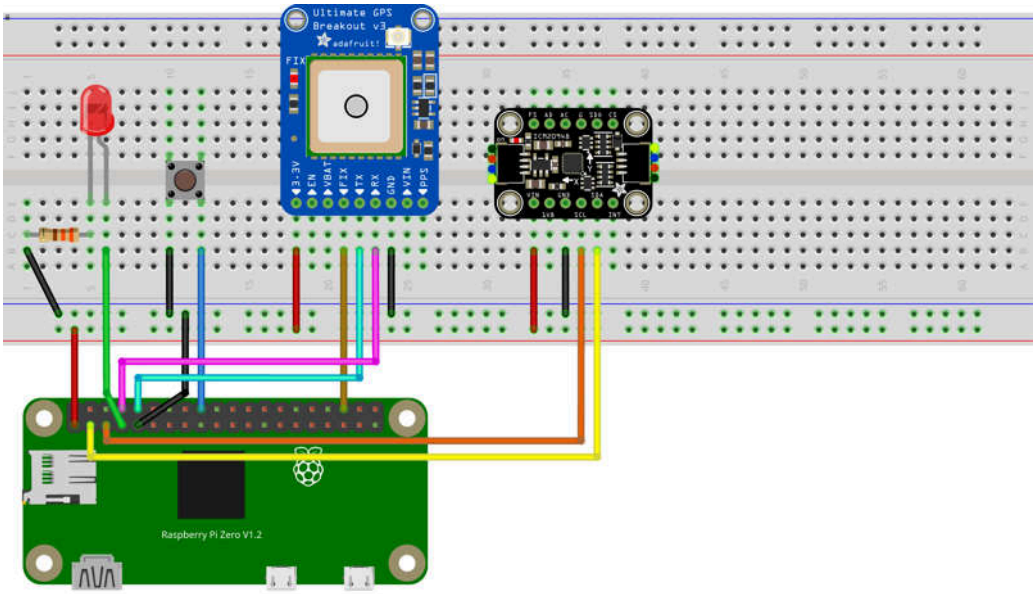


Fig. 6. – Représentation du montage effectué.

Sur cette représentation, on a utilisé pour l'IMU un breakout Adafruit de l'ICM20948, alors que le breakout réellement utilisé est un Pimoroni. Cependant, la puce utilisée et leur pinout sont identiques, la seule différence est donc visuelle. Ce choix est dû au fait que Pimoroni n'a pas créé de représentation graphique de son breakout utilisable dans Fritzing.

Pour procéder au montage, il a fallu souder chaque broche de l'IMU et du GPS, soit 14. Cet exercice de minutie a représenté une difficulté majeure, car les broches sont petites et extrêmement rapprochées les unes des autres (pas de 2.54mm), or, les soudures ne doivent en aucun cas se toucher. De plus, le fer à souder ne doit pas rester proche du composant trop longtemps, sous peine de faire fondre le plastique du support.

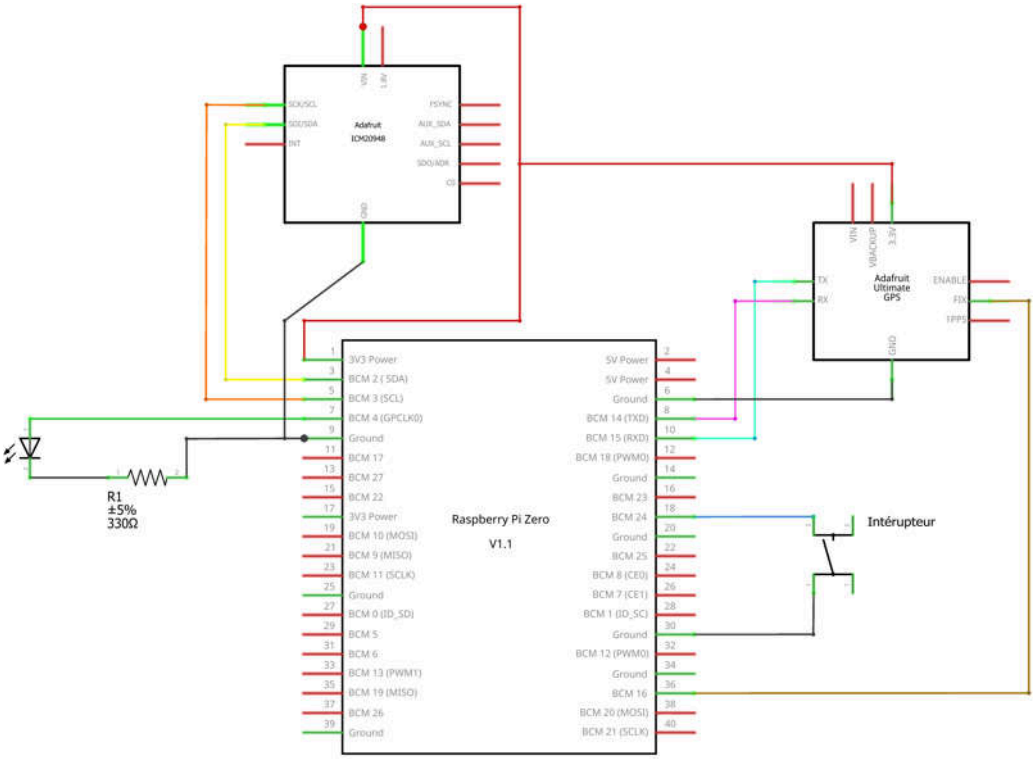


Fig. 7. – Représentation schématique du montage effectué.

5.3 Installation de l'OS

L'OS installé est *Raspberry Pi OS Lite (64-bit)*. Dépourvu d'interface graphique, il a été choisi en raison du faible espace disque utilisé, de sa simplicité d'utilisation et d'installation, de la grande documentation le concernant, mais surtout pour la faible charge processeur engendrée. En effet, la gestion des ressources, limitées sur un *Raspberry Pi Zero 2WH*, est un défi majeur de ce projet.

Pour son installation, le logiciel *Raspberry Pi Imager* a été utilisé. On télécharge via ce logiciel l'OS sur son ordinateur, avant de le flasher sur la carte SD qui servira de disque au Raspberry. Lors de l'installation, on configure le compte utilisateur, le mot de passe utilisateur, le mot de passe *root*, le WiFi (SSID et mot de passe), ainsi que l'usage du *SSH* avec authentification par chiffrement asymétrique (détaillé au Chapitre 5.4). Ainsi, une fois la carte SD insérée dans le Raspberry, tout est immédiatement fonctionnel, il n'est même pas nécessaire de le brancher à un écran ou à un clavier pour l'utiliser.

5.4 SSH par clés publiques

SSH est l'abréviation de *Secure Shell*. Ce protocole permet d'accéder à la console du Raspberry depuis un autre ordinateur avec la syntaxe suivante :

```
batch
ssh user@ip_raspberry
```

Avec `user` le nom d'utilisateur et `ip_raspberry` l'adresse IP du Raspberry (ici `192.168.1.13`). Par défaut, quand une demande de connexion entrante est détectée, le Raspberry demande le mot de passe du compte `user` pour autoriser l'accès à la console. Cependant cette méthode d'authentification n'est pas la plus sécurisée, car un mot de passe peut être cassé par force brute. Pour cette raison, il a été choisi d'utiliser le chiffrement asymétrique.

Si l'on choisit d'utiliser SSH pour l'accès à distance au lieu de systèmes de plus haut niveau plus instinctifs, c'est pour son faible usage de ressources, sa large compatibilité, sa sécurité optimale et sa stabilité éprouvée, justement car c'est un processus de bas niveau. En effet, ces processus sont bien plus permissifs et configurables dans le cas de projet de développement, et ont l'avantage de ne nécessiter aucun téléchargement supplémentaire[17].

Chiffrement asymétrique

Cette méthode peut être activée après l'installation de l'OS, mais pour des raisons de simplicité, elle a été configurée directement avec *Raspberry Pi Imager*. Pour se connecter en *SSH* de cette manière, il est nécessaire d'avoir une clé publique et une clé privée sur chaque appareil. La génération de ces clés se fait en utilisant

```
batch
ssh-keygen
```

sur l'ordinateur depuis lequel on veut accéder au Raspberry. Cette commande fonctionne aussi bien sur Linux, sur MacOS que sur Windows. Dans les trois cas, on suit les indications données par la console / le terminal sans renseigner plus d'options pour garder la configuration par défaut. Au terme de cette opération, on obtient une clé publique et une clé privée, stockées respectivement dans `.ssh/id_rsa.pub` et dans `.ssh/id_rsa`. Lors de la configuration de l'OS avec *Raspberry Pi Imager*, on renseigne la clé publique. La création de clés sur le Raspberry se fait automatiquement. Ainsi, dès le premier *boot* du Raspberry, on peut y accéder de façon distante sans renseigner de mot de passe, à condition d'être connecté au même réseau.

NAT-Loopback et Port mapping

Afin d'accéder au Raspberry depuis l'extérieur du réseau auquel il est connecté, on doit réaliser une opération sur la box (souvent `192.168.1.1`). Le port par défaut du SSH est le 22. Par conséquent, l'objectif est d'ouvrir le port 22 de la box et de rediriger les informations arrivant dessus vers le Raspberry. La marche à suivre dépend de la box, mais il est généralement possible d'y accéder depuis un navigateur en tapant `192.168.1.1` dans la barre de recherche.

[17] FAULKNER Andrew et BLANC Florian, « Avantages et faiblesses du protocole SSH », *Hakin9*, n°2, 2008, https://chk.me/qm9L8t2 (consulté le 21.10.2026).

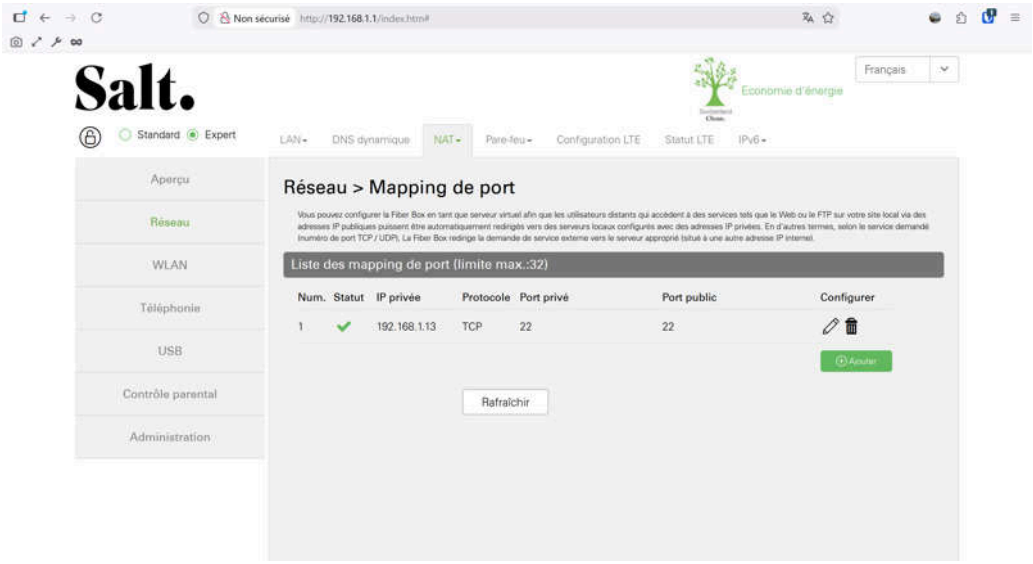


Fig. 8. – Page de configuration de mapping de port sur une box Salt.

Pour accéder au Raspberry depuis l'extérieur une fois le mapping de port réglé, il suffit d'exécuter

```
batch
ssh user@ip_publique_box
```

L'`ip_publique_box` est l'adresse IP du réseau visible depuis l'extérieur. On peut la trouver facilement grâce à des sites tels que https://www.myip.com/.

Ensuite, il faut autoriser le NAT-Loopback afin de permettre à l'ordinateur de se connecter au Raspberry depuis le même réseau, alors même que la porte d'entrée privilégiée est tournée vers l'extérieur. Le NAT-Loopback permet à la box d'interpréter des requêtes venant du réseau qu'elle génère comme des requêtes venant de l'extérieur. C'est-à-dire qu'une demande de connexion adressée à son IP publique faite depuis l'intérieur du réseau ne pourra pas être interprétée correctement sans NAT-Loopback.

Changement de clé publique du Raspberry

Il arrive parfois que cette clé change automatiquement. La nouvelle clé différera donc de celle enregistrée dans `.ssh/known_hosts`. Cette différence lève l'inquiétante erreur suivante :

```
bash
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@     WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!     @
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
IT IS POSSIBLE THAT SOMEONE IS DOING SOMETHING NASTY!
Someone could be eavesdropping on you right now (man-in-the-middle attack)!
It is also possible that a host key has just been changed.
The fingerprint for the rsa key sent by the remote host is
SHA256:7QeljpfXjWS1Q6INCVhjp1Hwn+MIA23TcOcgTKcIgVt4.
```

```
Please contact your system administrator.
Add correct host key in C:\\Users\\user/.ssh/known_hosts to get rid of this message.
Offending ECDSA key in C:\\Users\\user/.ssh/known_hosts:77
Host key for 192.168.1.13 has changed and you have requested strict checking.
Host key verification failed.
```

Pour se débarrasser de cette erreur, il convient d'exécuter :

```
batch
ssh-keygen -R 192.168.1.13
```

Cette commande supprime toutes les clés publiques de `.ssh/known_hosts` relatives à cette adresse IP. Cela permet une nouvelle connexion qu'il faudra valider en tapant `y` quand on y est appelé. ATTENTION ! N'exécuter cette commande que si l'on est certain que personne n'est en train de conduire un MITM, ce serait lui ouvrir la porte du réseau.

5.5 Transfert de fichiers

Au cours de cette démarche, il sera à plusieurs reprises nécessaire de transférer du code python sur le Raspberry. Pour ce faire, il existe plusieurs méthodes, nous allons en présenter deux :

- Avec SCP :

On enregistre sur l'ordinateur distant le fichier python. On le copie ensuite sur le Raspberry depuis l'ordinateur distant en utilisant SCP, un protocole de SSH, avec la syntaxe suivante :

```
batch
scp  chemin\absolu\de\départ\script.py  user@192.168.1.13:/home/user/destination/
script.py
```

Attention à bien travailler en chemin absolu et non en chemin relatif. Sur Windows, les « / » dans les chemins d'accès sont retournés.

- Avec `nano` :

Directement sur le Raspberry, on exécute :

```
bash
cd /home/user/destination
nano script.py
```

Puis on colle le code dans l'éditeur de texte qui s'ouvre. Un tel copié-collé n'est possible que si l'on est connecté via SSH au Raspberry, si ce n'est pas le cas, mieux vaut utiliser SCP. Pour enregistrer, on fait `Ctrl+X`, puis `y` pour valider, et `Enter` pour sortir de `nano`.

Dans la pratique

Dans le cadre de ce TM, j'ai créé un script `batch` utilisant SCP pour directement transférer tous les fichiers du software (présentés dans le Chapitre 6). Ainsi, à chaque modification, il me suffisait de l'exécuter pour actualiser automatiquement chaque fichier sur mon Raspberry.

```
batch
scp -r "C:\Users\user\Documents\GitHub\Abraxas\*.py"
        user@51.154.XXX.XXX:/home/user
```

5.6 Paquets et librairies

On commence par faire les mises à jour standards et par installer `tree`, `pip` et `python3-full` :

```
bash
sudo apt-get update
sudo apt-get -y upgrade
sudo apt install tree
sudo apt install python3-pip
sudo apt install python3-full
```

Tous les autres paquets qui pourraient être nécessaires ultérieurement pourront simplement être installés de la même manière.

Pour installer des librairies python, on utilise `pip`. On doit par exemple installer `time` :

```
bash
pip3 install time
```

Dans le cas où `pip` renverrait l'erreur suivante :

```
bash
error: externally-managed-environment

× This environment is externally managed
╰─> To install Python packages system-wide, try apt install
    python3-xyz, where xyz is the package you are trying to
    install.

    If you wish to install a non-Debian-packaged Python package,
    create a virtual environment using python3 -m venv path/to/venv.
    Then use path/to/venv/bin/python and path/to/venv/bin/pip. Make
    sure you have python3-full installed.

    For more information visit http://rptl.io/venv
```

```
note: If you believe this is a mistake, please contact your Python installation
or OS distribution provider. You can override this, at the risk of breaking your
Python installation or OS, by passing --break-system-packages.
hint: See PEP 668 for the detailed specification.
```

On peut ajouter une option pour forcer l'installation à la fin de la ligne :

```bash
pip3 install time --break-system-packages
```

5.7 Test des GPIO

Pour réaliser un premier essai, on souhaite faire clignoter une LED connectée au GPIO 7, soit le BCM 4. On crée le fichier `blink.py` avec le code suivant sur le Raspberry (par exemple dans `/home/user`) :

```python
import RPi.GPIO as GPIO
import time

LED_PIN = 4  # GPIO 7 = BCM 4

GPIO.setmode(GPIO.BCM)
GPIO.setup(LED_PIN, GPIO.OUT)

try:
    while True:
        GPIO.output(LED_PIN, GPIO.HIGH)
        time.sleep(0.5)
        GPIO.output(LED_PIN, GPIO.LOW)
        time.sleep(0.5)

except KeyboardInterrupt:
    pass

finally:
    GPIO.cleanup()
```

On installe ensuite la librairie `RPi.GPIO` sur le Raspberry :

```bash
pip3 install RPi.GPIO
```

On lance finalement le script sur le Raspberry avec :

```bash
python3 blink.py
```

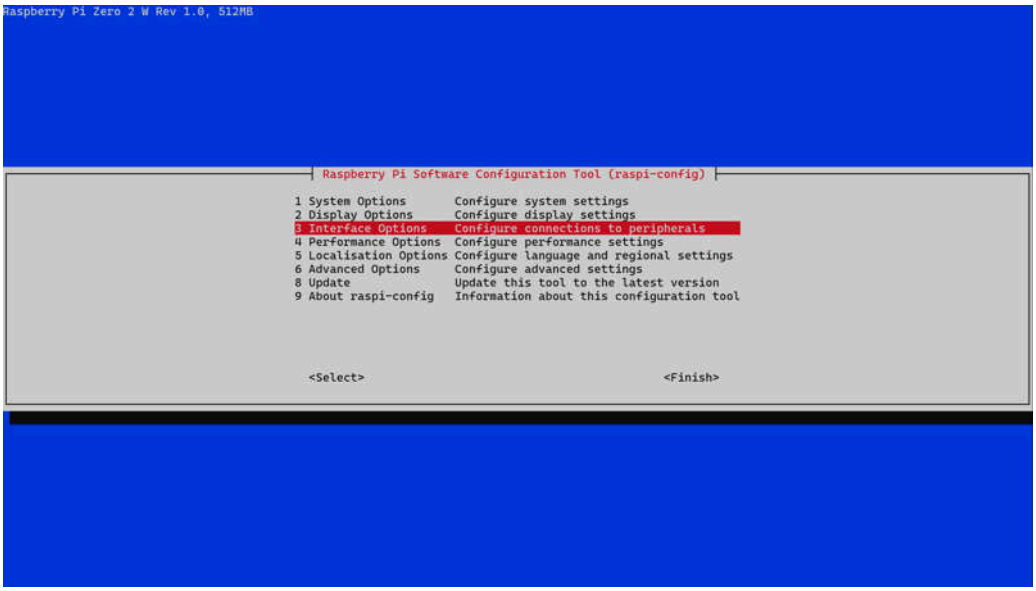
On observe la LED clignoter avec une période d'une demi-seconde, on peut l'arrêter avec un `KeyboardInterrupt` (`Ctrl+C`).

5.8 Réglage de Serial Port

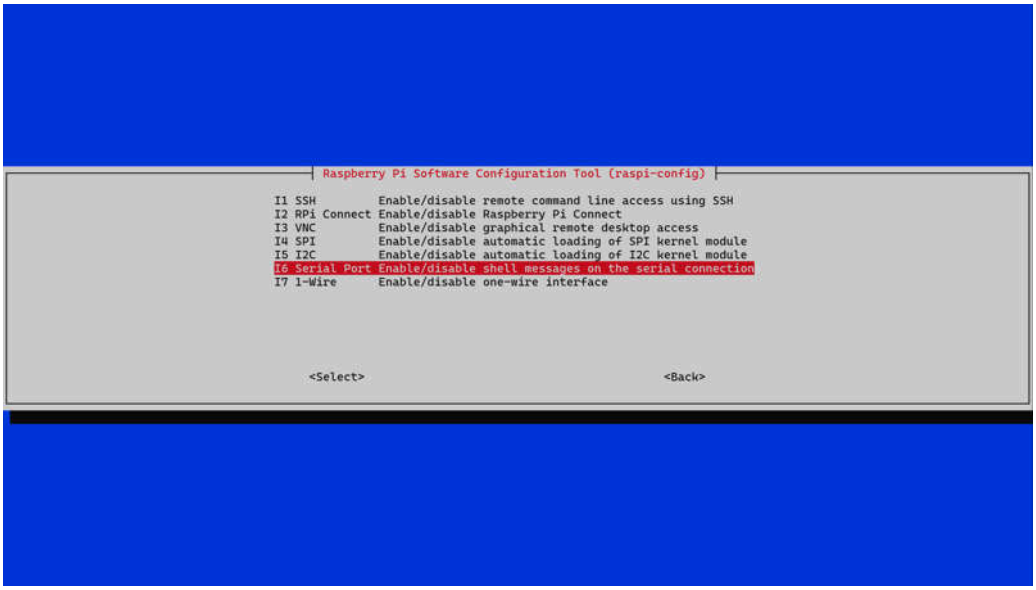
Pour permettre au port UART (TxD et RxD) de fonctionner correctement, on doit procéder à un réglage du port série : on veut désactiver le `serial login shell` et activer le `serial interface`. On exécute :

```bash
sudo raspi-config
```

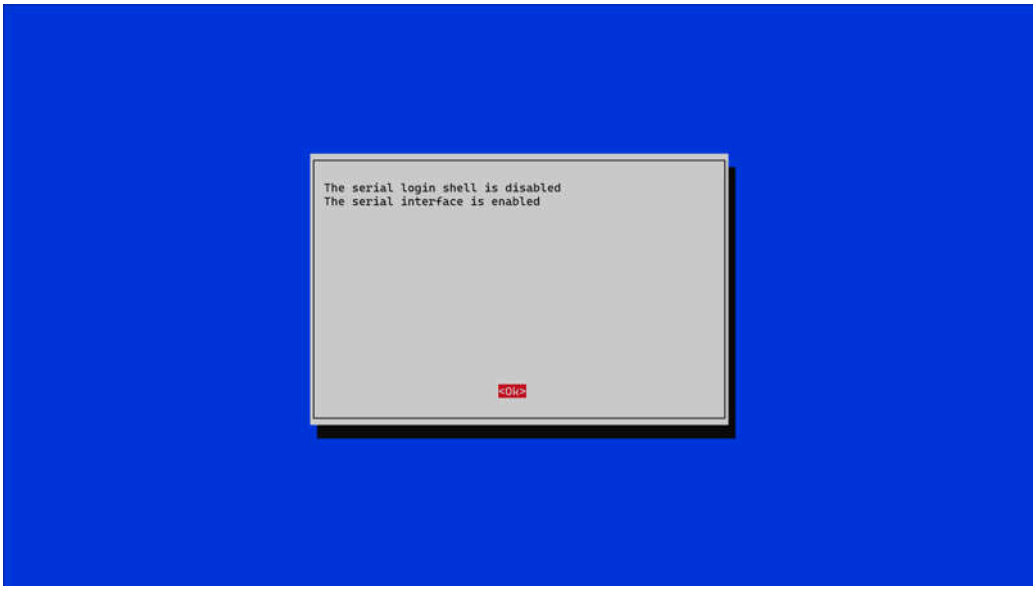
Ensuite, on sélectionne Interface Option



Puis Serial Port



On choisit `no` pour le `serial login shell`, `yes` pour le `serial interface`. On devrait avoir une fenêtre de ce type :



On valide, puis on entre `finish`, et afin que les changements prennent effet (si le reboot n'est pas déjà proposé à la sortie de `raspi-config`) :

```bash
sudo reboot
```

Le Raspberry va redémarrer, la connexion SSH va se clore, mais il sera possible de se reconnecter après quelques minutes.

5.9 Usage de l'IMU

L'IMU 9DoF fonctionne en bus I²C, soit avec les ports SDA (informations) et SCL (horloge)[18]. Pour l'initialiser, on installe les bibliothèques, les paquets et les scripts du repository GitHub[19]. Ensuite, on exécute le script d'initialisation `install.sh`. Pour procéder ainsi, on a besoin d'avoir `git` installé :

```bash
sudo apt install git
```

On exécute ensuite les commandes suivantes :

```bash
cd /home/user
git clone https://github.com/pimoroni/icm20948-python
cd /home/user/icm20948-python
./install.sh
```

Au cours de l'installation, il nous est demandé si l'on souhaite télécharger des exemples. Cela peut être utile pour tester le bon fonctionnement de l'IMU. On valide avec `y`. On refuse par contre la documentation avec `n`, on ne veut pas en encombrer le Raspberry, et elle est accessible sur le repository. Il nous reste encore à installer la librairie `icm20948` depuis PyPi :

```bash
pip3 install icm20948
```

On teste l'IMU en exécutant le script d'exemple `read-all.py` :

```bash
python3 /home/user/icm20948-python/examples/read-all.py
```

On devrait voir défiler des blocs d'information de ce type :

```python
Accel: -0.16 00.00 01.04
Gyro:  -1.00 01.84 -0.75
Mag:   29.85 -41.25 128.85
```

Pour l'arrêter, on fait un `KeyboardInterrupt` (`Ctrl+C`).

[18] HOWARD Philip *et al.*, « icm20948-python », *GitHub*, 28.06.2024 (dernier commit), https://github.com/pimoroni/icm20948-python (consulté le 21.10.2026).
[19] Id.

5.10 Usage du GPS

Avant de commencer l'initialisation du GPS, s'assurer que le réglage de Serial Port (Chapitre 5.8) a bien été effectué. Le GPS fonctionne en UART asynchrone, c'est-à-dire avec les ports RxD et TxD[20]. Contrairement au bus I²C, il ne s'agit pas d'une architecture maître/esclave, mais d'un processus point to point, permettant des interactions plus complexes entre le Raspberry et le GPS.

Pour l'initialisation du GPS, on s'est appuyé sur les guides suivants :

- Mode d'emploi du GPS (PDF)[21]
- Exécution du code CircuitPython sans interpréteur CircuitPython[22]
- Installer Blinka sur un Raspberry Pi[23]

Une partie des bibliothèques, des paquets et des scripts de test viennent du dépôt du GPS[24]. La démarche d'installation exacte a dû être adaptée aux besoins spécifiques du projet. On la retranscrit donc entièrement ici.

Le *Adafruit Ultimate GPS Breakout* fonctionne selon une logique CircuitPython, une version allégée de Python. Pour pouvoir le faire tourner sur un Raspberry (nativement dépourvu de CircuitPython), il faut installer un interpréteur : Blinka.

Installation de Blinka

On commence par upgrader les `setuptools` :

```
bash
sudo apt install --upgrade python3-setuptools
```

À ce moment-là, le Raspberry demandera peut-être un reboot. On l'effectue si nécessaire, puis on installe un environnement virtuel pour y faire tourner Blinka.

```
bash
cd
sudo apt install python3-venv
python3 -m venv env --system-site-packages
```

[20] HERRADA Eva *et al.*, « Adafruit_CircuitPython_GPS », *GitHub*, 09.09.2026 (dernier commit), https://github.com/adafruit/Adafruit_CircuitPython_GPS (consulté le 21.10.2026).

[21] HERRADA Eva, « Adafruit Ultimate GPS », *Adafruit*, 20.08.2026 (dernière mise à jour), https://cdn-learn.adafruit.com/downloads/pdf/adafruit-ultimate-gps.pdf (consulté le 21.10.2026).

[22] HERRADA Eva et LEBLANC Williams, « Running CircuitPython Code without CircuitPython », *Adafruit*, 05.07.2023 (dernière mise à jour majeure), https://chk.me/O2xfxk0 (consulté le 21.10.2026).

[23] HERRADA Eva et LEBLANC Williams, « Installing Blinka on Raspberry Pi », *Adafruit*, 05.07.2023 (dernière mise à jour majeure), https://chk.me/Y40lkS7 (consulté le 21.10.2026).

[24] « Adafruit_CircuitPython_GPS », op. cit., p.33.

Puis on l'active :

```bash
source env/bin/activate
```

On procède ensuite à l'installation automatique de Blinka à l'aide d'un script fourni par Adafruit :

```bash
cd
pip3 install --upgrade adafruit-python-shell
wget https://raw.githubusercontent.com/adafruit/Raspberry-Pi-Installer-Scripts/master/raspi-blinka.py
sudo -E env PATH=$PATH python3 raspi-blinka.py
```

Le processus va demander une validation, on y procède en cliquant `y`. L'installation prend quelques minutes, au terme desquelles le Raspberry demande un reboot. On l'accepte en cliquant encore `y`.

On crée le fichier `blinkatest.py` (par exemple dans `/home/user`) contenant le code suivant :

```python
import board
import digitalio
import busio

print("Hello, blinka!")

# Try to create a Digital input
pin = digitalio.DigitalInOut(board.D4)
print("Digital IO ok!")

# Try to create an I2C device
i2c = busio.I2C(board.SCL, board.SDA)
print("I2C ok!")

# Try to create an SPI device
spi = busio.SPI(board.SCLK, board.MOSI, board.MISO)
print("SPI ok!")

print("done!")
```

On lance ensuite le script python pour tester Blinka :

```bash
source env/bin/activate
python3 blinkatest.py
```

On devrait avoir :

```bash
Hello, blinka!
Digital IO ok!
I2C ok!
SPI ok!
done!
```

Précaution particulière

CircuitPython contient une librairie nommée `board`, or Python possède une librairie du même nom. Tenter d'installer la librairie Python sur tout le système reviendrait à casser Blinka de façon irrémédiable, et il faudrait flasher à nouveau un OS vierge sur la carte SD pour recommencer toute l'installation. Je l'ai appris à mes dépens. Si l'erreur suivante est levée :

```bash
Traceback (most recent call last):
  File "/home/user/blinkatest.py", line 1, in <module>
    import board
ModuleNotFoundError: No module named 'board'
```

Ne surtout pas tenter de la résoudre en important `board`. Le problème n'est pas que cette bibliothèque n'est pas importée : elle est bien présente, mais uniquement en CircuitPython dans le `venv` de Blinka. Pour se débarrasser de cette erreur, il faut vérifier que l'on se trouve bien dans l'environnement virtuel voulu, et au besoin s'y rendre à nouveau avec

```bash
source env/bin/activate
```

Si on a ultérieurement besoin de `board` de Python (bien qu'il soit préférable de trouver un autre moyen), il faut veiller à circonscrire son installation à un environnement virtuel séparé de Blinka.

Installations de la bibliothèque relative au GPS

On doit d'abord installer `adafruit-circuitpython-gps` (des `DEPRECATION` pourraient être levées, mais elles sont sans gravité) :

```bash
sudo pip3 install adafruit-circuitpython-gps --break-system-packages
                                             --root-user-action=ignore
```

L'option `--root-user-action` avec l'attribut `ignore` permet d'éviter l'erreur suivante quand on exécute `pip` en `sudo` :

```bash
WARNING: Running pip as the 'root' user can result in broken permissions and
conflicting behaviour with the system package manager, possibly rendering your
system unusable. It is recommended to use a virtual environment instead:
https://pip.pypa.io/warnings/venv. Use the --root-user-action option if you know
what you are doing and want to suppress this warning.
```

Test du GPS

On crée le fichier `GPS-test.py` (par exemple dans `/home/user`) avec le code suivant[25] :

```python
# SPDX-FileCopyrightText: 2021 ladyada for Adafruit Industries
# SPDX-License-Identifier: MIT
# Will wait for a fix and print a message every second with the current location
and other details.
import time
import board
import busio
import adafruit_gps
import serial

# Create a serial connection for the GPS connection using default speed
rx = board.RX
tx = board.TX
uart = serial.Serial("/dev/serial0", baudrate=9600, timeout=10)

# Create a GPS module instance.
gps = adafruit_gps.GPS(uart, debug=False)

# Turn on the basic GGA and RMC info
gps.send_command(b"PMTK314,0,1,0,1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0")

# Set update rate to once a second (1hz)
gps.send_command(b"PMTK220,1000")
```

[25] Version modifiée de /exemples/gps_simpletest.py dans Adafruit_CircuitPython_GPS, op. cit., p.33.

```
# Main loop runs forever printing the location, etc. every second.
last_print = time.monotonic()
while True:
    gps.update()
    # Every second print out current location details if there's a fix.
    current = time.monotonic()
    if current - last_print >= 1.0:
        last_print = current
        if not gps.has_fix:
            # Try again if we don't have a fix yet.
            print("Waiting for fix...")
            continue
        # If we have a fix (gps.has_fix is true)
        print("=" * 40)
        print(
            "Fix timestamp: {}/{}/{} {:02}:{:02}:{:02}".format( # noqa: UP032
            gps.timestamp_utc.tm_mon,
            gps.timestamp_utc.tm_mday,
            gps.timestamp_utc.tm_year,
            gps.timestamp_utc.tm_hour,
            gps.timestamp_utc.tm_min,
            gps.timestamp_utc.tm_sec,
            )
          )
# gps.send_command(b'PMTK220,500')
print(f"Latitude: {gps.latitude:.6f} degrees")
        print(f"Longitude: {gps.longitude:.6f} degrees")
                      print(f"Precise   Latitude:   {gps.latitude_degrees}   degs,
{gps.latitude_minutes:2.4f} mins")
                    print(f"Precise   Longitude:   {gps.longitude_degrees}   degs,
{gps.longitude_minutes:2.4f} mins")
        print(f"Fix quality: {gps.fix_quality}")
        if gps.satellites is not None:
            print(f"# satellites: {gps.satellites}")
        if gps.altitude_m is not None:
            print(f"Altitude: {gps.altitude_m} meters")
        if gps.speed_knots is not None:
            print(f"Speed: {gps.speed_knots} knots")
        if gps.speed_kmh is not None:
            print(f"Speed: {gps.speed_kmh} km/h")
        if gps.track_angle_deg is not None:
            print(f"Track angle: {gps.track_angle_deg} degrees")
        if gps.horizontal_dilution is not None:
            print(f"Horizontal dilution: {gps.horizontal_dilution}")
        if gps.height_geoid is not None:
            print(f"Height geoid: {gps.height_geoid} meters")
```

Pour le lancer :

```bash
python3 GPS-test.py
```

Pour que le GPS acquière un premier fix, il peut être nécessaire de le laisser pendant plusieurs minutes (voire heures) à proximité d'une fenêtre ou à l'extérieur, antenne pointée vers le ciel. Le temps nécessaire varie en fonction de l'environnement physique, de la météo, et de la présence de facteurs perturbateurs tels qu'une grande densité d'ondes. Une fois le GPS connecté, on devrait voir défiler des blocs d'information de ce type :

```python
Fix timestamp: 2/9/2026 20:26:21
Latitude: 46.188553 degrees
Longitude: 6.145146 degrees
Precise Latitude: 46 degs, 11.3132 mins
Precise Longitude: 6 degs, 8.7088 mins
Fix quality: 1
# satellites: 4
Altitude: 107.4 meters
Speed: 0.93 knots
Speed: 1.79 km/h
Track angle: 355.59 degrees
Horizontal dilution: 3.43
Height geoid: 48.2 meters
```

La version publiée par Adafruit sur GitHub[26] depuis laquelle ce code a été adapté était originellement prévue pour des microcontrôleurs avec CircuitPython nativement installé. Le fait que le code soit interprété par Blinka a nécessité quelques ajustements pour créer une correspondance au niveau du pinout.

Ces ajustements ont été une difficulté importante, car la question des environnements virtuels, de la différence entre CircuitPython et Python, des protocoles de communication et des ports physiques est très nouvelle pour moi. Or, ces modifications regroupaient toutes ces problématiques. De plus, si Blinka et CircuitPython font l'objet d'une documentation de qualité, la question de la correspondance au niveau physique reste plus floue, et les informations disponibles, de sources amatrices, sont souvent peu claires, obsolètes, ou simplement fausses.

5.11 Usage de la caméra

L'initialisation de la caméra est une étape que j'avais mal anticipée. Pour la connecter au Raspberry, j'avais prévu un câble CSI classique, mais il s'est avéré que la nappe de connexion CSI du *Raspberry Pi Zero 2* est particulière. Elle ne possède pas 15 contacts (comme la majorité des CSI), mais 22. Il a donc fallu se procurer un nouveau câble spéci-

[26] Id.

fique. En ayant poussé mes recherches au moment du prototypage initial, j'aurais pu éviter cela.

Une fois le matériel approprié à disposition, l'installation physique est facile. Aucune soudure n'est nécessaire. Il suffit d'ouvrir les ports CSI de la caméra et du Raspberry en tirant délicatement sur le loquet noir, puis d'insérer la nappe dans le port. Les contacts de la nappe doivent faire face à ceux du port. On referme ensuite les loquets, et le câble est verrouillé.

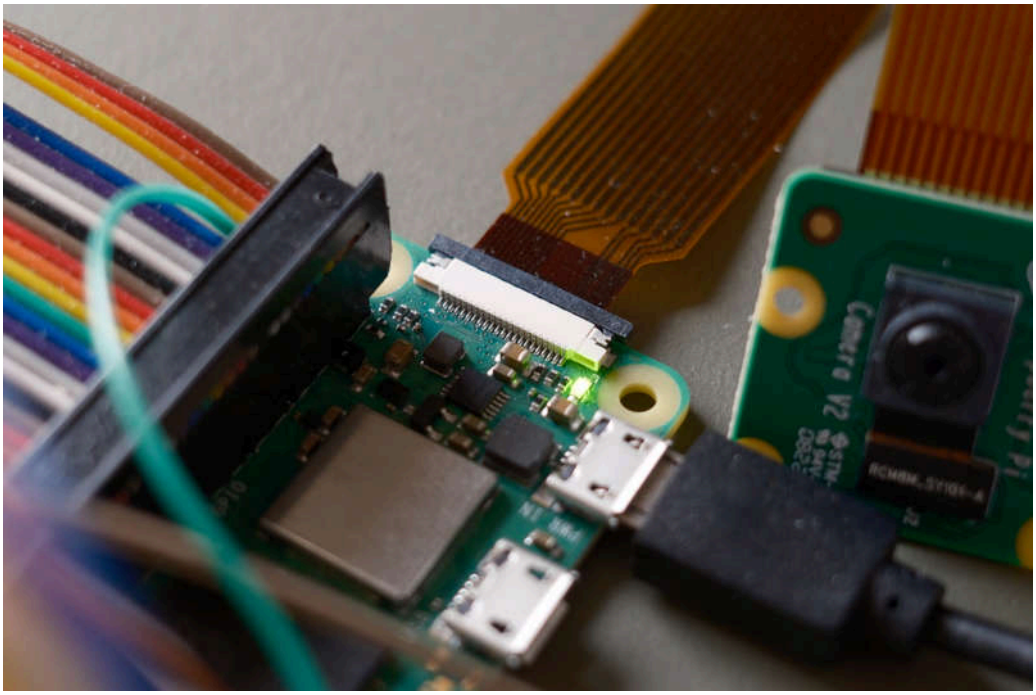


Fig. 12. – La nappe CSI dorée entre dans le port sur le Raspberry

Une fois le raccordement effectué, aucun ajustement logiciel supplémentaire n'est nécessaire. On peut directement utiliser la caméra. On la teste avec :

```bash
rpicam-still -o test.jpg --nopreview
```

Après un délai de cinq secondes durant lequel l'image serait affichée si on avait une interface graphique, l'image est capturée et enregistrée dans le répertoire courant en tant que `test.jpg`. On la supprime pour ne pas encombrer le stockage.

```bash
rm test.jpg
```

5.12 Usage de l'anémomètre

Choix de l'anémomètre

Avant de choisir l'anémomètre, j'ai dû me renseigner sur les différents types d'appareils disponibles. Les plus courants sont les anémomètres à coupelles, dits de Robinson[27]. Ils sont composés de trois coupelles montées sur un axe rotatif central. Leur fonctionnement repose sur une extrapolation de la mesure de la vitesse de rotation des coupelles pour déterminer la vitesse du vent. Afin d'avoir une mesure concernant la direction du vent, il est nécessaire d'y ajouter une girouette[28]. Ce système de capteurs a l'avantage d'être largement disponible dans le commerce et bon marché.

Cependant, il est composé de plusieurs pièces mobiles, ce qui le rend plus complexe à installer sur un bateau et moins durable dans le temps[29]. De plus, du fait de l'inertie des pièces qui le composent, il n'est pas capable d'enregistrer des pics brefs dans les valeurs : la résolution des valeurs mesurées est basse par conception.



Fig. 13. – « Anémomètre à coupelles », *Wikipedia*[30].

[27] « Anémomètre » in *Wikipedia*, s.d., https://chk.me/XaOHDIC (consulté le 21.10.2026).
[28] Id.
[29] Id.
[30] Id.

Pour remédier à ces deux problèmes, j'ai choisi un anémomètre à ultrasons. Il mesure la vitesse de déplacement de la masse d'air dans laquelle il se trouve, selon deux axes, grâce à l'analyse du décalage entre la durée de l'aller et celle du retour du son. Ce type d'anémomètre a l'avantage d'être dépourvu de pièces mobiles et de fonctionner même en cas de vent turbulent, grâce à la haute résolution que sa conception permet[31].

Il est cependant moins répandu en dehors des applications professionnelles ou industrielles, et est donc plus compliqué à trouver. De fait, il est également plus cher. C'est malgré tout vers ce type d'anémomètre que mon choix s'est porté.

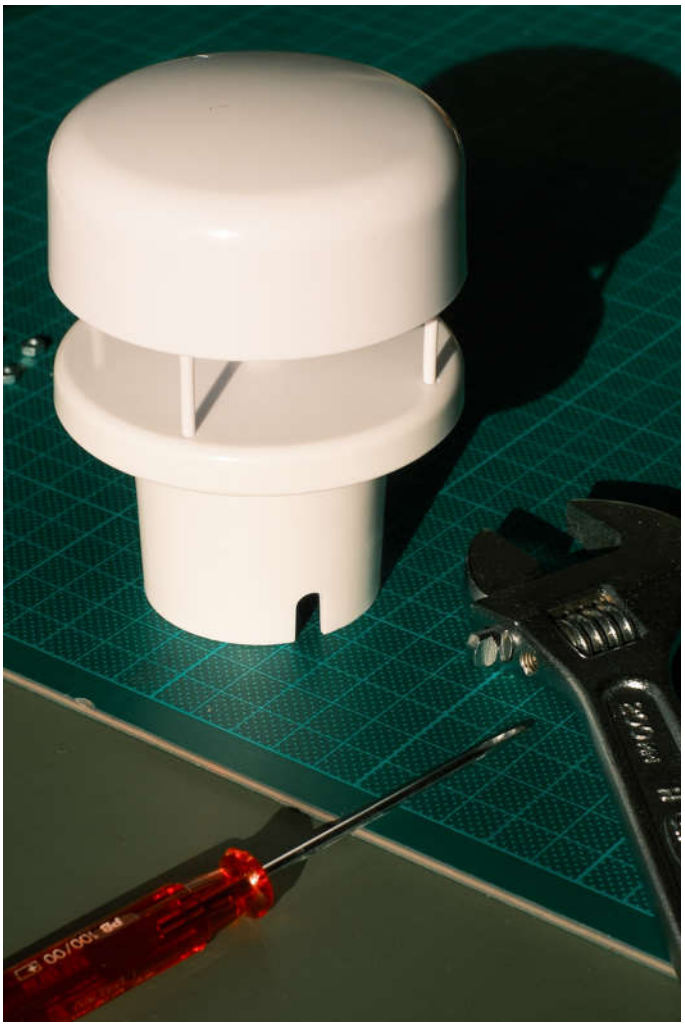


Fig. 14. – Anémomètre à ultrasons HY-WDC2E

Raccordement au prototype

Étant donné que ce type d'anémomètre est plus répandu dans le domaine industriel, les interfaces de communication utilisées sont différentes de celles employées dans le cadre de petits projets. L'anémomètre que j'ai choisi peut communiquer soit en RS232, soit en RS485, deux interfaces fonctionnant avec des niveaux de tension de 12V. Elles sont incompatibles avec le GPIO du Raspberry et des autres cartes de développement, qui fonctionnent en logique 3,3V. Les brancher directement provoquerait la destruction

[31] Id.

immédiate de la carte. Je l'ai appris à mes dépens : peu avant la rentrée, j'ai perdu mon Raspberry en y connectant directement l'anémomètre. J'ai dû en commander un nouveau et recommencer tout le processus de configuration.

Pour rendre la communication possible, j'utilise un module de conversion RS485 vers USB, un breakout SparkFun. Les fils de l'anémomètre sont soudés sur le breakout, puis le breakout lui-même est relié au Raspberry par un câble USB. Une fois les fiches techniques lues correctement, l'installation est aisée.

Les informations sont transmises par trames NMEA 0183[32], le même protocole qu'utilise le GPS. L'anémomètre n'a pas besoin de configuration particulière : il envoie des sorties non sollicitées sur un port USB ouvert (`ttyUSB0` dans le cadre de mon projet). Pour le tester, on récupère simplement les entrées de ce port :

```bash
cat /dev/ttyUSB0
```

On devrait voir apparaître des sorties selon le modèle suivant :

```bash
$WIMWV,180,R,5.12,M,A*06<CR><LF>
```

Le 180 correspond à la direction du vent en degrés, et le 5.12 à sa vitesse en $m \cdot s^{-1}$. Le reste de la trame contient les paramètres (ici ceux d'usine) et la somme de contrôle. Pour utiliser ces données dans un code Python, il ne reste qu'à les parser.

5.13 Base de données SQL

Pour l'enregistrement de données de l'IMU et du GPS, on choisit une base de données SQLite. Elle permet de sauver rapidement et simplement les informations depuis python. De plus, une telle base de données a l'avantage d'être très robuste tout en forçant une organisation entre les données (ce que ne ferait pas un json par exemple)[33].

On définit trois tables : `sessions`, `imu_data` et `gps_data`. Chaque table possède une clé primaire `id` en plus des autres champs. De plus, les tables `imu_data` et `gps_data` ont une clé étrangère permettant de relier chacun de leurs enregistrements à une session (enregistrement de la table `sessions`). Ainsi, il sera facile de séparer les mesures des différents entraînements.

`gps_data` et `imu_data` sont séparées, car leurs enregistrements sont asynchrones. En effet, on ne peut pas enregistrer les données de l'IMU et du GPS dans une même table si les informations que l'on reçoit d'eux n'arrivent pas en même temps.

[32] « User Manual », *Hongyuv*, 30.04.2024, https://www.hongyuv.com/download/WDC2E_Manual.pdf (consulté le 21.10.2026).
[33] EYGAY Erwan, « JSON ou SQL : Quel format choisir pour gérer vos données ? », *DataBird*, 19.03.2026 (dernière mise à jour), www.data-bird.co/blog/json-ou-sql (consulté le 21.10.2026).

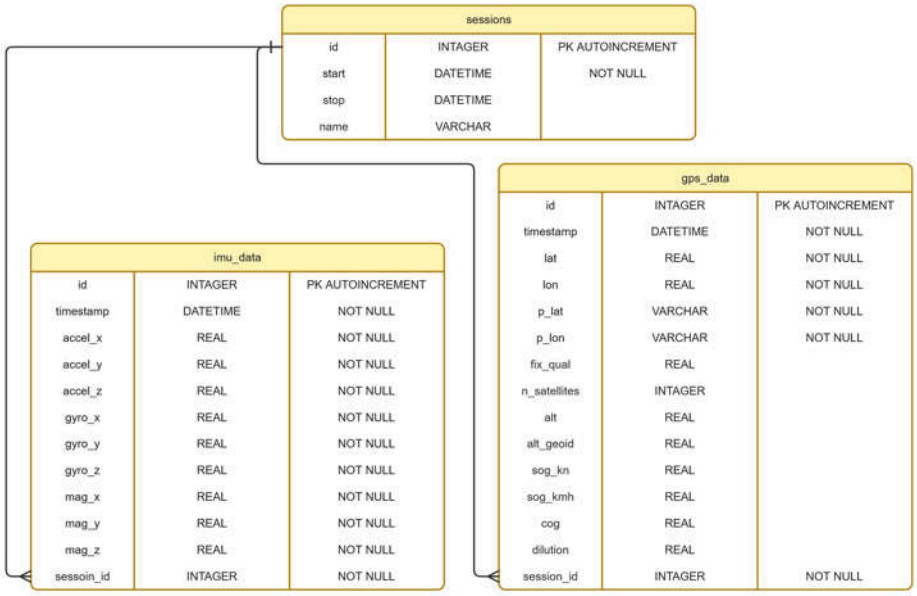


Fig. 15. – Schéma de la base de données relationnelle `sys.db`.
Schéma personnel produit avec Miro.

Avant même de commencer à vouloir travailler avec une base de données en local, il faut installer `sqlite`.

```bash
sudo apt install sqlite3
```

Pour construire la base de données dont le schéma est présenté ci-dessus, on commence par créer le fichier `sys.db`. On l'ouvre ensuite avec `sqlite`.

```bash
touch sys.db
sqlite3 sys.db
```

On construit ensuite la première table en envoyant la requête SQL suivante :

```SQL
CREATE TABLE sessions (
    id INTEGER PRIMARY KEY AUTOINCREMENT,

    start DATETIME NOT NULL,
    stop DATETIME,

    name VARCHAR);
```

Pour chaque champ, on a spécifié le nom, le type et une éventuelle contrainte. On fait de même pour les autres tables.

```SQL
CREATE TABLE imu_data (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    timestamp DATETIME NOT NULL,

    accel_x REAL NOT NULL,
    accel_y REAL NOT NULL,
    accel_z REAL NOT NULL,
    gyro_x REAL NOT NULL,
    gyro_y REAL NOT NULL,
    gyro_z REAL NOT NULL,
    mag_x REAL NOT NULL,
    mag_y REAL NOT NULL,
    mag_z REAL NOT NULL,

    session_id INTEGER NOT NULL);

CREATE TABLE gps_data (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    timestamp DATETIME NOT NULL,

    lat REAL NOT NULL,
    lon REAL NOT NULL,
    p_lat VARCHAR NOT NULL,
    p_lon VARCHAR NOT NULL,
    fix_qual REAL,
    n_satellites INTEGER,
    alt REAL,
    alt_geoid REAL,
    sog_kn REAL NOT NULL,
    sog_kmh REAL,
    cog REAL NOT NULL,

    dilution REAL,
    session_id INTEGER NOT NULL);
```

On quitte finalement la base de données (l'enregistrement est automatique).

```SQL
.exit
```

Pour consulter le contenu d'une table (p.ex `imu_data`) de façon visuelle, on peut exécuter :

```bash
sqlite3 sys.db -header -column "SELECT * FROM imu_data;"
```

6 SOFTWARE DU PREMIER PROTOTYPE

Les codes eux-mêmes sont à retrouver en annexe. On donne ici les commentaires.

Le fonctionnement du prototype est le suivant : on appuie sur un bouton pour lancer une session. Les enregistrements de l'IMU, du GPS et de la caméra commencent. On appuie une deuxième fois sur le même bouton pour arrêter la session. Les enregistrements se finalisent et le prototype reste en stand-by, prêt pour une nouvelle session.

Pour ce faire, on procède comme suit : un code python `interface.py` surveille le bouton et pour chaque appui lance soit le démarrage du code `master.py` soit son arrêt propre. Ce code « maître » est celui qui gère tout le déroulement de la session. Il lance `gps.py`, `imu.py` et `vid.py` qui gèrent respectivement les enregistrements du GPS, de l'IMU et de la caméra. À son extinction, il arrête ces trois processus et lance `grp.py`, dont le but est de finaliser l'enregistrement vidéo et de transformer les informations brutes en fichier `.mp4` lisible.

Tous ces fichiers s'appuient sur un fichier de classes commun, `classes.py`. Les logs s'enregistrent dans un fichier `sys.log`.

Pour la structure générale, j'ai choisi de séparer les différentes tâches en processus distincts et non pas en threads d'un même processus. Je ne pense pas qu'il y ait de « bonne » ou de « mauvaise » réponse à cette question, mais après avoir attentivement pesé le pour et le contre, j'ai choisi cette option en raison de l'excellente isolation qu'elle propose, ce qui est utile si un des processus venait à planter. Cependant, cette méthode complexifie grandement la communication entre les différentes tâches, les processus forçant l'utilisation de pipes.

6.1 Fichier de classes — classes.py

Beaucoup de fonctions et de code sont communs à plusieurs fichiers utilisés. Afin d'éviter de réécrire dans chaque fichier ces parties, on a recours à un fichier de classes. Il suffit de l'importer dans chaque autre fichier avec `from classes import *`

Dans ce fichier, on règle notamment le logging. Ainsi, à chaque fois qu'un log est effectué, quel que soit le fichier, il sera formaté convenablement, puis imprimé dans la console avec la couleur appropriée et enregistré dans le fichier de log `sys.log`.

On définit également la classe `SysDataBase` qui servira à interagir avec la base de données `sys.db`. Elle permet d'utiliser `cursor` sans avoir à l'initialiser manuellement, d'accéder facilement à l'identifiant de la session en cours, et de finaliser proprement un enregistrement.

Finalement, on retrouve aussi la définition de la fonction décoratrice `non_blocking`. Elle prend en argument une fonction et en retourne la version non bloquante (tournant dans un thread séparé). Elle est utilisée pour modifier la fonction de reboot d'urgence, également définie dans ce fichier.

Je n'ai pas créé ce fichier dès le départ, mais à mesure que je codais et que je voyais les répétitions s'accumuler, il est devenu évident qu'il était nécessaire d'y avoir recours.

Bibliothèque pour le logging

Afin de réaliser nos logs, on a besoin de `colorlog`. On peut l'importer comme suit :

```
bash
sudo apt install python3-colorlog
```

6.2 Surveillance du bouton — interface.py

Le bouton est branché entre le pin 18 (BCM24) et la terre. Il n'est pas nécessaire d'ajouter de résistance car le GPIO 24 est réglé en pull up interne.

On commence par créer une instance de la classe `Button` qui sera utilisée pour la surveillance du pin. On définit l'événement `when_pressed` de cette instance comme un daemon. Il attend en arrière-plan. Quand il se réveille, il appelle le callback `bouton_pressed` puis meurt immédiatement.

Lors de l'exécution de la fonction `bouton_pressed`, le flag global `witness_button` passe à `True` et le callback créé par le daemon meurt rapidement (aucune action bloquante ne peut être exécutée dedans).

L'inversion du flag est détectée par la boucle infinie et déclenche, selon le statut du booléen `session_is_running`, soit le lancement de `master.py`, soit l'envoi du signal `SIGUSR1` qui sera intercepté par `master.py` et provoquera son extinction. Le booléen `session_is_running` est ensuite inversé pour que l'action exécutée au prochain appui soit correcte, un `time.sleep(1)` est lancé pour éviter les rebonds, puis le flag `witness_button` retourne à `False`.

Dans les lignes suivant la boucle, on a la gestion d'erreurs. Un reboot est provoqué en cas d'interruption inopinée du script, car le prototype n'est d'aucune utilité s'il ne s'exécute plus.

Dans une précédente version, tout le contenu de la boucle était contenu dans l'événement `bouton_pressed()`, mais le callback créé pour son exécution n'est pas fait pour des tâches bloquantes et se terminait avant la fin de l'exécution du programme contenu dedans. Il en résultait des comportements imprévisibles particulièrement complexes à déboguer car issus d'effets de bord, tels que des inversions de booléens sans raison apparente.

Exécution automatique

Afin que `interface.py` s'exécute automatiquement au démarrage du Raspberry, on utilise `cron`. Il aurait été plus habituel d'utiliser `systemctl`, mais un tel service est beaucoup plus complexe à configurer et le fait d'être exécuté dans un ring plus extérieur au boot n'est pas un problème dans notre cas.

On doit commencer par rendre notre fichier exécutable, puis on ouvre l'éditeur de `cron` :

```bash
sudo chmod +x interface.py
crontab -e
```

À la première ouverture, `cron` va demander quel éditeur on souhaite utiliser. On choisit `nano` en tapant 1, c'est le plus simple dans notre situation. Ensuite, on crée le service suivant :

```cron
@reboot python3 /home/user/interface.py
```

On valide puis on quitte avec `Ctrl+X`, puis `y` pour valider, et `Enter` pour sortir de `nano`.

6.3 Gestion de la session — master.py

L'interception du signal `SIGUSR1` est gérée à la ligne 12. Elle déclenche le callback `handle_usr1` qui passe le flag global `ending_session` à `True`. Ainsi, le callback reste court et non bloquant, ce qui est généralement une bonne pratique.

Ce script va dans un premier temps créer une instance de classe `SysDataBase` afin de pouvoir interagir avec la base de données (notamment récupérer l'identifiant de la dernière session). La première action concrète est de créer la nouvelle session. Son identifiant est déterminé automatiquement avec `AUTOINCREMENT` directement en SQL. La fonction de classe `actualise_session_id()` ajoute 1 au compteur afin de passer de l'identifiant de la session précédente à celui de celle qui vient d'être créée.

La création de la session est directement suivie du lancement des scripts `imu.py`, `gps.py` et `vid.py` qui liront et enregistreront les données des capteurs. On passe en argument l'identifiant de la session à `vid.py`, car mettant un peu plus de temps à démarrer, il est possible que le programme récupère le mauvais identifiant si les sessions sont très rapprochées. Lui donner le même identifiant de session que le master évite toute confusion.

Une fois les processus lancés, on attend de recevoir un signal avec `signal.pause` avant de continuer. Afin d'éviter de terminer le master en recevant un autre signal que celui qu'envoie `interface.py`, on ne continue que si le flag global `ending_session` est vrai (c'est-à-dire si le callback `handle_usr1` a été déclenché par la réception de `SIGUSR1`, le signal envoyé par `interface.py`).

Pour l'extinction des sous-processus, on utilise la fonction `shutdown()`. Elle envoie à chacun un `SIGTERM` pour les terminer. Si ce n'est pas suffisant et qu'après 5 secondes ils sont encore vivants, elle les tue avec `SIGKILL`.

On termine ensuite l'enregistrement de la session en complétant le tableau SQL avec l'heure de fin, puis on la ferme en utilisant la fonction de classe prévue à cet effet.

Finalement, on lance `grp.py` afin qu'il finalise l'enregistrement vidéo correctement. Son exécution peut être lente, mais il tourne en arrière-plan et un nouveau master peut directement être lancé par `interface.py` pour démarrer une nouvelle session.

De même que pour `interface.py`, ce code n'a été nécessaire que pour la coordination des autres, et a été pour cette raison un des derniers à être écrit. Aucune syntaxe de ce code n'est particulièrement complexe, mais il représente la clef de voûte de la logique du code et de son architecture. Il a à ce titre nécessité une importante réflexion préalable.

6.4 Gestion de l'IMU et du GPS — imu.py & gps.py

Ces deux codes, bien que distincts, fonctionnent exactement de la même manière. Ils sont tous deux basés sur le code fourni par le fabricant et présenté dans le Chapitre 5.9 et le Chapitre 5.10.

Tout d'abord, on trouve la gestion des signaux d'arrêt. Quand on reçoit un `SIGTERM` ou un `SIGINT`, le callback `stop_handler` est appelé pour passer le flag global `running` à `False`, ce qui interrompra proprement la boucle de lecture et d'enregistrement des données.

Suit la fonction `acquisition()` contenant le code du fabricant pour la récupération des données. On définit ensuite une instance de classe `SysDataBase` pour l'enregistrement des données, la fréquence et la période, ainsi que d'autres variables comme dans le code du fabricant.

Dans la boucle, on commence par acquérir les données avec la fonction prévue à cet effet, puis on réalise un enregistrement dans la base de données. On attend ensuite le temps restant avant le prochain enregistrement (fréquence - temps passé pour cet enregistrement).

L'écriture n'est validée qu'une fois par seconde afin de ne pas surcharger le Raspberry avec des opérations inutiles. Pour ce faire, on utilise un compteur. Lorsqu'il atteint la fréquence, alors une seconde s'est écoulée, l'enregistrement est réalisé et le compteur réinitialisé.

Finalement, on essaye de valider les derniers enregistrements puis on ferme la connexion. Ces deux opérations sont effectuées par la fonction de classe `terminate_database_connexion()`.

Ces deux codes sont issus du remaniement d'autres codes. Ils sont les premiers que j'ai écrits pour mon TM et ont beaucoup évolué. À partir d'une simple lecture de données imprimées dans la console, ils sont devenus un système d'enregistrement entrelaçant python et requêtes SQL.

Bien que moins complexes que certains autres codes, ils m'ont servi de laboratoires pour des syntaxes qui m'étaient inconnues, telles que le logging, les requêtes SQL, les callbacks, les signaux et la structure try/except/finally. Je n'avais jamais fait de gestion

d'exception auparavant, et travailler avec des capteurs physiques est exigeant sur ce plan-là, puisque les entrées ne sont par définition pas contrôlées.

6.5 Enregistrement de la vidéo — vid.py

On commence par récupérer l'identifiant de la session qui a été passé avec `subprocess.Popen`. Si on lançait l'enregistrement indépendamment du master, l'identifiant serait récupéré à partir de la base de données.

On crée les dossiers dans lesquels seront enregistrées les vidéos. Le dossier de destination est nommé d'après l'identifiant de la session, il se trouve dans le dossier `sessions`.

Avant de commencer à enregistrer, on crée un thread dont le rôle est de surveiller la création des premiers segments vidéos. Ainsi, on a le timestamp le plus précis possible quant au début de la vidéo. Il peut persister un léger décalage dû au fait que l'on ne surveille pas le début de la prise de vue mais celui de l'enregistrement. Malheureusement, il n'est pas possible d'obtenir d'indication plus précise quant au timestamp du démarrage de la vidéo. On définit le thread comme un daemon afin qu'il ne survive pas après la mort de son programme maître.

On exécute ensuite avec `subprocess.Popen` la commande `bash` lançant l'enregistrement de la vidéo. Cette commande est tirée de la documentation de `rpicam-vid`[34]. On enregistre la vidéo par segments de deux secondes en H264 brut. Ainsi, le risque d'obtenir une vidéo endommagée est au plus faible. En effet, des formats complexes tels que MP4 nécessitent un encodage final, alors que H264 reste lisible même en cas d'interruption brutale de l'enregistrement sans finalisation appropriée.

Le code attend ensuite de recevoir le signal d'extinction. Une fois reçu, il essaye de terminer l'enregistrement de la vidéo en envoyant `SIGTERM`. Si au bout de quatre secondes le processus est encore vivant, il envoie `SIGKILL` pour le tuer.

Si une connexion à la base de données avait été ouverte, elle est finalement fermée.

Le code pour l'enregistrement de la vidéo a sans doute été celui qui a été le plus complexe pour moi à écrire sur le plan technique.

En effet, il était pour moi désorientant de ne pas pouvoir lancer une vidéo sans indiquer sa durée à l'avance. J'ai mis du temps à comprendre qu'il fallait en réalité en lancer plusieurs courtes d'une durée définie pour en composer une grande.

De plus, j'essayais au début d'enregistrer au format MP4, mais les fichiers que j'obtenais étaient systématiquement corrompus. Même en passant sur du H264 brut, j'avais encore le problème que la première image de chaque séquence ne coïncidait pas avec la dernière de la séquence précédente. Ce problème a été résolu par l'ajout de `--inline` qui jalonne clairement le début et la fin de chaque séquence.

[34] « Camera software », *Raspberry Pi*, s.d., https://www.raspberrypi.com/documentation/computers/camera_software.html (consulté le 21.10.2026).



Fig. 16. – Photo prise sur le lac lors du critérium Psaros 2026 à Vidy avec une résolution proche de celle de la caméra utilisée

Finalement, j'ai eu beaucoup de difficulté à obtenir un timestamp précis pour le début de la vidéo, étant donné que `rpicam-vid` n'en fournit pas de lui-même et qu'il y a quasiment deux secondes de décalage entre l'exécution de la commande et le début réel de la vidéo. La solution que j'ai adoptée n'est pas parfaite, mais je n'en ai pas de meilleure pour l'instant.

6.6 Finalisation de l'enregistrement de la vidéo — grp.py

De même que pour `vid.py`, on essaye de récupérer l'identifiant de la session via un passage d'argument système depuis `master.py`. Si on n'y parvient pas, on utilise l'identifiant de la session la plus récente présente dans la base de données. On définit ensuite les chemins absolus et relatifs du dossier dans lequel sont enregistrées les vidéos H264 à assembler.

On crée un fichier texte contenant la liste des chemins absolus de chaque extrait H264 classée par ordre chronologique pour FFmpeg. On récupère également le timestamp du début d'enregistrement de la vidéo.

Une fois cela fait, on peut exécuter la commande utilisant FFmpeg pour la fusion des vidéos indiquées dans le fichier texte. Pour gagner du temps processeur, on ne ré-encode pas chaque vidéo, on se contente de les concaténer puis d'enrober le résultat dans un MP4. On passe en métadonnée l'heure de création. La vidéo finale s'appelle `final.mp4` et est enregistrée à côté des extraits dont elle provient.

Finalement, on supprime le fichier texte ayant servi à FFmpeg, puis on ferme la connexion à la base de données si elle a été ouverte.

Une fois le fichier `vid.py` créé, la majorité des problèmes concernant la vidéo avaient déjà été tirés au clair. L'écriture de ce fichier n'a pas été particulièrement complexe étant donné qu'elle découle de la résolution des problèmes qui ont été rencontrés. La seule surprise a été de constater qu'il n'était pas possible de fusionner les vidéos via python directement, mais qu'il fallait passer par une commande `bash`.

6.7 Installation automatique

Au long de ce document, nous avons vu comment, de façon manuelle, configurer un *Raspberry Pi Zero 2 WH* pour les besoins de ce projet, puis comment y transférer les fichiers de code python pour la collecte de données.

Afin de simplifier la réalisation d'un deuxième prototype et plus généralement pour rendre cette configuration reproductible, un système de configuration automatique a été créé.

Le fonctionnement de l'installation automatique est le suivant : on curl le fichier `config.sh` sur le Raspberry, on le rend exécutable, puis on le lance avec

```
bash
cd
curl -LO https://raw.githubusercontent.com/anonyme-sjf-26/
                                            Abraxas/refs/heads/main/config.sh
sudo chmod +x config.sh
sudo ./config.sh
```

Une barre de progression apparaît, puis, au bout de quelques minutes, un message avertissant de la fin de l'installation devrait s'afficher.

6.8 Fichiers de configuration

progress_bar.sh

Le code gérant l'affichage de la barre de progression est adapté depuis un code open source[35]. Après l'avoir vérifié attentivement puis adapté aux besoins du projet, je l'ai intégré. Il utilise des concepts qui me sont inconnus, le fonctionnement n'en est donc pas exactement décrit : je l'utilise comme un plug-in. La barre de progression a l'apparence suivante :

```
plain text UTF-8
Configuring the Raspberry 6/30 ▓▓▓▓▓▓▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒ 20%
```

[35] AVONTURE Christophe, « Linux - Using a progression bar in your script », *avonture.be*, 07.09.2024, https://www.avonture.be/blog/bash-progression-bar/ (consulté le 21.10.2026).

Le principe de fonctionnement est que si un autre code, en faisant tourner une fonction, démarre `progress_bar.sh` comme processus enfant et lui en redirige les sorties (`echo`), `progress_bar.sh` affiche une barre de progression pour cette fonction. Au lancement, le processus enfant reçoit en argument le titre de la barre et le nombre d'étapes nécessaires à sa complétion. Pour faire augmenter le remplissage de la barre, le processus enfant se base sur les sorties du code mère. Il intercepte les expressions contenant `Progress=` et met à jour la barre en fonction du chiffre qui suit.

config.sh

On commence par définir quatre fonctions pour le logging en fonction du niveau des alertes : succès, information, mise en garde et erreur critique.

On définit ensuite une fonction de vérification de l'identité : le script ne devrait être exécuté qu'avec les permissions `root`. La vérification est effectuée juste avant la fonction principale.

On initialise un compteur à zéro pour la barre de progression, ainsi qu'une fonction pour l'incrémenter. Elle sera lancée à chaque étape effectuée avec succès dans la fonction principale.

Suit la définition de la fonction principale elle-même. Elle contient toutes les étapes nécessaires à la configuration du Raspberry, à l'exception du SSH, comme décrit dans le présent document. Les seules étapes ayant demandé une adaptation sont celles faisant appel à une interface interactive et pas uniquement à des lignes de commandes. Seules ces adaptations sont détaillées ci-après.

▸ Configuration de `serial`

Au lieu d'utiliser l'interface interactive de `raspi-config`, on exécute directement les commandes désirées avec le built-in non interactif de ce programme.

▸ Configuration de l'environnement virtuel pour l'IMU

Lors de son exécution, le code de configuration de l'IMU pose trois questions auxquelles il faut répondre successivement oui, non et non. Pour ce faire, on a besoin d'un autre fichier, `auto_install.exp`. Son fonctionnement sera détaillé ultérieurement.

▸ Initialisation de la base de données

Étant donné que le code nécessaire à la création de la base de données n'est pas du `bash` mais du `sql`, il est nécessaire de l'exécuter depuis un autre fichier, `config.sql`.

▸ Configuration de `cron`

La configuration de ce service est interactive par excellence. À ce titre, la rendre possible uniquement via un fichier s'exécutant linéairement a représenté un défi majeur.

On commence par récupérer les règles `cron` existantes et par les copier dans un fichier temporaire (`/tmp/cron_config`). On ajoute ensuite à la fin de ce fichier la règle `cron` qui nous

intéresse, à savoir l'exécution de `interface.py` (rendu exécutable au préalable) au démarrage du système. On remplace ensuite les règles `cron` actuelles par le fichier temporaire. On finit par supprimer le fichier temporaire.

La fonction principale se termine par un nettoyage des fichiers nécessaires à l'installation devenus inutiles.

auto_install.exp

Durant l'initialisation de l'IMU, on utilise un fichier de configuration bash fourni par Pimoroni. Ce fichier demande des confirmations pour trois actions. Il est facile de fournir une réponse unique à un script bash à son lancement grâce à la syntaxe suivante :

```bash
./script.sh <<< 'réponse'
```

Elle est d'ailleurs utilisée dans la configuration non-interactive du GPS. Mais ici, le cas est différent, car les réponses à fournir ne sont pas toujours les mêmes. On a donc besoin d'un fichier expect : `auto_install.exp`. Ce format est prévu pour lancer l'exécution d'un fichier bash, puis pour attendre certaines sorties et pour y répondre selon ce qui a été configuré au préalable. C'est exactement ce qui se produit avec `auto_install.exp`. Il lance le fichier `install.sh` de Pimoroni avec `spawn`, puis attend les trois questions et y répond selon ce qui est décrit dans le fichier, c'est-à-dire, oui, non et non.

config.sql

Ce fichier sql contient exactement les commandes décrites dans le Chapitre 5.13. Elles ont simplement été réunies dans un seul fichier exécuté depuis `config.sh`.

6.9 Passage sur GitHub

Afin de faciliter à la fois la traçabilité des différentes versions du code écrit et son transfert vers le Raspberry, j'ai pris la décision de le transférer sur un dépôt GitHub. Il est accessible publiquement. Ce transfert a été effectué à peu près au milieu de l'écriture du code. Par la suite, d'autres dépôts GitHub ont été ajoutés. Au total, on en a quatre :

- Pour le code de l'appareil : https://github.com/anonyme-sjf-26/Abraxas
- Pour le code du serveur : https://github.com/anonyme-sjf-26/Abraxas_server
- Pour les fichiers 3D : https://github.com/anonyme-sjf-26/Abraxas_3D-files
- Pour les fichiers relatifs au PCB : https://github.com/anonyme-sjf-26/Abraxas_PCB

En effet, ce service permet de continuer à modifier les fichiers via PyCharm comme je le faisais auparavant, mais au lieu d'en enregistrer une copie à chaque modification majeure, il me permet de conserver une trace des modifications sous forme d'arborescence dans un seul fichier, tel que visible sur la Fig. 17. Cela facilite le retour à une version précédente et permet un meilleur suivi des améliorations, permettant de retrouver facilement le fonctionnement d'un code complexe, même après une longue pause.



Fig. 17. – Vue de l'arborescence des modifications dans le fichier `config.sh` sur PyCharm

De plus, le fait que le code soit rendu publiquement accessible permet de facilement demander de l'aide ou de partager l'avancée du projet. Mais surtout, cela permet de télécharger le code depuis les serveurs GitHub sur n'importe quel appareil grâce à une simple ligne de commande :

```bash
git clone https://github.com/anonyme-sjf-26/abraxas.git
```

Pour l'organiser, on utilise ensuite les commandes shell classiques, telles que `mv`, `rm`, `cp`, etc. Dans mon cas, cela permet de transférer la dernière version de mes fichiers sur le Raspberry simplement et efficacement.

Sur le dépôt GitHub, on retrouve également les instructions pour l'initialisation du prototype, la présente documentation et la licence MIT qui place l'ensemble du projet dans le domaine public (open source) tout en limitant ma responsabilité en cas de réutilisation par autrui.

Chaque modification du code (commit) doit s'accompagner d'une très courte description des changements. J'ai fait le choix de les générer automatiquement (sans prompt) par IA, car bien que leur existence soit indispensable, leur écriture est triviale. De façon analogue, les notes de version et les README de tous les dépôts sauf du principal ont été générés de façon automatique par IA. Toutes les informations qu'ils contiennent sont déjà écrites dans ce rapport. En faire un n-ième résumé par moi-même présentait peu d'intérêt.

7 DEUXIÈME PROTOTYPE

Après l'obtention d'un premier prototype fonctionnel comprenant un GPS, un IMU et une caméra, il me restait à ajouter l'anémomètre et à placer le tout dans une boîte étanche, de manière à pouvoir embarquer le prototype pour des tests.

7.1 Alimentation autonome

Jusqu'à maintenant, le prototype était alimenté par le secteur. Afin de pouvoir l'utiliser sur un bateau, il devait être autonome. Pour ce faire, j'utilise une batterie et un PowerBoost.

Batterie et autonomie

J'ai choisi un modèle quasiment plat, de type lithium-polymère (LiPo) fournissant une tension de 3.7V, dotée d'une capacité de 8000mAh. Elle offre une autonomie d'environ 4h d'enregistrement avec la vidéo, et de plus de 8h d'enregistrement de données simple. L'autonomie sans affichage des données en temps réel n'a pas été testée, mais est estimée à plus de 12h.

PowerBoost

Afin de monter la tension de 3.7V à 5V, j'utilise le PowerBoost 1000C d'Adafruit. Il permet également de gérer la charge de la batterie via USB et dispose d'un témoin pour la charge complète (vert), pour la charge en cours (orange), et pour la faible tension (rouge). Ce dernier est connecté au port Low Battery Output (LBO), qui passe à HIGH en 5V lorsque la tension chute en dessous de 3.2V[36].

ON/OFF SHIM

L'extinction et l'allumage du dispositif sont gérés par le ON/OFF SHIM de Pimoroni. Il s'agit d'un interrupteur à verrouillage (latch) composé d'un transistor NPN et d'un PNP.

Par défaut, le PNP est fermé : le 5V ne passe pas. Un appui bref sur le bouton ouvre le PNP, ce qui laisse passer le 5V vers le circuit, allume le dispositif et actionne le NPN (il devient conducteur). Après le relâchement de la pression sur le bouton, le NPN conducteur tient le PNP ouvert en simulant l'appui sur le bouton. Quand, depuis le logiciel, la base du NPN est passée en LOW, il cesse de conduire et donc de tenir le PNP ouvert. Le PNP se referme et le 5V est coupé[37].

[36] « Adafruit Powerboost 1000C », *Adafruit*, 08.03.2024, https://learn.adafruit.com/adafruit-powerboost-1000c-load-share-usb-charge-boost (consulté le 21.10.2026).

[37] « OnOff SHIM », *Pimoroni*, s.d., https://shop.pimoroni.com/en-ch/products/onoff-shim (consulté le 21.10.2026).

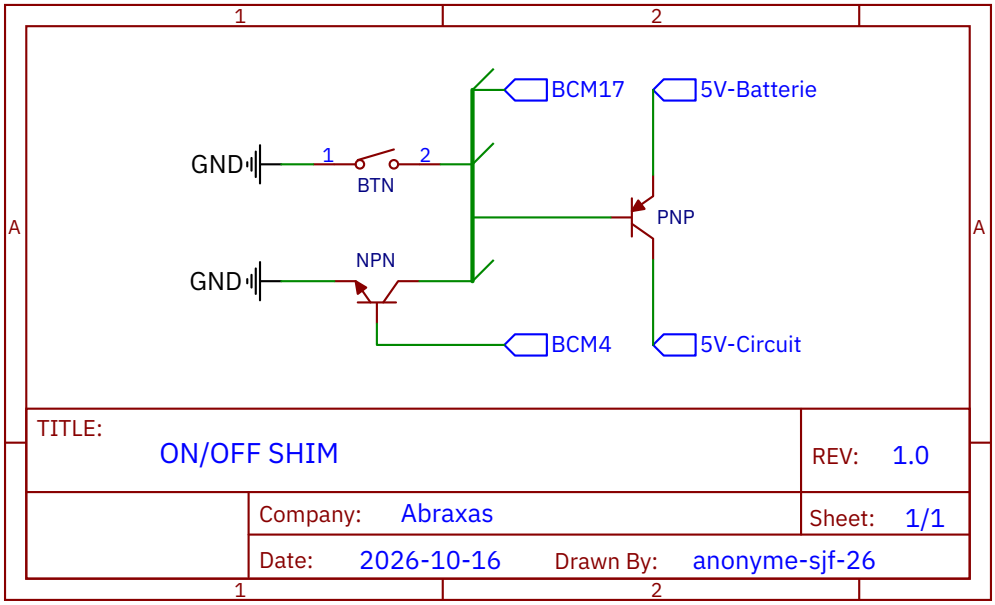


Fig. 18. – Schéma électrique du ON/OFF SHIM.
Schéma personnel créé avec EasyEDA.

Pour mieux comprendre, on prend une analogie : le PNP est une porte par défaut fermée. L'appui sur le bouton l'ouvre et le NPN vient caler la porte pour éviter qu'elle se referme, même quand le bouton l'a lâchée. Ainsi, le 5V peut passer. Si le Raspberry vient couper l'alimentation de la base du NPN, celui-ci tombe et la porte se referme. Le 5V ne peut plus passer.

7.2 Carte mère

Afin d'augmenter la compacité de mon dispositif, j'ai développé, sur conseil de mon professeur accompagnant, une carte mère, ou, en anglais un printed circuit board (PCB). L'avantage de cette solution réside également dans la propreté et dans la robustesse du résultat final. En effet, cela dispense d'un grand nombre de fils et permet d'ajouter divers systèmes de sécurité tels que des condensateurs ou des fusibles.

Pour le développement de ce PCB, j'ai utilisé le logiciel EasyEDA. Il est directement connecté à JLCPCB, l'entreprise qui fabrique les PCB. Ainsi, il est possible, en dessinant la carte mère, d'intégrer directement les composants que l'on soudera dessus ultérieurement. On a leurs empreintes et leurs caractéristiques électroniques, ce qui simplifie grandement la conception.

Schéma électrique

La première étape est le dessin du schéma électrique. Cela représente comment les composants seront connectés entre eux. On utilise pour ça un système d'adresse pour ne pas avoir à dessiner chaque fil. Cela alourdirait beaucoup trop le dessin.

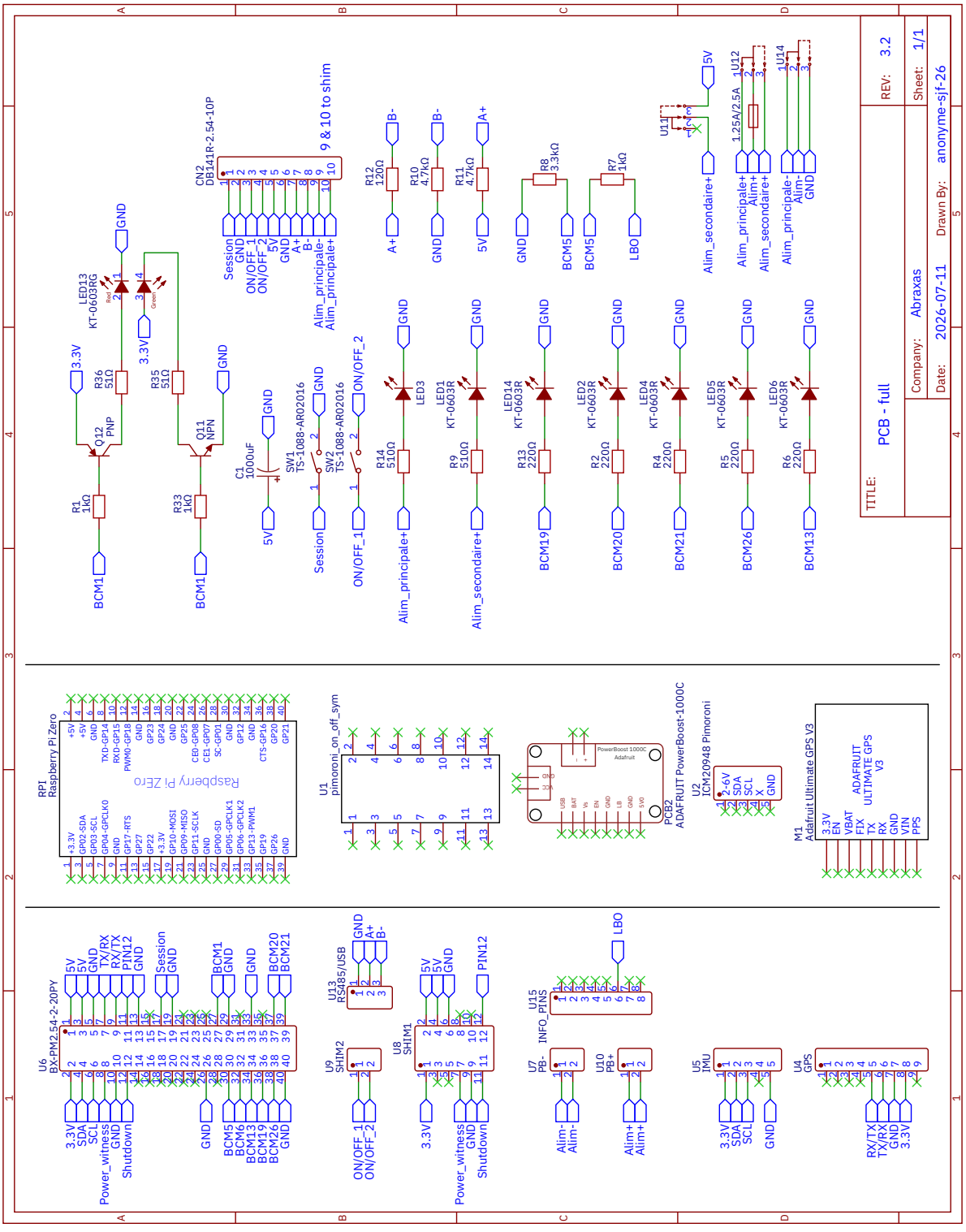


Fig. 19. – Schéma électrique du PCB.
Schéma personnel créé avec EasyEDA.

▸ Alimentation

L'alimentation est divisée en deux circuits distincts : l'alimentation principale et l'alimentation secondaire. La secondaire est prévue comme un plan de secours en cas de dysfonctionnement de la principale.

Dans le cas de l'alimentation principale, le chemin du 5V est le suivant :

```
schéma
Batterie →(via câble)→ PowerBoost →(via PCB)→ bornier →(via câble)→ SHIM
→(via PCB)→ fusible →(via PCB)→ circuit
```

Dans le cas de l'alimentation secondaire :

```
schéma
Batterie →(via câble)→ PowerBoost →(via PCB)→ switch U11 →(via PCB)→ fusible
→(via PCB)→ circuit
```

Le problème de l'alimentation secondaire est que l'allumage et l'extinction du dispositif se font à la main. Ce n'est donc plus réalisable une fois la boîte scellée. Dans le cas de l'alimentation principale, le 5V est coupé par le SHIM, donc de façon contrôlable depuis l'extérieur de la boîte via un interrupteur. Le choix entre les deux alimentations se fait en réglant la position des switch U12 et U14.

Comme sécurité supplémentaire, deux pads ont été prévus en dessous du PowerBoost, directement connectés au + et au - régulés à 5V, à la première étape après la sortie de la batterie, en cas de défaillance totale. Ce sont finalement ces pads que j'utilise dans mon prototype final, car l'interrupteur U12 est défectueux. Je le ponte à partir des pads, de façon à tout de même utiliser le SHIM via l'alimentation principale.

▸ Raspberry

Le Raspberry est lui directement planté sur le bornier U6. Ainsi, la totalité de ses ports GPIO sont accessibles via le PCB.

▸ IMU et GPS

Ils sont branchés comme indiqué précédemment, directement au GPIO du Raspberry. L'IMU se branche sur le bornier U5, le GPS sur le U4.

▸ Anémomètre

Le breakout se raccorde au bornier U13. On retrouve bien les A+ et B- du RS485. La liaison USB entre le breakout et le Raspberry ne passe pas par le PCB. Par conséquent, elle n'est pas représentée. L'anémomètre lui-même se branche au bornier CN2. La liaison au breakout se fait par le PCB.

Pour protéger le RS485 contre la réverbération, on a besoin de trois résistances : une sur le A+, une sur le B-, et une entre les deux. Ce sont respectivement les R11, R10 et R12.

▸ Caméra

La nappe CSI connectant la caméra est indépendante du PCB.

▸ Témoins lumineux

On a sept témoins lumineux. Un pour l'alimentation principale, un pour l'alimentation secondaire, et cinq librement attribuables via les GPIO BCM13, 19, 20, 21 et 26. Chacune de ces LEDs est protégée par une résistance.

▸ Témoin lumineux à inversion

On a un témoin lumineux dit à inversion, c'est-à-dire deux LEDs côte à côte, une rouge et une verte, pouvant s'allumer en alternance. Cette inversion est régie par un seul GPIO, le BCM1, et par deux transistors, un NPN et un PNP (respectivement le Q11 et le Q12). Lorsque le BCM1 est LOW, le PNP conduit et le NPN bloque : la LED rouge s'allume. Quand le BCM1 passe à HIGH, le PNP bloque et le NPN conduit : la LED verte s'allume.

▸ Condensateur

On a placé un condensateur (C1) de 1000µF au plus proche des broches du Raspberry. Ainsi, les pics d'intensité de plus de 2A (capacité en sortie de la batterie) pouvant survenir lors de l'allumage du WiFi seront absorbés sans souci.

▸ Pont diviseur de tension

Les résistances R7 et R8 constituent un pont diviseur de tension. L'indicateur LBO du PowerBoost Adafruit est passé à HIGH lorsque la tension de la batterie chute en dessous de 3.2V, indiquant une batterie faible. Cependant, la tension de ce signal est de 5V. On doit l'abaisser à la logique 3.3V du Raspberry. Pour calculer leur valeur on utilise la formule du pont diviseur de tension :

$$U_2 = U_1 * \frac{R_2}{R_1 + R_2}$$

▸ ON/OFF SHIM

Le SHIM Pimoroni est connecté au PCB par deux borniers : le U8 et le U9. Ses deux bornes court-circuitant son bouton sont reliées à un interrupteur. La base du NPN devant être passée à LOW pour provoquer l'extinction est sur le BCM4. Par ailleurs, le bouton du SHIM est également relié au BCM17 afin que le Raspberry puisse passer le BCM4 à LOW lors d'un appui sur le bouton (signal d'extinction).

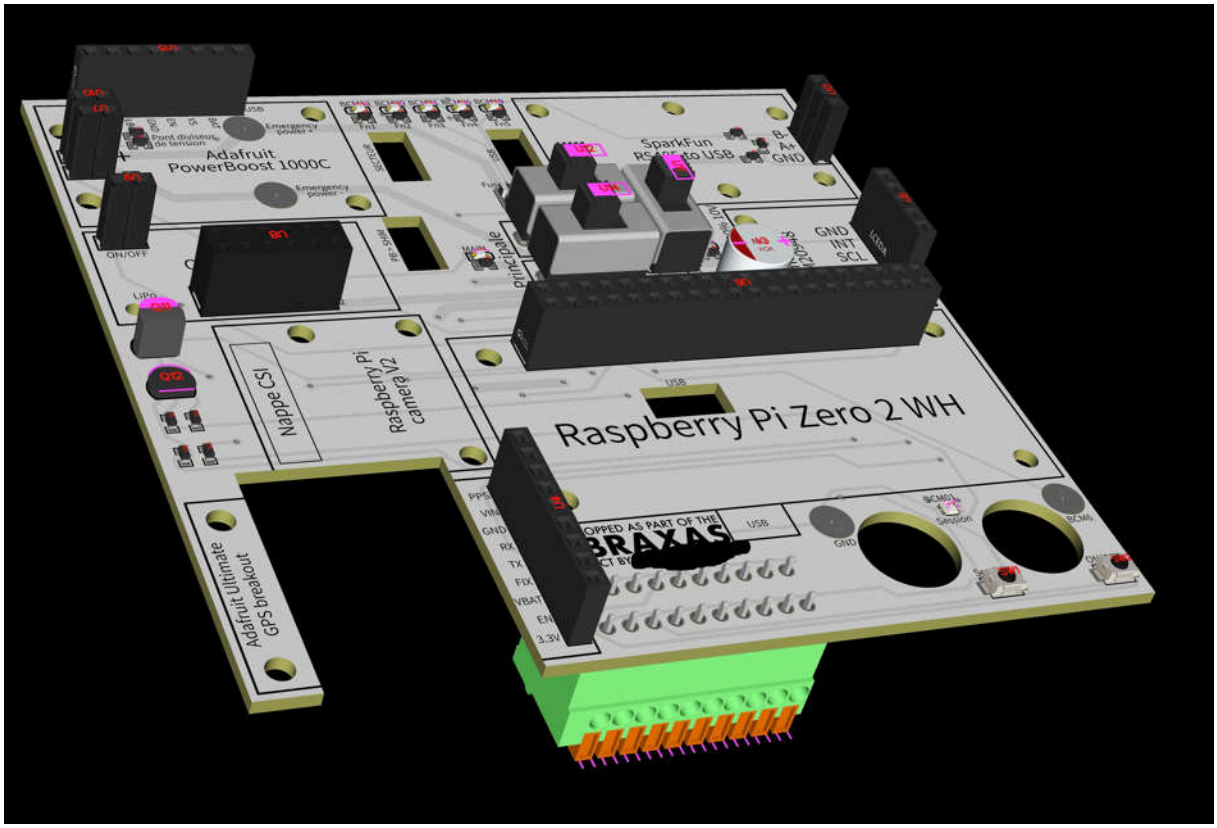


Fig. 20. – Modélisation 3D du PCB avec les composants.
Représentation personnelle créée avec EasyEDA.

- PowerBoost

Le PowerBoost Adafruit est repris sur les U7, U10 et U15. C'est sur le U15 que se situe l'indicateur LBO.

- Interrupteurs

On a deux interrupteurs : un pour démarrer et arrêter la session (court-circuitant GPIO17 et GND), un pour allumer et éteindre le dispositif (court-circuitant les deux bornes BTN du SHIM). Ils sont tous deux repris sur le bornier CN2. Pour chacun des deux interrupteurs, deux ersatz (SW1 et SW2) sont placés à même le PCB de façon à pouvoir tester la fonction des interrupteurs avant qu'ils soient connectés au bornier. Les ersatz pontent les mêmes contacts que les interrupteurs principaux.

7.3 Empreinte du PCB

Après avoir dessiné le schéma électrique, il reste à arranger concrètement ces connexions sur la plaque qui deviendra le PCB. Il est possible d'avoir des connexions (tracks) sur chaque côté de la carte. Pour les faire changer de face, on a recours à des vias, dont on tente de minimiser le nombre par souci d'optimisation. La largeur des pistes et des vias dépend de la tension et de l'intensité pour lesquels ils sont conçus. J'utilise des pistes faisant entre 0.5 et 1.5mm, ainsi que des vias de diamètre intérieur de 0.6mm et de diamètre extérieur de 1.2mm. Cela constitue une marge confortable pour éviter que l'ensemble ne chauffe[38].

Les différents breakouts sont maintenus sur le PCB par le biais d'entretoises. On prévoit des trous dans le PCB pour pouvoir les fixer. Lors de la conception d'une carte mère, il faut avoir une excellente vision dans l'espace pour arranger au mieux tous les composants en un minimum d'espace. Étant donné que la compacité était pour moi une contrainte importante, le travail d'optimisation était particulièrement crucial. Trouver de la place pour chaque élément en limitant l'espace vide, cela en gardant de l'espace pour les différents câbles, relevait du casse-tête. Cette forme inattendue de Tetris fut particulièrement chronophage.

De plus, je n'avais aucune expérience en conception de carte mère, par conséquent j'ai eu jusqu'au moment du test l'inquiétude de savoir si j'avais commis une erreur critique. C'est pour cela que j'ai multiplié les sécurités et les moyens de récupération. Heureusement, le résultat est satisfaisant.

On reproduit une représentation des deux faces superposées, en noir et blanc.

[38] « Calculateur de largeur de piste de circuit imprimé », *DigiKey*, s.d., https://www.digikey.be/fr/resources/conversion-calculators/conversion-calculator-pcb-trace-width (consulté le 21.10.2026).

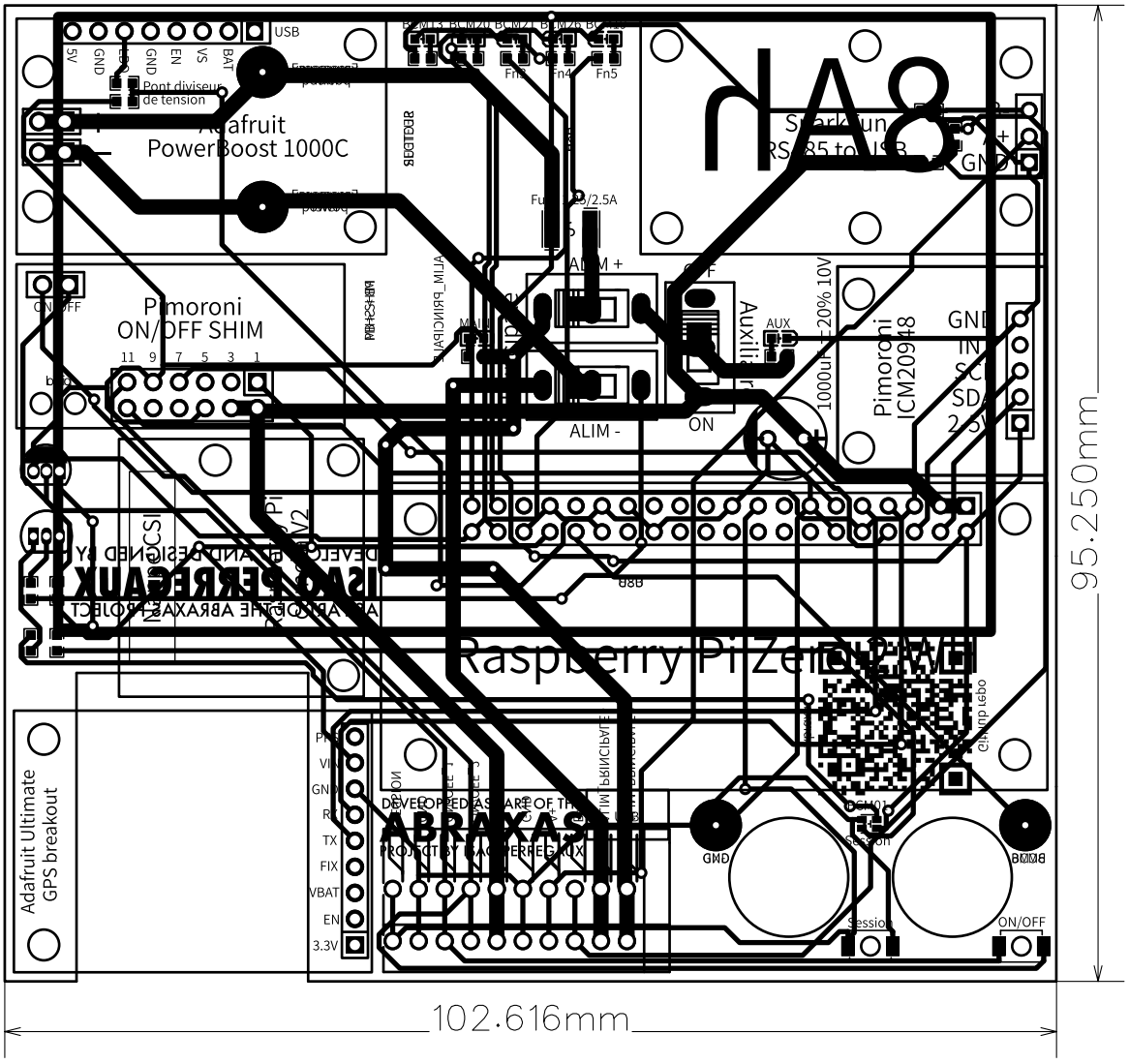


Fig. 21. – Empreinte du PCB, deux faces, noir et blanc.
Représentation personnelle créée avec EasyEDA.

8 TRAVAIL 3D

Afin de pouvoir prendre le dispositif sur le lac et l'utiliser à bord d'un bateau, il était nécessaire d'avoir une boîte étanche dans laquelle placer les composants électroniques. Elle devait respecter plusieurs contraintes :

- Être étanche
- Avoir un couvercle transparent pour la caméra et les témoins lumineux
- Avoir deux passe-câbles pour l'anémomètre et pour l'antenne GPS
- Avoir deux boutons
- Avoir un port USB-C pour recharger le dispositif
- Permettre de fixer solidement les composants à l'intérieur

Pour respecter la totalité de ces contraintes, la solution la plus simple m'a semblé être de modéliser cette boîte en 3D, puis de l'imprimer.

8.1 Modélisation

Je n'avais jamais travaillé avec un logiciel 3D auparavant, il m'a donc fallu commencer par apprendre les bases. La courbe de progression n'est pas excessivement raide, mais j'ai eu besoin d'un temps d'adaptation. Le simple fait de pivoter la pièce dans l'espace n'était par exemple pas une évidence dès le début.

D'une boîte relativement simple, je suis arrivé à un modèle complexe, ayant différents systèmes d'entretoises s'empilant les uns sur les autres et de couches se combinant. Le fichier du modèle se trouve en annexe, en STL (`.stl`) et en STEP (`.step`). Le modèle CAD (conception assistée digitalement) OnShape est également disponible en annexe.

Boîte

En partant du fond du boîtier, on a les couches suivantes :

1. 3mm : fond de la boîte.
2. 14mm : batterie entourée de cales, entretoises fixes sur le côté pour soutenir le PCB. À côté de la batterie se situe le bornier dont sortent les câbles. Il est suspendu au-dessous du PCB (cf. Fig. 20).
3. 4mm : espace réservé aux câblages sortant du bornier et allant vers le passe-câble de l'anémomètre, les deux boutons, le SHIM et le PowerBoost (cf. Chapitre 7.2).
4. 11mm : PCB. Il est soutenu par des entretoises fixes de 18mm, imprimées sur le fond de la boîte. Au sein de cette couche se trouvent les entretoises amovibles, soit vissées sur une entretoise fixe, prenant le PCB en sandwich, soit fixées au PCB depuis le dessous.
5. 15mm : espace libre pour les différents composants. Quasiment totalement exempt de câbles.

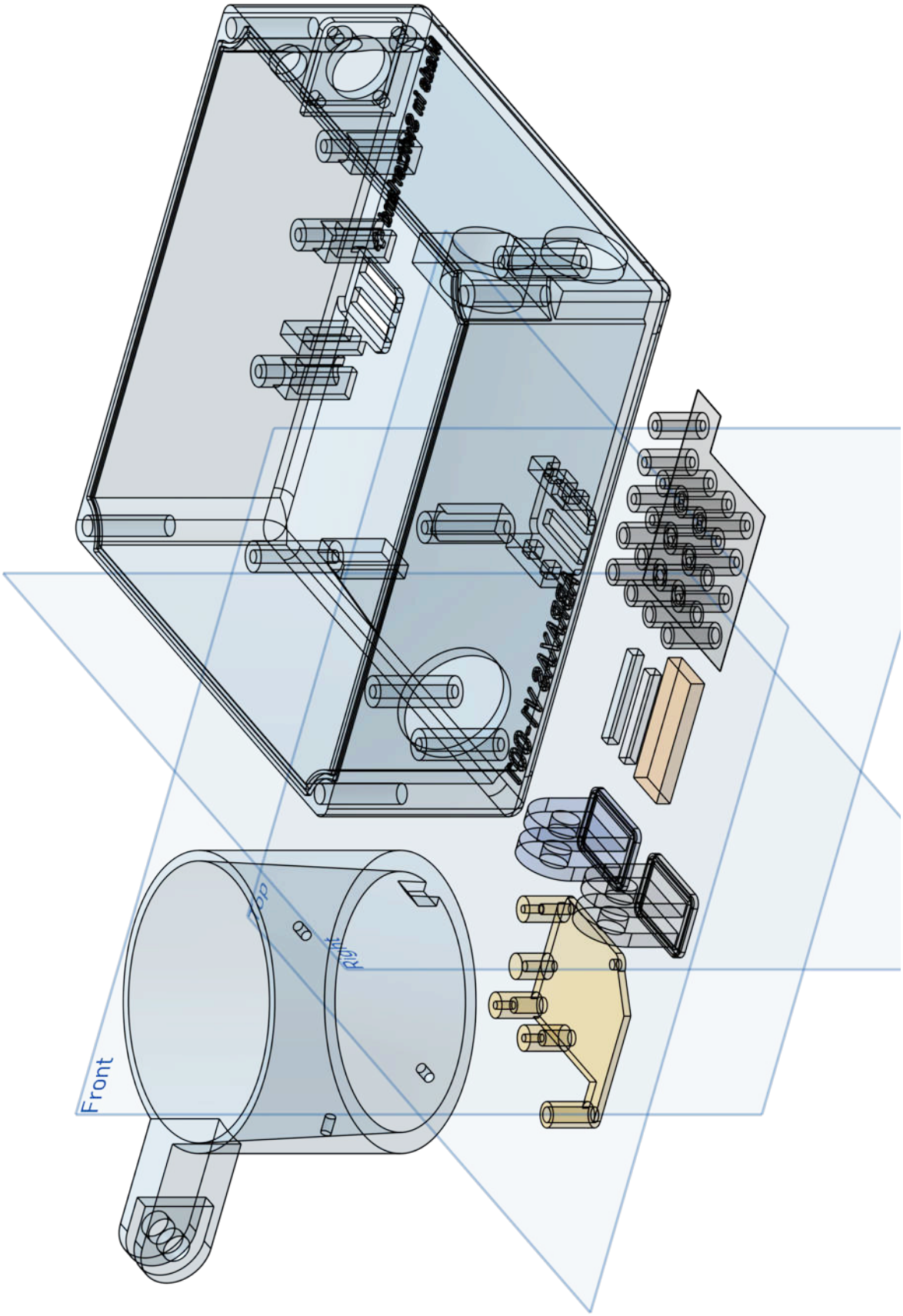


Fig. 22. – Vue d'ensemble du modèle CAD sur OnShape, faces translucides.
Représentation personnelle créée avec OnShape.

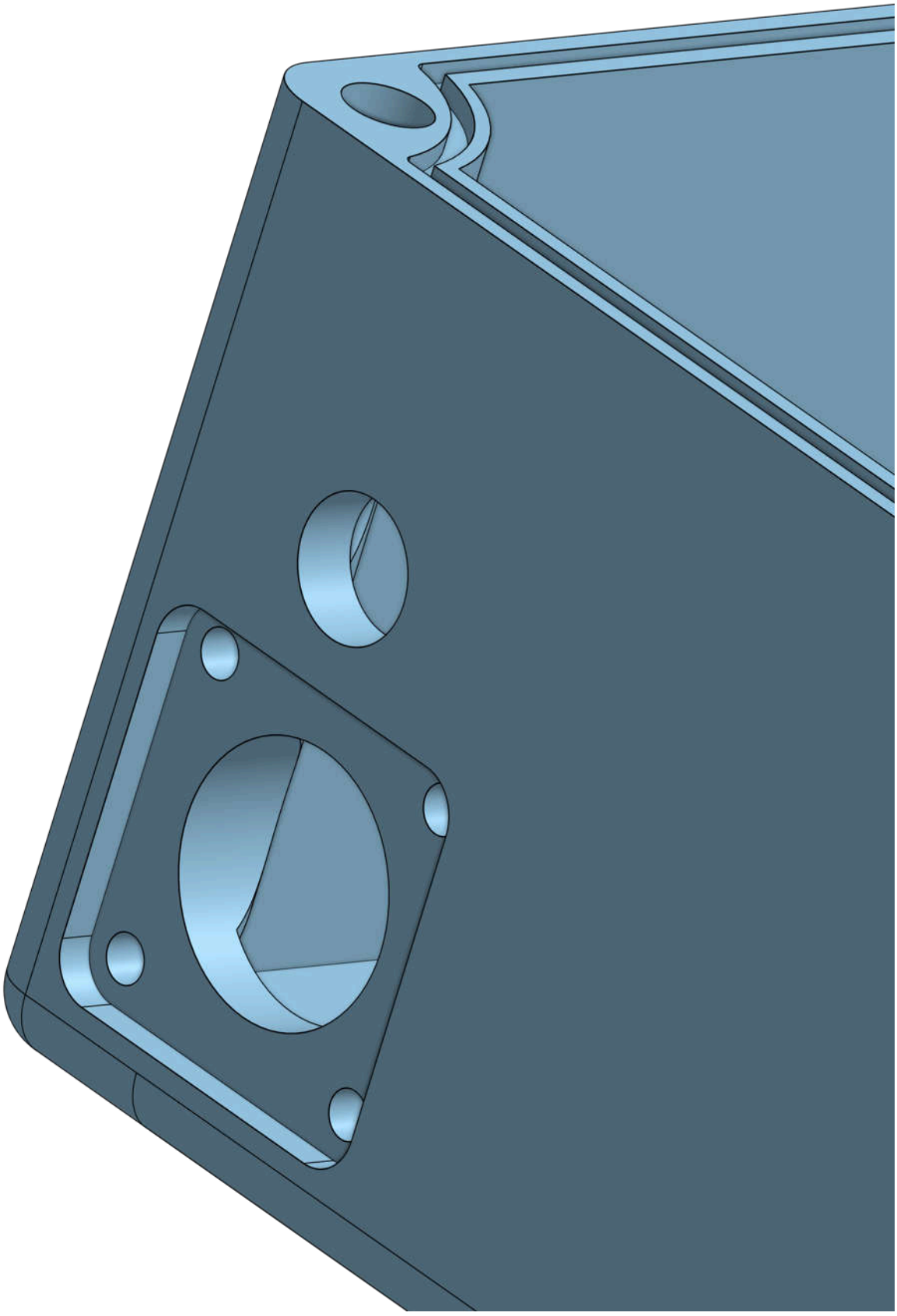


Fig. 23. – Réservation pour les passe-câbles de l’anémomètre et du GPS, face arrière. Représentation personnelle créée avec OnShape.

Au fond de la boîte se trouvent deux réservations prévues pour glisser les fixations compatibles GoPro.

Les parois font 3mm d'épaisseur et 44mm de haut. Le fond fait 116.816mm par 109.45mm. Chaque angle dispose d'un renfort. Sur le sommet, on a une rigole pour placer un joint silicone et une perforation dans chaque angle pour visser le couvercle. Sur la face avant, on a un trou pour le port USB-C. Sur la face arrière, on a le port pour l'anémomètre et celui pour l'antenne GPS. Sur la face droite, on retrouve les deux boutons. L'un pour l'extinction, l'autre pour lancer et arrêter l'enregistrement. Sur le dessous, on peut voir l'inscription « Made in Switzerland + » ainsi que le nom du modèle et son numéro de série.

Couvercle

Le couvercle, en plexiglas transparent, a été découpé au laser. Il fait 3mm d'épaisseur.

Entretoises amovibles

On a 18 entretoises amovibles de trois types différents. Elles se situent entre le PCB et les composants supplémentaires, elles servent à tenir ces derniers. 17 sont « simples », c'est-à-dire qu'elles prennent la forme d'un tube traversé par une ou deux vis. Elles peuvent soit être dans la prolongation d'une entretoise fixe, soit être décalées. Le modèle de l'entretoise diffère selon les cas, car les vis utilisées ne sont pas les mêmes.

La dernière entretoise, celle tenant la caméra, est plus complexe. En effet, la caméra devait être fixée au-dessus d'un espace laissé vide dans le PCB pour laisser passer le socle USB-C. L'entretoise sert donc de « pont » au-dessus de ce vide.

Fixations compatibles GoPro

Afin de fixer le dispositif au bateau, il a été choisi d'utiliser des fixations de type GoPro. En effet, la majorité des clubs disposent d'ores et déjà de supports GoPro sur leur bateau. Ainsi, sans accessoires supplémentaires, le boîtier et l'anémomètre peuvent être fixés simplement et profiter de la grande variété de supports existants.

Il n'était pas possible d'imprimer les supports solidairement à la boîte, car elle n'aurait plus eu de base sur laquelle elle aurait pu être posée durant l'impression. Ils ont donc été prévus comme des inserts séparés à glisser depuis l'intérieur de la boîte. Une fois en place, ils sont maintenus par des trumeaux que l'on coulisse au-dessus d'eux.

Support pour l'anémomètre

La base de l'anémomètre est un cylindre légèrement évasé. Le support épouse cette forme et est imprimé solidairement avec la fixation compatible GoPro. Il comprend trois perforations pour pouvoir y visser l'anémomètre, ainsi qu'un ergot correspondant à une réservation dans l'anémomètre pour en garantir l'alignement avec le support.

8.2 Impression

Lors de la modélisation, j'ai travaillé avec le logiciel OnShape, on peut depuis là exporter un fichier `.stl`. Afin d'imprimer le modèle, il faut encore le « découper ». En effet, un fichier STL est monolithique, il contient les formes pleines, alors que l'imprimante a besoin d'un chemin pour le stylet, du détail du mode de remplissage des formes, des vitesses de déplacement, du type de filament à utiliser, etc. Pour passer de l'un à l'autre, on utilise CrealityPrint, le logiciel fourni par l'imprimante. On obtient un fichier `.gcode`, utilisable par une imprimante spécifique. On peut aussi extraire un fichier `.3mf`, le modèle découpé générique.

Le temps d'impression total de la dernière version s'élève à 3h15. L'impression consomme 36.75m de filament et a un poids de 109.61g. Le G-code et le 3MF se trouvent en annexe.

Pour arriver à cette dernière version, il a été nécessaire de réimprimer trois fois la boîte. Les deux premières comportaient des erreurs soit de conception, soit de marge pour l'imbrication des pièces. En effet, j'ignorais lors de la première impression qu'il fallait laisser des marges entre les pièces. De plus, les apprécier est parfois difficile sans expérience.

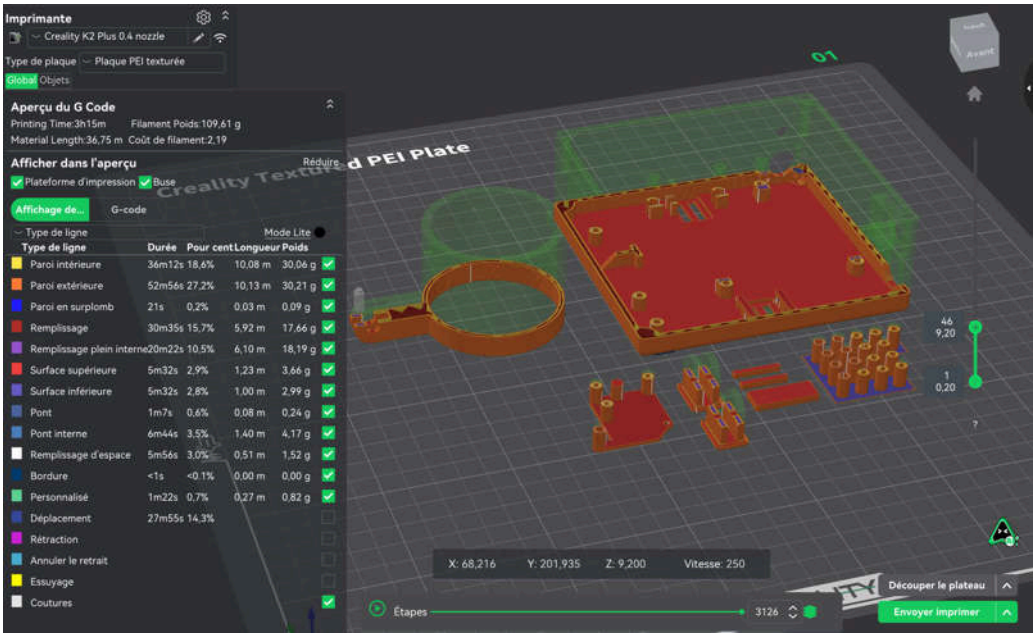


Fig. 24. – Détail de l'étape 46 lors de l'impression du modèle, extrait du gcode.

8.3 Montage

Éléments supplémentaires pour le montage

Pour le montage, on a besoin, en plus des composants du prototype, de composants spécifiques à la boîte. Les références peuvent être retrouvées dans le Chapitre 5.1.

- Deux boutons IP67
- Deux connecteurs de quatre contacts au moins, IP67
- Un connecteur pour antenne active GPS IP67
- Une antenne GPS active IP67
- Un port USB-C IP67
- Un câble USB plat mini-B coudé vers micro-B plat
- Deux fiches USB micro-B dénudées avec deux contacts

Soudures

On coupe le câble sortant de la prise USB encastrable au plus loin du bloc encastrable, on en extrait les câbles d'alimentation, puis on les dénude. Ensuite, on soude :

- Les câbles que l'on vient de dénuder à une fiche USB micro-B.
- Chaque breakout à ses broches. Afin d'être le plus précis possible, on chauffe l'ensemble breakout-broches avant d'approcher l'étain.
- Les câbles aux interrupteurs et aux fiches USB micro-B de façon à pouvoir les connecter au bornier.
- On soude les câbles de l'anémomètre sur le connecteur femelle : Le 5V sur le contact 1, le GND sur le 2, le A+ sur le 3 et le B- sur le 4. Le 5 reste libre. On soude également des câbles sur les contacts 1 à 4 du connecteur mâle.

Toutes les soudures (à l'exception de celles sur les broches directement sur un PCB) doivent impérativement être protégées par des gaines thermorétractables pour empêcher les courts-circuits.

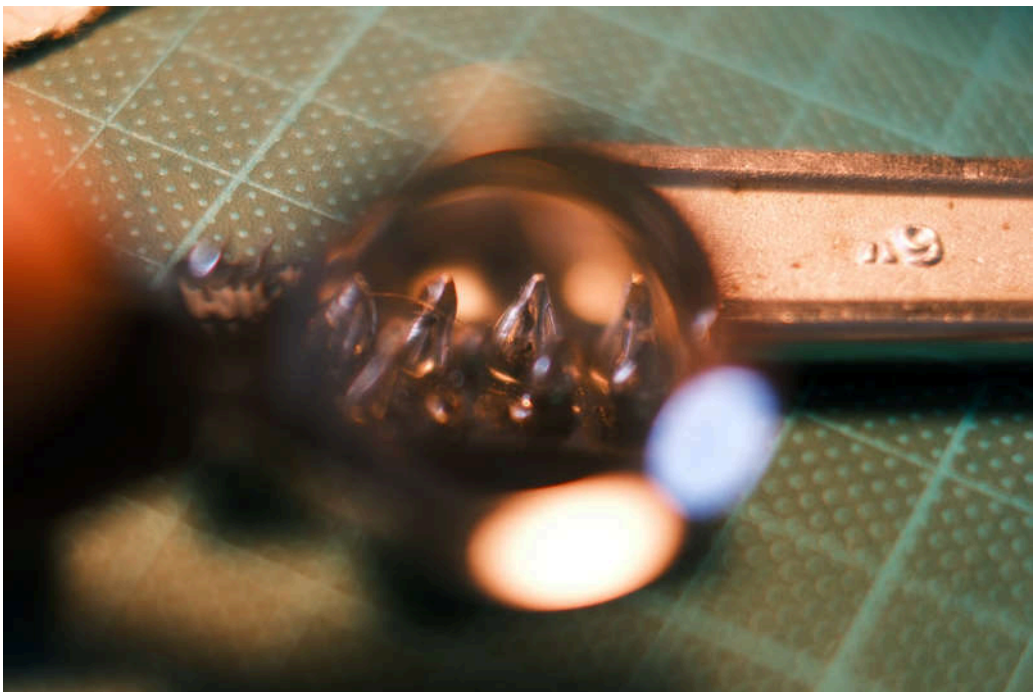


Fig. 25. – Soudure au pas de 2.54mm réalisée à la main, vue à la loupe

Plan de câblage

- Pattes de l'interrupteur supérieur vers GND et vers Session sur le bornier.
- Pattes de l'interrupteur inférieur vers ON/OFF_1 et ON/OFF_2 sur le bornier.
- Pattes 1 à 4 du connecteur sur respectivement 5V, GND, A+ et B- sur le bornier.
- Câbles soudés à la fiche micro-B du PowerBoost au bloc USB-C encastrable.
- Câbles soudés à la fiche micro-B du SHIM vers ALIM_PRINCIPALE+ et ALIM_PRINCIPALE- du bornier.
- Connecteur JST de la batterie vers le connecteur JST du PowerBoost.
- Nappe CSI entre la caméra et le Raspberry.
- Câble USB entre le breakout SparkFun USB/RS485 et le port USB (pas celui dédié à l'alimentation) du Raspberry.

Assemblage

1. On commence par insérer les fixations GoPro au fond de la boîte, puis on les fixe en coulissant les trumeaux dans leurs logements juste au-dessus.
2. On visse les entretoises amovibles qui ne sont pas au-dessus d'entretoises fixes au PCB depuis sa face inférieure. On visse également l'entretoise spéciale pour la caméra.
3. On positionne ensuite les breakouts et le Raspberry sur le PCB, puis on les visse aux entretoises qui ont déjà été positionnées. La nappe CSI peut être repliée sous le Raspberry pour ne pas être visible.
4. On insère ensuite le bouton inférieur, le passe-câble pour l'anémomètre et le port USB-C.
5. On connecte chaque câble au bornier situé sous la carte mère. Ceux connectés au bouton supérieur passent d'abord par sa perforation dans la paroi de la boîte, puis par la perforation circulaire de gauche en bas du PCB. Ceux reliant le PowerBoost au port USB-C passent par la perforation rectangulaire devant le PowerBoost. Ceux reliant le SHIM au bornier passent par la perforation rectangulaire devant le SHIM.
6. La batterie est placée dans son logement au fond de la boîte. Son connecteur JST est connecté à celui du PowerBoost.
7. On positionne le PCB sur les entretoises fixes en prenant garde à arranger correctement les câbles.
8. On positionne le reste des entretoises amovibles, puis on fixe le PCB en le vissant à la boîte avec de longues vis au travers des breakouts, des entretoises amovibles et des entretoises fixes.
9. On insère le bouton supérieur et le passe-câble de l'antenne GPS. Le câble de l'antenne GPS est connecté au GPS. On peut le faire serpenter sous les composants pour éviter qu'il ne soit apparent.
10. On vérifie que les interrupteurs à glissières ALIM+ et ALIM- soient bien sur « PRINCIPALE », et que l'interrupteur à glissière « AUXILIAIRE » soit sur OFF.
11. On pose un joint silicone dans la rigole sur le sommet des parois, puis on pose le couvercle en plexiglas avant séchage pour garantir l'étanchéité. On visse le couvercle.
12. On injecte du silicone dans les perforations du passe-câble de l'anémomètre pour en assurer l'étanchéité.
13. On finit par visser l'anémomètre sur son support. Le plastique étant très dur, il peut être nécessaire de faire une perforation dedans au préalable.

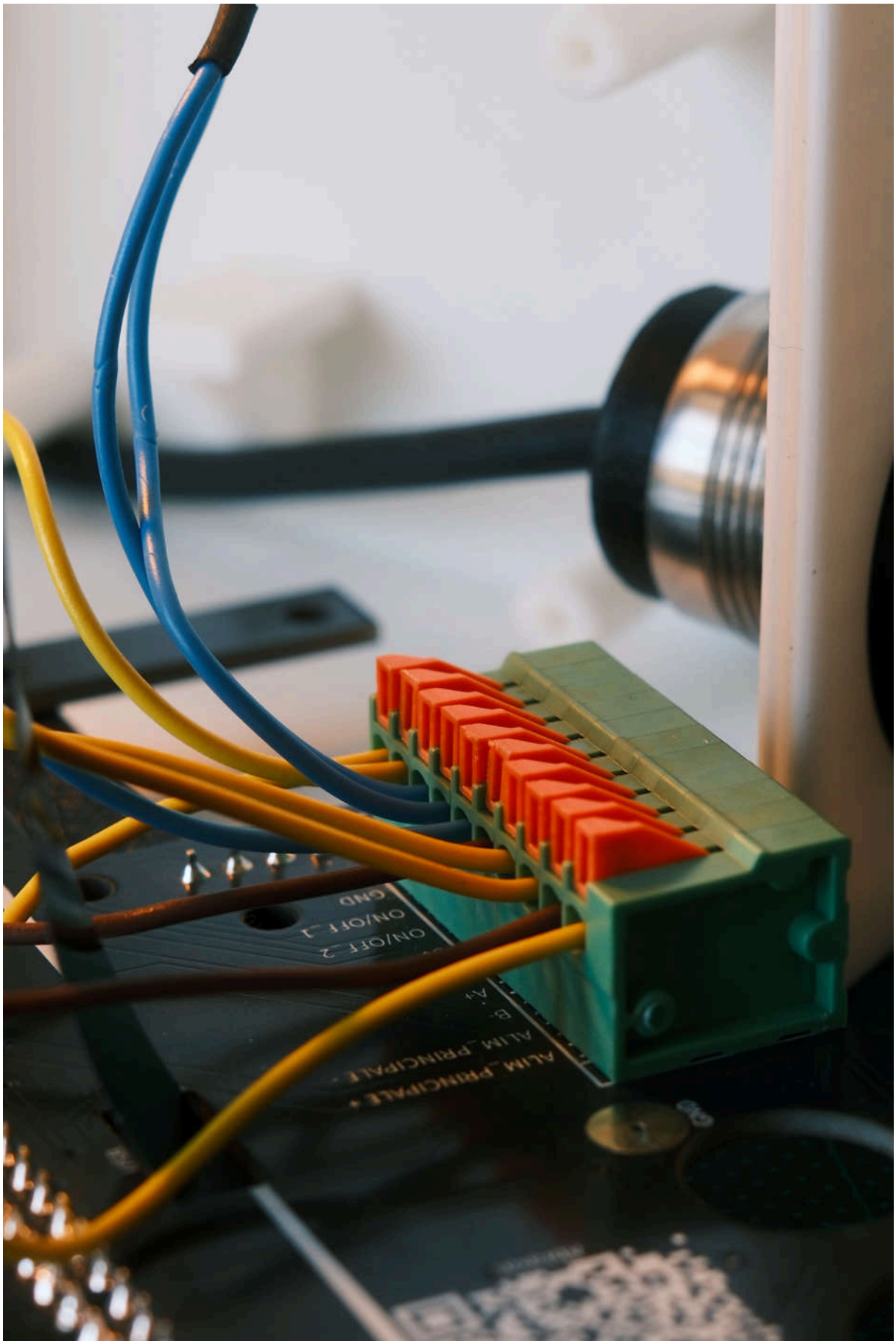


Fig. 26. – Bornier à la fin de l’étape 5

9 SOFTWARE DU DEUXIÈME PROTOTYPE

Au terme de la première étape, j'avais appris des bases dans plusieurs nouveaux langages, notamment le bash, le batch, le JavaScript, le expect, le SQL. Fort de mon expérience dans le développement web pour le site internet d'une association d'élèves, je suis en mesure d'utiliser également du PHP, du HTML, du CSS, du JS et des structures de données en JSON dans le cadre d'applications de faible complexité. Avec ce bagage j'étais armé pour la réalisation de la deuxième partie du software.

La description très verbeuse de ces premiers codes m'a énormément apporté, car elle m'a forcé à réfléchir en détail à leur fonctionnement et à ne pas simplement écrire des codes fonctionnels, mais aussi propres. Pour citer mon professeur accompagnant, il s'agissait de « faire mes gammes ». Désormais, je suis plus fluide dans l'écriture de mes codes, et n'ai plus besoin de passer par cette étape. Par conséquent, les codes seront moins commentés : seule l'architecture générale sera décrite.

9.1 Usage

Une fois monté totalement, le fonctionnement du dispositif est le suivant :

Mise en place du dispositif

Le boîtier est calibré pour être installé à la verticale, par exemple sur le mât, en dessous de la bôme. Il pourrait cependant être installé différemment. L'important est qu'il soit fixé solidairement au bateau, de sorte que l'IMU réalise des mesures fiables.

L'antenne GPS et l'anémomètre doivent être branchés avant d'allumer le dispositif ; sinon, ils ne sont pas reconnus. Pour un fonctionnement efficace, l'antenne devrait être placée pointée approximativement en direction du ciel, dans un lieu dégagé. L'anémomètre devrait, dans l'idéal, être placé au sommet du mât. Cependant, un autre lieu dégagé sur le bateau et où le flux d'air n'est pas excessivement perturbé convient et permet de réaliser des mesures d'une fiabilité acceptable.

Allumer et éteindre le dispositif

On allume et on éteint le dispositif par une courte pression sur le bouton supérieur. Il faut compter une petite minute pour que le système soit opérationnel. Il ne faut éteindre le dispositif que lorsqu'aucun des témoins ne clignote et quand les deux témoins de gauche sont allumés.

Témoins

Les témoins sont situés en haut du PCB. Ils sont au nombre de cinq. De gauche à droite :

1. Interface : s'allume lorsque le dispositif est prêt à recevoir les input donnés par les boutons. Devrait s'allumer une vingtaine de secondes après que le dispositif a été allumé.

2. Serveur : s'allume lorsque le serveur est prêt et disponible via le réseau du dispositif. Devrait s'allumer une minute après que le dispositif a été allumé.
3. Sauvegarde de la vidéo : Indique par un clignotement d'une période de deux secondes que la vidéo est en cours de sauvegarde. S'active après la fin d'un enregistrement. Peut rester actif jusqu'à environ la moitié de la durée de l'enregistrement après qu'il a été coupé. Le fait qu'il soit actif n'empêche pas de lancer un deuxième enregistrement.
4. Non attribué
5. Non attribué

On a un dernier témoin situé au bas du PCB : le témoin d'enregistrement. S'il est vert, une phase d'enregistrement est en cours. Il est rouge sinon.

Lancer et arrêter un enregistrement

On lance et on arrête les enregistrements par une pression courte sur le bouton inférieur. Le lancement peut avoir quelques secondes de délai.

Il est souhaitable d'attendre que le témoin de sauvegarde de la vidéo ait cessé de clignoter pour arrêter l'enregistrement. Il est probable que la sauvegarde s'effectue convenablement de toute manière, mais les tests ont montré que ce cas d'usage manquait de constance.

Affichage des données en temps réel

Pour afficher les données en temps réel, on se connecte au WiFi ouvert ayant le SSID « Abraxas ». Ensuite, dans n'importe quel navigateur, on tape :

```
Adresse IP et port
10.42.0.1:3000
```

Un dashboard présentant les mesures réalisées en temps réel par le dispositif est chargé. Aucune latence notable n'a été constatée.

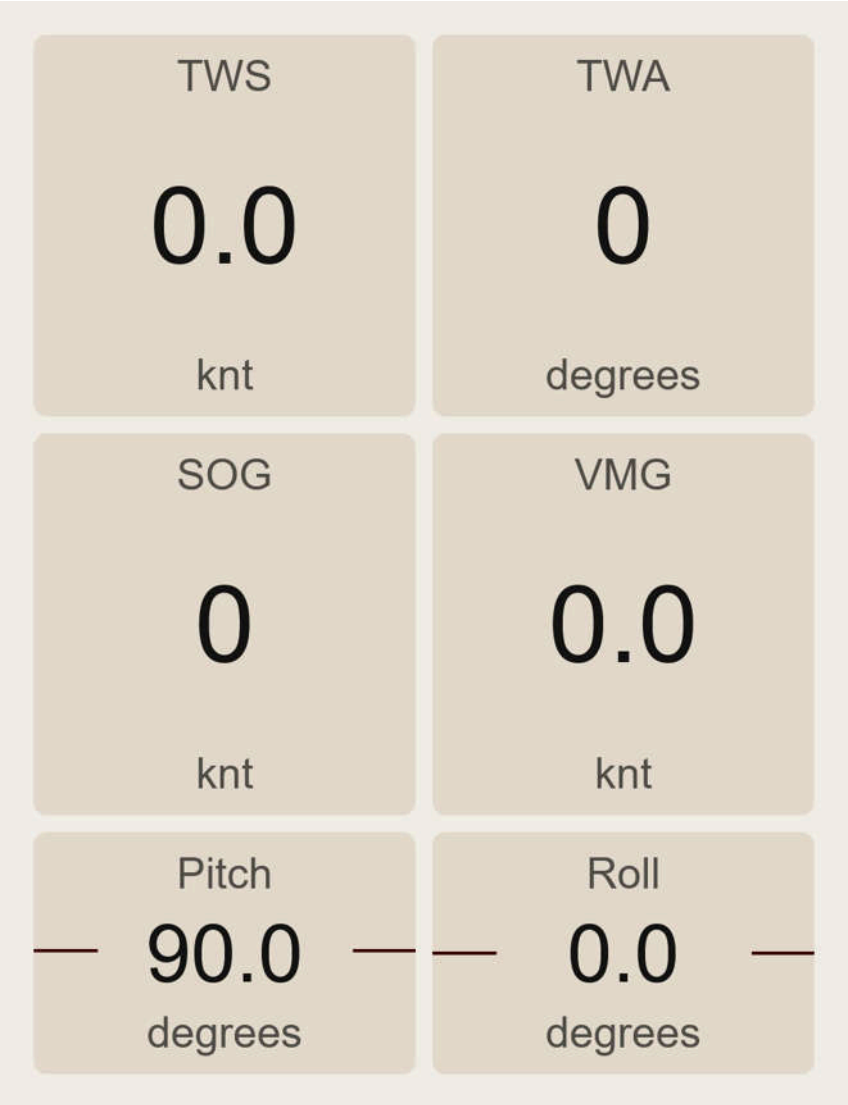


Fig. 27. – Dashboard présentant les données en temps réel

Accès au debrief

L'accès au debrief se fait en deux étapes :

1. On commence par se connecter au WiFi ouvert ayant le SSID « Abraxas ». On se rend ensuite dans un navigateur et on accède à l'adresse suivante :

```
Adresse IP et port
10.42.0.1:3001
```

On y télécharge la base de données et les vidéos souhaitées.

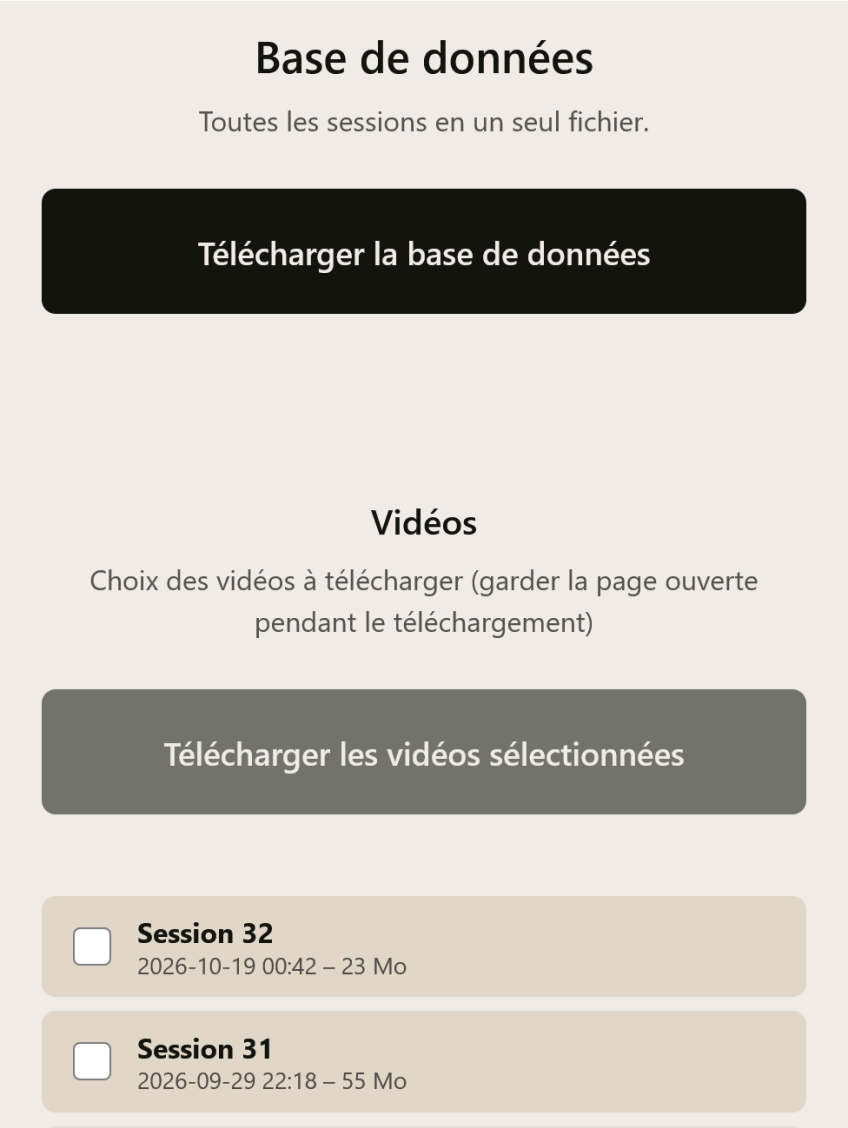


Fig. 28. – Interface de téléchargement

2. On se déconnecte du WiFi « Abraxas » et, par réseau cellulaire ou par WiFi classique, on accède à https://abraxas.example.org/upload. Là, on importe les vidéos et la base de données précédemment téléchargée. Ensuite, dans l'onglet « Sessions », on retrouve les debriefs des entraînements qui ont été importés. (Les cartes indiquant une moyenne peuvent être retournées pour afficher la médiane). Les actions sur ce serveur sont protégées par un mot de passe.

9.2 Installation automatique

L'installation automatique se déroule exactement comme décrit dans le Chapitre 6.7. On exécute :

```bash
cd
```

```
curl -LO https://raw.githubusercontent.com/anonyme-sjf-26/
                                            Abraxas/refs/heads/main/config.sh
sudo chmod +x config.sh
sudo ./config.sh
```

La barre de progression apparaît, puis, au bout de quelques minutes, le système est opérationnel.

On utilise pour ce faire plusieurs fichiers de configuration. La majorité d'entre eux n'a que peu changé depuis leur première version. On peut trouver une description plus verbeuse de leur version originale au Chapitre 6.8.

config.sh

Il liste toutes les commandes à exécuter en bash, charge les autres fichiers de configuration et les exécute. C'est le chef d'orchestre. Il réalise la majorité des configurations.

config.sql

Il crée la base de données pour les enregistrements. L'architecture de la base de données est semblable à celle décrite dans le Chapitre 5.13. La seule différence réside dans l'ajout d'une table pour l'anémomètre. De façon totalement analogue aux autres, elle contient les champs `id`, `timestamp`, les champs de données, à savoir `awa` et `aws`, ainsi que la clé étrangère `session_id`.

progress_bar.sh

Pas de différence par rapport à la première version.

auto_install.exp

Pas de différence par rapport à la première version.

9.3 Système d'enregistrement

L'architecture du système d'enregistrement a subi un changement majeur : auparavant, chaque code créait un curseur pour l'enregistrement dans la base de données et les réalisait lui-même. Cela s'est avéré être une faiblesse majeure durant les tests. En effet, cela peut créer des conflits et interrompre brutalement l'exécution des scripts. Par conséquent, j'ai déplacé la totalité des enregistrements dans un seul script python.

MQTT

Pour la communication des données au sein du Raspberry, j'ai mis en place un système basé sur MQTT via Mosquitto. Le broker est local (à l'adresse de bouclage `127.0.0.1`) et activé en permanence. Le flux est organisé en sujets (topic), classés par groupes. Chaque script a un groupe de topic et émet sur plusieurs topics : un pour la totalité de ses données, destiné à l'enregistrement dans la base de données, et un par donnée qui devra être affichée en temps réel sur le serveur.

db_recording.py

Ce script python s'abonne au topic contenant la totalité des données de chaque fichier. À chaque message reçu, il réalise un enregistrement.

anemo.py, imu.py, gps.py

Le fonctionnement de `anemo.py` est analogue à celui des deux autres scripts : il collecte les données avec une fonction d'acquisition, les parse, puis les traite. Le traitement de ces données est commun aux trois. Ils émettent un message MQTT à chaque entrée de données en suivant l'organisation des topic décrite ci-avant.

interface.py

Son fonctionnement est analogue à la première version. Les changements apportés sont l'ajout de deux fonctions :

1. Lors de son fonctionnement, il allume le témoin numéro 1 en utilisant le service dédié.
2. Lors de l'appui sur le bouton d'extinction, il passe le GPIO BCM4 à LOW, provoquant la chute du NPN du SHIM, entraînant la chute du PNP, qui lui-même coupe l'alimentation en 5V du dispositif.

master.py

Son fonctionnement est analogue à la première version. On a uniquement ajouté le démarrage du script de l'anémomètre et la gestion du clignotement du témoin numéro 3 pour la sauvegarde de la vidéo.

vid.py

Pas de différence par rapport à la première version.

grp.py

Son fonctionnement est analogue à la première version. La seule différence est la gestion du clignotement du témoin numéro 3.

classes.py

Son fonctionnement est analogue à la première version. Les fonctions `emergency_reboot()` et `non_blocking()` ont été adaptées aux nouveaux standards en matière de gestion de l'alimentation (mettre le GPIO BCM4 à LOW pour éteindre le système).

gpio_daemon.py

La gestion des témoins se fait de manière centralisée. Comme je l'ai appris avec les problèmes rencontrés avec la base de données, il est important de séparer le niveau logique et l'interaction avec des services extérieurs au code, surtout s'ils sont physiques. Les différents scripts ne changent pas eux-mêmes l'état des témoins. Ils transmettent l'action à effectuer au daemon via un socket, une manière légère et efficace de communiquer entre différents processus. Le daemon va lui lire ce socket et effectuer les modifications de l'état physique des GPIO en fonction.

L'avantage de cette architecture réside en deux points majeurs. D'abord, elle permet de gagner significativement en robustesse. Aucun risque de conflit dans les changements d'état des GPIO, car tout est centralisé. Ensuite, quel que soit le langage du script voulant effectuer une modification sur l'état d'un témoin, la démarche reste la même. Il n'effectue

pas d'interaction physique complexe, mais une simple modification d'un socket. Un script JavaScript, Python, bash, ou même, si dans l'avenir je devais en utiliser, Rust, PHP ou C peut sans risque de conflit et sans utiliser de méthode différente interagir avec les témoins. C'est un gain énorme en termes d'évolutivité.

9.4 Hotspot

Le hotspot est une nouveauté du nouveau prototype. Le Raspberry devient un serveur envoyant deux interfaces via le réseau local émis par le dispositif. Son fonctionnement est indépendant de toute connexion Internet. Afin de garder le serveur léger pour le Raspberry, dont les capacités restent limitées, j'ai fait le choix de ne pas utiliser de PHP, ni de serveurs réseau complexes tels que Nginx ou Apache. Il est servi par un JavaScript exécuté avec Node.js.

server.js

Il s'exécute via Node.js directement sur le Raspberry. Son rôle est de servir tous les fichiers nécessaires à l'affichage du dashboard d'affichage des données en temps réel, c'est-à-dire `client.html`, `client.css`, `client.js` et `horizon.png` sur le port 3000.

Il écoute les messages entrants par MQTT sur les topic précis (pas ceux contenant toutes les données), les parse et les transmet vers chaque client via WebSocket. L'usage de WebSocket a été décidé en raison de sa faible latence, de sa simplicité de mise en œuvre, de sa légèreté d'exécution et de sa robustesse.

Son rôle est également de servir sur le port 3001 `export.html`, `export.css` et `export.js`, les fichiers constituant la page servant au téléchargement des données. Lors du téléchargement de la base de données, `server.js` ne transmet pas directement `sys.db`. En effet, elle risquerait d'être corrompue car `db_recording.py` pourrait être en train d'y écrire. Il commence par en effectuer une copie temporaire, puis l'envoie. La copie est supprimée immédiatement après. Pour ce qui est des vidéos, il modifie leurs métadatas avant de les envoyer. Il y ajoute un timestamp précis (auparavant situé dans `start.txt`, à côté de la vidéo dans le répertoire). Les vidéos sont envoyées les unes après les autres et l'envoi est retenté en cas d'erreur.

La logique contenue dans ce code étant plus complexe que dans le reste du projet (gestion de la file d'attente, robustesse et protocoles internet notamment), je me suis davantage appuyé sur une aide de la part d'IA au travers de LLMs pour écrire le code. Cependant, je me considère toujours comme l'auteur principal de ce code.

client.html

Il contient la structure de la page. Les données du dashboard d'affichage des données ont des attributs spécifiques récupérables via JavaScript afin que le contenu de la balise puisse aisément être remplacé.

client.css

Contient le style du dashboard d'affichage des données.

horizon.png

Il s'agit de l'horizon artificiel affiché par le dashboard pour représenter visuellement la valeur de roll et de pitch.

client.js

Il est responsable de l'animation des horizons artificiels et de la mise à jour du DOM dans `client.html`. Pour ce faire, il utilise `querySelector` pour récupérer les balises dans le html en fonction des attributs uniques qui leur sont donnés.

Il doit aussi calculer les valeurs à afficher à partir de celles reçues. À partir du SOG, du COG, de l'AWA et de l'AWS, il détermine la VMG, le TWA et le TWS. L'exécution de ces calculs a été déplacée chez le client pour décharger le Raspberry.

export.html

Il contient la structure simple de la page.

export.css

Il contient le style de la page. Ne représentant pas d'intérêt de codage particulier mais étant chronophage à créer proprement, sa création a été déléguée à une IA.

export.js

Il contient la logique pour demander le téléchargement des vidéos et de la base de données, le tout avec de la gestion d'erreur. Il s'occupe également de lister les vidéos, d'afficher leur taille, etc. Globalement, toute la partie interactive de la page repose sur ce fichier. Ce code fonctionnant en vis-à-vis de `server.js` (pour le hotspot), la logique complexe de ce code se retrouve en partie ici. À ce titre, j'ai également utilisé des LLMs pour m'aider à faire correspondre ces deux codes. À nouveau, je me considère toujours comme l'auteur principal malgré tout.

9.5 Serveur distant

Le serveur distant est hébergé sur un sous-domaine du site d'une association d'élèves que son comité me prête. Il est accessible à https://abraxas.example.org. L'architecture prévue pour lui fournir les données des entraînements provient d'une contrainte technique importante : il est généralement impossible d'avoir accès simultanément au WiFi local du dispositif et à Internet. Même sur les appareils ayant deux cartes réseau où cela serait théoriquement possible, il est très complexe de construire quelque chose de stable à partir de là.

J'ai par conséquent décidé d'utiliser la mémoire du téléphone ou de l'ordinateur effectuant le transfert comme relai : les données (base de données et vidéos) sont d'abord stockées dans les téléchargements de l'appareil avant d'être retransférées manuellement. Cela représente un désavantage majeur en termes d'ergonomie, mais pour automatiser ce transfert, il eût fallu passer par le biais d'une application. En effet, les pages Internet n'ont pas les permissions nécessaires à ce type d'automatisations.

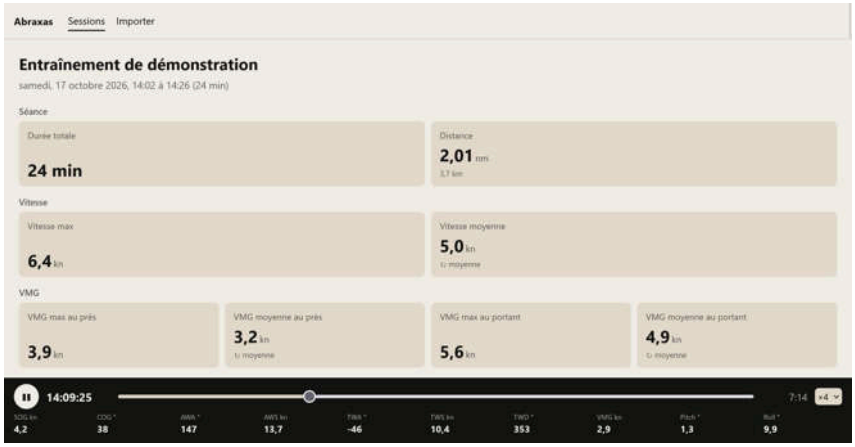


Fig. 29. – Début du debrief d'un entraînement factice

Système du serveur

Le serveur lui-même ne fonctionne pas sur une base standard Apache PHP, mais sur du Node.js. J'ai choisi ce système en raison de l'excellente gestion des processus asynchrones qu'il propose. En effet, chaque session ne doit être analysée qu'une seule fois. Node.js est tout désigné pour ce type d'application.

Fonctionnement et objectif du serveur

Le serveur peut, pour chaque entraînement chargé, produire un rapport complet. Il dépasse la simple lecture de données et le calcul de moyenne que l'on retrouve chez beaucoup de solutions disponibles sur le marché. Il offre une visualisation synchronisée des données de vent, de vitesse, de position, de gîte ainsi que de la vidéo. Les différents graphiques et polaires permettent une analyse fine des données par le coach, tandis que le résumé IA permet aux navigateur·trice·s de donner un sens aux données enregistrées en mettant le doigt sur les éléments positifs et ceux à améliorer qu'elles dénotent.

Le résumé IA est généré une fois pour chaque session grâce à une clé API stockée comme variable d'environnement dans `.env`, à la racine du serveur. Pour interagir avec le serveur, j'ai utilisé du SSH, le FTP étant incompatible avec Node.js.

L'objectif du serveur n'est pas de remplacer le coach : bien que les données soient porteuses de nombreuses informations et permettent d'identifier certains problèmes, elles ne permettent ni de les analyser en profondeur, ni d'en déterminer les causes exactes. Cela provient du fait que la voile est un sport collectif, la dynamique d'équipage ne peut pas être négligée. Le debrief IA a pour but de susciter des questions auxquelles l'entraîneur·euse peut répondre et de permettre de s'orienter dans une quantité massive de données. Le reste des analyses est là pour fournir une base tangible et facilement analysable pour le debrief fait par le coach. Le principal ici est de ne pas oublier que le debrief IA est partiel, qu'il n'est qu'un élément d'une panoplie d'outils.

Le contenu exact et la structure du debrief ont été mis au point suite à une discussion avec Arnaud Psarofaghis[39], triple champion du monde en Moth et en GC32, skipper du challenger suisse Alinghi Red Bull Racing à la trente-septième Coupe de l'America[40] et régleur d'aile sur le F50 suisse à SailGP[41]. Coutumier des bateaux où l'électronique joue un rôle prépondérant, il a pu me renseigner sur les données qui étaient le plus utiles en navigation ou en debrief et sur la façon de les présenter.

Debrief IA

Le debrief IA est généré avec Claude Haiku 5.5. Sa fenêtre de contexte s'étend à la totalité des données collectées durant l'entraînement, ainsi qu'au reste des indicateurs calculés par le serveur. Pour des raisons de confidentialité, de rationalisme énergétique et d'économie de tokens, la vidéo n'est pas incluse dans la fenêtre de contexte.

Le debrief n'est pas généré librement : en effet, il suit une structure réfléchie et stricte. Son objectif n'est pas de prendre des décisions dans l'approche de la performance ou dans le type de debrief, mais uniquement de remplir des cases prédéfinies par du texte décrivant les mesures. La structure suivie (fournie dans le prompt) est la suivante :

> 1. Résumé exécutif
>
>   - l'équipage doit comprendre en 30 secondes ce qui s'est passé et ce qui mérite de l'attention.
>
> 2. Conditions
>
>   - conditions de vent : force, direction, bascules majeures
>   - conditions de mer : vague, houle si applicable
>
> 3. Analyse de vitesse et de VMG
>
>   - approximation du comportement du bateau selon l'angle au vent
>   - petite analyse de type courbe polaire
>   - recherche d'une zone de fonctionnement du bateau
>   - analyse de la dispersion.
>
> 4. Analyse de la stabilité du bateau
>
>   - moyenne et écart-type du roll ;
>   - amplitude des oscillations ;
>   - fréquence des variations ;
>   - corrélation entre roll et SOG ;
>   - corrélation entre variation de roll et variation de vitesse ;
>   - identification des périodes où le bateau devient instable ;

[39] PSAROFAGHIS Arnaud, Compétiteur professionnel, discussion du 22.09.2026.

[40] « Arnaud Psarofaghis », *Alinghi*, s.d., https://www.alinghi.com/crew/33_ArnaudPsarofaghis# (consulté le 21.10.2026).

[41] « Arnaud Psarofaghis », *SailGP*, s.d., https://sailgp.com/teams/switzerland/arnaud-psarofaghis/ (consulté le 21.10.2026).

- comparaison entre navigation stable et navigation rapide.

5. Analyse des manœuvres
    - phases d'accélération après manœuvre
    - vitesse avant
    - vitesse minimale
    - perte maximale
    - temps pour retrouver 95 %
    - temps pour retrouver vitesse initiale
    - TWA avant
    - TWA stabilisé après
6. Analyse des transitions de vent
    - variations de pression
    - bascules du vent
    - comportement du bateau pendant ces événements
    - réaction de l'équipage
    - impact sur les performances
7. Analyse de la régularité
    - vitesse moyenne
    - vitesse médiane
    - 10e / 25e / 75e / 90e percentile
    - dispersion
    - fréquence des pertes importantes
    - durée passée dans une zone de performance cible
8. Bilan
    - retour sur les points importants

Rôle du serveur au sein du projet

Le serveur n'est pas pour moi le centre de mon projet, au contraire. Les debriefs et leur affichage consistent majoritairement en de l'analyse de données. Ce qui me tenait le plus à cœur était la collecte de données, le travail avec un micro-ordinateur, le travail sur Linux par ligne de commande et, dans une certaine mesure, les calculs directement liés à la pratique de la voile. Ce sont ceux que j'effectue dans le dashboard d'affichage des données en temps réel, que j'ai déterminés à la main et détaillés dans le Chapitre 3.

Néanmoins, l'accès à des analyses sur les entraînements enregistrés contribue largement à l'atteinte de mon objectif initial, à savoir permettre une réelle progression des équipages utilisant ce dispositif. De plus, il permet de mettre en valeur le travail que j'ai effectué en lui offrant une visualisation confortable.

Code du serveur

Face à cette situation – nécessité d'un élément non strictement compris dans mon projet, complexe et long à développer – j'ai pris la décision, en concertation avec mon professeur accompagnant, de déléguer sa création à une IA. J'ai utilisé le modèle Claude Opus 5.5 d'Anthropic pour générer ce code.

Dans ce processus, j'ai tout de même souhaité garder le plus grand contrôle sur ce qui a été codé. Par conséquent, je n'ai laissé au LLM utilisé quasiment aucune latitude dans l'architecture choisie, dans l'aspect de l'UI et dans la structure du debrief, tout particulièrement dans la section debrief IA (chaque sous-section a été explicitée). De plus, par souci de sécurité et de rigueur, j'ai relu la totalité du code qui m'a été fourni en m'assurant d'en comprendre le fonctionnement. Ainsi, je suis certain que, même si je ne suis pas l'auteur de ce code, j'en garde tout de même la maîtrise.

Étant donné que je n'ai pas écrit ce code moi-même, je ne vais pas en faire la description complète. Il est accessible dans le dépôt GitHub Abraxas_server. Je ne le considère pas comme faisant partie de mon travail de maturité. Par contre, le résultat qu'il crée, l'outil qu'il constitue est, lui, une brique primordiale dans le système que je développe.

9.6 Abraxas OS

Une image du système d'exploitation après configuration a été réalisée sur un Raspberry Pi Zero et sur un Raspberry Pi Zero 2. Elle contient la version modifiée de *Raspberry Pi OS* permettant de faire fonctionner le dispositif du projet Abraxas. On l'appelle Abraxas OS. Au lieu de passer par un système d'installation automatique sur un OS vierge, on peut directement télécharger l'image de la version d'Abraxas OS souhaitée (Raspberry Pi Zero ou Raspberry Pi Zero 2) et la flasher sur une carte SD. On insère la carte SD dans le Raspberry, et le système est opérationnel dès le premier boot, sans aucune configuration supplémentaire.

10 ESSAI

Rapport de l'entraînement accessible à l'adresse https://abraxas.example.org/session?id=33.

Un essai en conditions réelles a été réalisé le mercredi 21 octobre 2026, au Yacht Club de Genève. Il s'est déroulé entre environ 14h et 15h30, sur un Open 5.70 (lesté bi-safrans léger), dans un vent du sud variable, de force oscillant entre 12 et 20knt. Le lac était peu agité, la couverture nuageuse forte, devenant moyenne, la température était d'environ 10 degrés. Des précipitations moyennes, devenant faibles, ont commencé durant l'entraînement. Deux membres de l'équipe de compétition étaient à l'embraque et à la grand-voile ; j'ai barré le bateau durant l'entraînement.

10.1 Installation du dispositif

L'installation de l'anémomètre s'est faite avec une fixation ventouse GoPro à même la coque. Elle a bien tenu pendant l'entier de l'entraînement. La fixation de la boîte s'est avérée être plus complexe : elle est prévue pour être attachée au mât, mais celui de l'Open, rotatif, ne s'y prêtait pas. Nous l'avons finalement accrochée avec une ventouse également, ce qui s'est avéré être très stable et rigide. L'installation nous a pris une quinzaine de minutes, mais, maintenant que nous savons comment procéder, nous serions probablement beaucoup plus rapides. Les différents câbles ne nous ont finalement pas du tout gênés et, globalement, le dispositif s'est fait oublier durant la navigation. Le fonctionnement durant la navigation était bon : il a suffi d'un appui sur le bouton pour démarrer la session, et d'un autre pour l'arrêter.

10.2 Déroulement de l'entraînement

Nous avions prévu d'enchaîner les manœuvres afin de tester les capacités d'analyse du système. Après être partis du Yacht Club, nous avons commencé par monter jusqu'à la hauteur de la plage des Eaux-Vives. La première montée s'est bien passée, le système n'a demandé aucune attention particulière, nous permettant de nous concentrer sur la navigation. J'ai pendant cette remontée consulté le dashboard présentant les mesures en direct, elles semblaient cohérentes. Le vent a graduellement monté à mesure que nous approchions de Genève, ce qui est habituel dans ce type de régime. Au cours de cette première remontée, nous avons rencontré une avarie sur le système de direction (au temps 04:07:55 dans le dashboard) : la rotule bâbord de la barre de liaison s'est rompue. Cette pièce sert à relier les deux gouvernails du bateau. Sans elle, il est toujours possible de naviguer, mais le bateau est beaucoup plus difficilement manœuvrable. Il est nécessaire de tenir les deux barres franches à la main.



Fig. 30. – Abraxas fixé à l’aide d’une ventouse GoPro sur le pont.



Fig. 31. – Vue du rapport au moment de la rupture de la rotule.

Une fois arrivés à la limite du plan d'eau, nous avons envoyé le spi et entamé la descente. La manœuvre a présenté une difficulté inattendue : une partie de la voile s'était enroulée autour du tangon. Ce problème a rapidement pu être résolu. Durant cette descente, la pluie s'est installée et est allée en s'intensifiant. L'étanchéité du boîtier a pu être éprouvée. Le vent a baissé à mesure que nous nous éloignions de Genève, comme prévu. Nous ne sommes pas descendus beaucoup en dessous du Yacht Club.

Lors de la deuxième remontée, nous avons navigué groupés avec le reste de l'équipe. Cela a été l'occasion de tester nos manœuvres et notre vitesse par rapport à d'autres bateaux. Durant cette remontée s'est produit un incident qui a mis fin à l'entraînement (au temps 05:06:30 dans le dashboard). La barre franche bâbord a cédé au large du Yacht Club. Cette avarie critique de l'appareil de direction ôte tout moyen de contrôle sur la route du bateau. L'urgence était d'affaler le spi afin de nous mettre nous-mêmes et le bateau en sécurité. Suite à cela, l'enregistrement a été coupé et nous avons été tractés jusqu'au port par un semi-rigide.

10.3 Fonctionnement technique du dispositif

Le fonctionnement du dispositif en lui-même s'est révélé être majoritairement conforme aux attentes. La qualité de la vidéo est bonne, les mesures de position, de vitesse et d'orientation sont présentes et précises. Le seul problème est relatif aux données de vent : l'anémomètre a enregistré la quasi-totalité de la navigation comme étant au portant. Il n'a presque pas détecté de près. Le fait que quelques mesures de TWA soient tout de même positives montre que le problème ne vient pas d'une erreur de calcul qui rabattrait toutes les valeurs dans la plage négative. Mon hypothèse est que c'est le positionnement de l'anémomètre qui est en cause : sur la coque, le flux d'air qu'il capte est hautement perturbé et dévié par les voiles. Afin de remédier à ce problème, ma première tentative serait de placer l'anémomètre au sommet du mât, là où l'air n'est pas perturbé.

Globalement, je considère que si mon dispositif a résisté à cet entraînement, il pourra résister à tous les autres. Il a subi dès sa première sortie la pluie, le vent, le départ au tas, la casse matérielle, le remorquage d'urgence, et ne semble pas avoir subi de dommage particulier.

10.4 Qualité de l'analyse et de la visualisation

Sans données de vent analysables, on perd tout le volet relatif à la VMG et aux courbes polaires, un des plus intéressants. En ce qui concerne l'analyse des manœuvres, on voit beaucoup de faux positifs et de faux négatifs en raison de la pollution des données de vent. Les analyses de celles qui ont été correctement détectées sont pertinentes. Toutes les analyses relatives à la vitesse et à l'orientation sont non seulement correctes, mais aussi particulièrement précises. La visualisation de la position est bonne. Globalement, la synchronisation des différentes visualisations est bonne (une fois rectifiée) et l'interface fonctionne correctement sur téléphone comme sur ordinateur.

À la fin de l'entraînement, j'ai lancé le transfert des données pendant que nous nous changions. Il était terminé et l'interface était prête lorsque nous sommes sortis des vestiaires, ce qui est un temps tout à fait acceptable puisqu'il aurait permis d'effectuer un debrief sans attente.

10.5 Inconvénients de l'usage du dispositif pour l'équipage

Les deux seuls inconvénients pour l'équipage lors de cet entraînement ont été le temps de montage au début de l'entraînement ainsi que la contrainte de devoir se soucier de prendre un appareil supplémentaire à l'entraînement et de s'assurer qu'il soit chargé. Je considère ces deux inconvénients comme largement surmontables, car le montage, une fois connu et les supports appropriés prévus, sera beaucoup plus rapide. Quant au fait de devoir avoir un appareil électronique chargé avec soi, il est analogue à ce que tout un chacun fait au quotidien avec son téléphone portable.

10.6 Bénéfices de l'usage du dispositif pour l'équipage

Dans le cadre de cet entraînement particulier, marqué par de la casse matérielle et passablement écourté, il n'a pas été possible de réaliser de réelle analyse de performance constructive. Il a cependant été possible de recomposer exactement les deux incidents qui se sont produits, de mieux les comprendre et d'expliquer à notre entraîneur précisément où, quand et comment ces incidents s'étaient produits. Cela laisse entrevoir une perspective très prometteuse pour l'analyse de performance. Même à ce stade, sans avoir d'informations parfaitement exploitables sur le vent, on voit que la carte, les points clés au début du rapport ainsi que la vidéo sont des éléments extrêmement utiles et plaisants. La synchronisation entre les différentes visualisations est un réel plus par rapport aux autres combinaisons manuelles d'outils que nous avions pu expérimenter. Globalement, je suis satisfait de ce que j'entrevois comme possibilités avec ce système pour notre équipage.

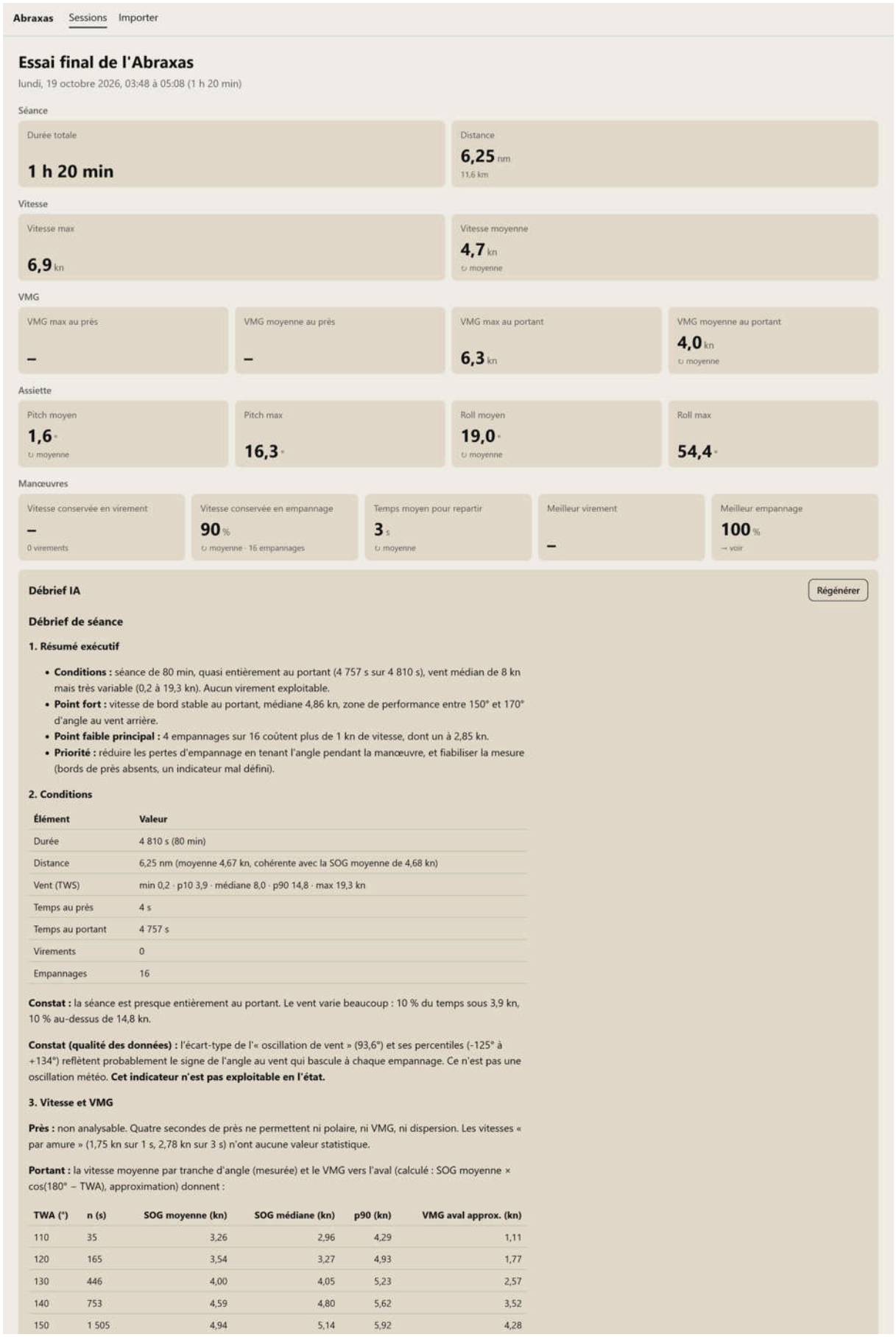


Fig. 32. – Vue de la page de debrief, à 05:01:26, durant un bord de près (partie 1).

150	1 505	4,94	5,14	5,92	4,28
160	**951**	**4,98**	**5,15**	**5,94**	**4,68**
170	460	4,75	4,87	5,73	4,68
180	442	4,63	4,66	5,77	4,63

Constat :

- La vitesse monte régulièrement de 110° à 160°, puis recule légèrement à 170° et 180°.
- Le VMG aval forme un plateau entre 160° et 180° (≈4,6–4,7 kn). Il chute nettement sous 140°.
- 61 % du temps (2 916 s sur 4 757) est passé entre 150° et 170°.
- VMG aval moyen 4,02 kn, médian 4,20 kn, max 6,34 kn.

Hypothèse : au-delà de 160°, le gain de VMG vers l'aval devient marginal. Une partie du temps passé à 150–160° pourrait être gagnée en route directe, mais cela reste à confirmer par la mesure du cap et de la route.

Dispersion (SOG) : p10 3,18 · p25 4,03 · p50 4,86 · p75 5,42 · p90 5,79 kn. L'écart interquartile est de 1,39 kn, ce qui est large : la vitesse dépend fortement des conditions ou de la conduite.

4. Stabilité

Indicateur	Valeur
Gîte absolue moyenne	19°
Écart-type de la gîte	12,4°
Amplitude p5–p95	37,6°
Tangage moyen	1,6°
Écart-type du tangage	3,1°
Corrélation gîte / vitesse (« près »)	0,96

Constat : la gîte est forte et variable (amplitude de 38°). Le tangage est faible et régulier. Les périodes instables « au près » ne sont pas calculables (aucune fenêtre listée, 4 s de près).

Constat (incohérence) : la corrélation de 0,96 est étiquetée « près », alors qu'il n'y a que 4 s de près. Elle a donc probablement été calculée sur toute la séance. **Ne pas l'interpréter comme une relation gîte-vitesse au près.**

Hypothèse : gîte et vitesse montent ensemble avec la pression de vent (rafales). Une corrélation élevée ne prouve pas que la gîte fait gagner ou perdre de la vitesse.

Constat (empannages) : les gîtes les plus fortes avant empannage (38,4° ; 42,5° ; 39,9°) n'ont pas produit de perte importante (0,29 ; 0,71 ; 0,02 kn). À l'inverse, des pertes de 1 kn ou plus se sont produites à 5–11° de gîte. **La gîte avant manœuvre ne prédit pas la perte.**

5. Manœuvres

Virements : aucun. Rien à analyser.

Empannages (16) :

Indicateur	Valeur
Rétention médiane (v_min / v_entrée)	0,951
Perte médiane (calculée)	≈ 0,22 kn
Perte maximale	2,85 kn (t = 1420 s)
Empannages avec perte > 1 kn	4 / 16
Empannages avec perte ≥ 0,5 kn	6 / 16
Reprise à 95 % mesurée (non nulle)	6 cas : 3, 4, 7, 13, 19, 20 s

Empannages les plus coûteux :

t (s)	Vitesse entrée → min (kn)	Perte (kn)	TWA avant → après (°)	Reprise 95 % (s)
1420	4,63 → 1,78	2,85	157 → 119	19
1400	5,51 → 4,40	1,12	141 → 157	0 (incohérent, voir ci-dessous)
1307	5,54 → 4,47	1,07	159 → 143	13
3101	6,28 → 5,22	1,06	158 → 147	20

Constat :

- Le groupe 1307–1449 s concentre 4 empannages en 142 s, dont 3 avec perte. La vitesse descend à 1,78 kn à 1420 s. Le 1449 s est entré à 1,86 kn, sans perte mesurée car déjà bas.
- Les événements avec forte perte s'accompagnent d'un changement d'angle important (11 à 38°). Les empannages « propres » (2 à 4° de variation) coûtent peu, voire rien.
- Les 3 empannages des 3924 s, 4501 s et 4290 s ont peu coûté, malgré une gîte élevée ou une entrée à vitesse moyenne.

Hypothèse : les pertes importantes viennent d'une sortie de cap ou d'une conduite hésitante pendant la manœuvre, plus que de la manœuvre elle-même. L'épisode 1400–1449 s pourrait être lié à une phase de vent faible, mais aucune transition de vent n'est enregistrée à cet instant.

Constat (incohérence) : l'empannage à 1400 s a une perte de 1,12 kn mais une reprise de 0 s. Soit la définition de « reprise » est différente, soit la donnée est fausse. **À vérifier avant de la comparer.**

Recommandation : pour chaque empannage, relever l'angle au vent et la vitesse en 3 instants (avant, pendant, après) et conserver la même fenêtre de mesure pour toutes les manœuvres.

Fig. 33. – Vue de la page de debrief, à 05:01:26, durant un bord de près (partie 2).

6. Transitions de vent

Douze transitions sont enregistrées, toutes entre 16 s et 457 s. **Aucune transition n'est détectée après 457 s** (couverture limitée à ~10 % de la séance).

Type	n	Δ vent moyen (kn)	Δ SOG moyen (kn)	Δ gîte moyen (°)
Risées	4	+3,5	+0,34	+1,4
Molles	8	–4,6	–0,03	+2,4

Constat :

- Les risées (+3,2 à +3,8 kn) font monter la SOG en moyenne de 0,34 kn, de façon inégale (3 hausses sur 4).
- Les molles (–3,1 à –7,7 kn) ne font presque pas baisser la SOG en moyenne. Elles s'accompagnent de hausses et de baisses.
- La réaction maximale observée est de +0,69 kn et –0,65 kn.

Hypothèse : la réponse de vitesse est modeste, mais la fenêtre de mesure de la SOG après chaque transition n'est pas précisée. Un retard de réaction ou une mesure trop courte peut masquer l'effet. La molle à 457 s (–7,7 kn) n'a produit aucune baisse visible.

7. Régularité

Indicateur (SOG, kn)	Valeur
Moyenne	4,68
Médiane	4,86
p10 / p25 / p75 / p90	3,18 / 4,03 / 5,42 / 5,79
Part du temps à ± 0,25 kn de la médiane	non calculable (le champ n'est renseigné que pour le près, 0 s)

Constat : la médiane (4,86) est supérieure à la moyenne (4,68). La distribution est étirée vers le bas : les 10 % les plus lents descendent sous 3,18 kn, ce qui correspond aux phases de vent faible et aux empannages.

Constat : la part du temps dans une zone cible n'est pas calculable faute de zone cible définie pour le portant. On peut seulement dire que 61 % du temps est passé entre 150° et 170° d'angle au vent arrière.

8. Synthèse

Forces

- Vitesse portante stable, médiane 4,86 kn.
- Angle de performance clair : VMG aval en plateau entre 160° et 180°, meilleure vitesse à 160°.
- Bonne conduite générale : la plupart des empannages coûtent moins de 0,4 kn.

Faiblesses

- 4 empannages sur 16 coûtent plus de 1 kn, dont un à 2,85 kn.
- Vitesse basse et dispersée (p10 à 3,18 kn).
- Aucune donnée de près : performance au près inconnue.

Points à investiguer

- Causes des empannages du groupe 1307–1449 s (conduite, vent, trajectoire).
- Phase de vent faible vers 1400–1450 s.
- Définition exacte de la « reprise » et des corrélations (incohérence à 1400 s et sur la corrélation 0,96).
- Signification de l'indicateur d'oscillation de vent (signe de l'angle).

3 priorités

1. **Empannages :** tenir l'angle au vent pendant l'empannage (variation cible < 10°) et éviter les séquences de plusieurs empannages rapprochés en conditions faibles.
2. **Angle de vent arrière :** se maintenir entre 150° et 170°, avec un VMG aval proche du plateau, plutôt que de descendre à 180° ou de monter sous 140°.
3. **Mesure :** enregistrer des virements (au moins 20 pour une analyse exploitable), corriger le TWA signé, couvrir toute la séance par la mesure du vent, et définir précisément les indicateurs de reprise.

Objectifs quantifiés pour la prochaine séance (à comparer à cette séance)

Indicateur	Actuel	Objectif
Empannages avec perte > 1 kn	4 / 16	≤ 1
Empannages avec perte ≥ 0,5 kn	6 / 16	≤ 2
Rétention médiane des empannages	0,951	≥ 0,97
Part du temps entre 150° et 170°	61 %	≥ 70 %
SOG médiane au portant	4,86 kn	≥ 4,86 kn (sans baisse)
Virements analysables	0	≥ 20

Indicateurs à suivre à chaque séance

- Perte de vitesse et temps de reprise (95 %) par empannage, avec la variation d'angle associée.
- Distribution de l'angle au vent arrière (part par tranche de 10°).
- SOG médiane et p10, en distinguant vent faible (< 6 kn) et vent établi.
- Réaction de la SOG aux risées et aux molles, sur une fenêtre fixe et documentée.
- Gîte moyenne et amplitude, mais sans conclusion sur la vitesse tant que la corrélation n'est pas confirmée sur plusieurs séances.

Généré par claude-haiku-5-5 le 21.10.2026 17:20:55. Basé sur cette seule séance : les causes proposées sont des hypothèses.

Fig. 34. – Vue de la page de debrief, à 05:01:26, durant un bord de près (partie 3).

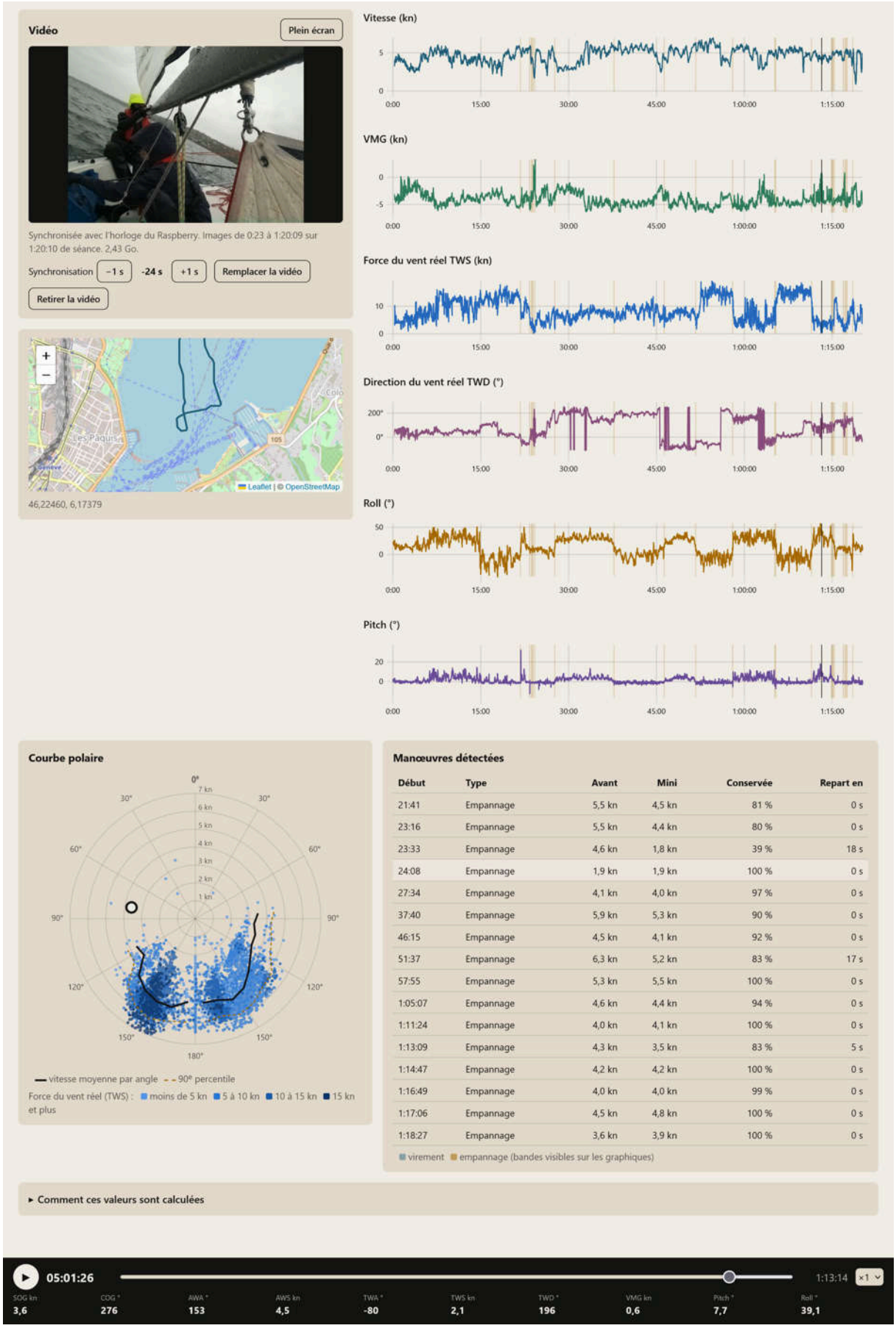


Fig. 35. – Vue de la page de debrief, à 05:01:26, durant un bord de près (partie 4).

11 CONCLUSION

Un peu plus d'un an après le critérium Longtze 2025, nous gagnions la Belle Étoile 2026[42]. Durant les deux saisons séparant ces deux événements, nos méthodes d'entraînement, de debrief et notre approche de la tactique ont été profondément retravaillées. Les différences tant de vitesse que d'angles sont observables, quantifiables. Néanmoins, la technique n'est pas le seul élément à avoir changé. L'équipage a aussi appris à mieux se connaître et à mieux fonctionner en tant que groupe. Une partie de la différence vient aussi de la nouvelle dynamique qui s'est installée. La performance en voile est une alchimie, et seule une partie de ce qui la compose est quantifiable.

11.1 Synthèse

Ce projet avait pour ambition de créer un dispositif permettant d'enregistrer des données dans le cadre d'un entraînement de voile inshore afin de donner un support tangible au debriefing et, par là, de donner l'opportunité à l'équipe l'utilisant de progresser.

Pour ce faire, il m'a fallu commencer par définir quelles étaient les données à collecter, puis concevoir un dispositif permettant de le faire, et enfin le produire. Mon projet s'est organisé selon l'architecture suivante : identifier le problème, se donner les moyens d'y remédier, mettre en œuvre une solution. Ce faisant, j'ai gardé en tête les cinq exigences que je m'étais fixées au début de mon travail :

- Abordabilité : alors que les solutions existant sur le marché affichent des fourchettes de prix s'étendant de 600 à 1100CHF (pour les équivalences directes, Orca et Vakaros), j'estime le prix de fabrication d'un second modèle identique au premier à environ 300 à 400CHF. Ce prix est difficile à estimer, car beaucoup des composants que j'ai utilisés ont été commandés séparément, ce qui a artificiellement fait augmenter leur prix à cause des multiples livraisons. De plus, si une réelle production était lancée, ce prix pourrait être drastiquement réduit.

- Adaptabilité aux clubs : le dispositif peut aisément être monté et démonté sur différents types de bateaux (montage testé sur Longtze, Skeeta, Viper F16, Nacra F16 et Open 5.70). Le fait de l'utiliser pour l'équipe de compétition ne complique aucunement l'utilisation du bateau par d'autres cours. Le seul inconvénient notable est la présence du câble de l'anémomètre, comme décrit plus bas.

- Open Source : le projet est placé dans son intégralité sous la licence MIT, y compris le présent rapport. L'entier du travail est disponible sur les dépôts GitHub Abraxas, Abraxas - serveur, Abraxas - fichiers 3D et Abraxas - PCB.

- No-nonsense : les résultats sont clairs, directs et facilement exploitables. Seule la rupture d'ergonomie dans l'affichage du debrief présentée ci-après est à déplorer.

[42] Première place temps réel petit parcours, première place temps compensé petit parcours, première place TCF3. « Classement de la Belle Étoile 2026 », *Manage2Sail*, 25.08.2026, https://chk.me/tlAsHST (consulté le 21.10.2026).

- Prise en compte multifactorielle : tous les facteurs mentionnés dans le cahier des charges sont mesurés. Il serait cependant possible et intéressant d'élargir cette fenêtre à d'autres paramètres, comme on le verra plus tard.

Le bilan de la réalisation elle-même peut être effectué sur deux plans principaux :

Sur le plan technique

La réalisation à proprement parler est réussie : le prototype fonctionne selon les attentes. On peut arguer que la durée de la sauvegarde de la vidéo ainsi que celle du démarrage du dispositif pourraient être optimisées. De plus, l'heure utilisée par le Raspberry n'est pas correcte, car son horloge s'arrête quand il est éteint. Cependant, ces problèmes n'entravent pas le bon fonctionnement du dispositif.

Sur le plan du bénéfice pour l'équipe

L'objectif de la collecte de données et de la possibilité d'analyse est atteint. Le panneau d'affichage du debrief est fonctionnel, le debrief en lui-même peut s'appuyer sur des données solides et permet de gagner en clarté lors d'une discussion concernant la navigation. Cependant, l'usage régulier du dispositif est limité par deux problèmes d'ergonomie importants. Tout d'abord l'installation du dispositif implique de relier l'anémomètre au boîtier par un câble. Or, cela représente un élément supplémentaire auquel il faut faire attention durant la navigation, ce qui ne peut pas être demandé d'une équipe en entraînement classique. De plus, le fait qu'il faille, pour accéder au debrief, commencer par télécharger les fichiers sur son téléphone avant de les transférer au serveur représente une rupture d'ergonomie qui peut décourager d'une utilisation régulière.

Malgré ces deux points d'amélioration possibles, je considère mon objectif initial atteint, au sens où j'ai apporté une réponse à ma problématique : j'ai identifié comment il était possible, grâce à un dispositif électronique, de collecter des données en vue de donner à une équipe l'opportunité de s'améliorer. J'ai ensuite produit un tel dispositif et vérifié qu'il représentait effectivement un avantage pour l'équipe qui l'utilisait, comme on a pu le constater lors de l'entraînement décrit dans le Chapitre 10.

11.2 Améliorations et perspectives

Résoudre ces deux ruptures dans l'ergonomie représenterait une amélioration majeure pour le système. Pour ce qui est de la suppression du câble, l'architecture n'aurait pas à être modifiée. En effet, j'utilise déjà du MQTT pour le transfert des données, et j'ai déjà un réseau ouvert auquel il est possible de se connecter. Ainsi, il suffirait de connecter l'anémomètre à un deuxième Raspberry et de faire transiter les messages MQTT par WiFi, ce qui est nativement supporté. En ce qui concerne le téléchargement des fichiers en local, mettre en place un système de transfert automatique demanderait de développer une application, car les pages internet ne prennent pas en charge la gestion du choix de réseau, et les automatisations nécessaires au transfert de ces fichiers. Remédier à ce problème représente un développement qui dépasse le cadre de ce projet.

Au terme de ce projet, je constate que certaines données auraient été au moins aussi, si ce n'est plus, intéressantes à mesurer. En effet, la tension dans les écoutes, dans le cunningham, dans le hale-bas et dans la bordure[43] sont des indicateurs extrêmement importants dans la compréhension du comportement du bateau et dans la reproductibilité des performances. Il aurait également été intéressant de filmer les voiles elles-mêmes afin de garder une trace de leur forme à un instant t. Il serait intéressant de créer un système de comptes sur le serveur d'analyse des données afin que plusieurs équipes différentes puissent avoir accès à leur propre debrief de façon privée.

11.3 Ouverture

Pour finir, je tiens à rappeler que ce système ne vise pas à remplacer l'entraîneur·euse dans une équipe. Je crois que la voile est et restera, malgré l'apparition de toujours plus d'électronique sur les bateaux, un sport fondamentalement humain. Il est une dimension dans la performance qui ne peut pas être mesurée : la dynamique de l'équipage. Il arrive un moment où, quand la connaissance de l'autre et la confiance mutuelle au sein de l'équipage sont totales, un regard suffit à prendre une décision. Les actions se succèdent naturellement, la coordination devient une évidence, car on connaît son équipier·ère comme soi-même et aucune de ses actions n'est une surprise. Dans les situations où le bateau est rapide, le vent fort et la mer formée, l'adrénaline, voire parfois la peur, font partie des éléments avec lesquels il faut composer pour atteindre une performance optimale. La confiance en soi et en l'autre est tout aussi primordiale que la réussite d'un empannage. Le travail d'un·e entraîneur·euse se situe également là, et je suis persuadé qu'aucun système informatique, pas même la plus moderne des IA ne peut remplacer cela.

[43] Différents réglages influant sur la forme de la voile.



Fig. 36. – Bord de portant lors de la Belle Étoile 2026. Nous sommes sur le Longtze Osteo 7/7.
Sans titre, 21.08.2026, collection personnelle,
avec l’accord de la photographe.

12 REMERCIEMENTS

Je tiens tout particulièrement à remercier mon professeur accompagnant dans le cadre de ce travail, pour sa disponibilité, son temps et ses conseils tant techniques que concernant la démarche et la structure du projet. Plus généralement, j'aimerais le remercier pour l'intérêt sincère qu'il a porté à mon travail, et d'avoir partagé avec moi certains de ses propres projets. Je n'ai pas trouvé en lui qu'un simple conseiller, mais une personne avec qui collaborer, et échanger des idées techniques.

Merci également à mon professeur de CAO, responsable du laboratoire de photographie argentique et ami, de m'avoir mis à disposition l'imprimante 3D de l'école et de m'avoir accompagné et conseillé pour la conception de mon boîtier. Plus largement, mes remerciements vont à l'ensemble des professeur·e·s qui m'ont apporté curiosité, méthode et connaissance durant mes années au collège.

Merci à mon entraîneur, pour ses conseils véliques et techniques, et surtout pour son indéfectible amitié. J'aimerais aussi remercier le chef de base de mon club, pour l'intérêt qu'il a porté à mon travail.

Merci à Arnaud Psarofaghis et à la philosophe consultée sur l'usage de l'IA, spécialistes dans leur domaine respectif, pour leurs conseils éclairés.

Merci à mes parents pour leur soutien, leur intérêt pour mon projet, le courage dont ils ont fait preuve dans la relecture des parties les plus techniques et leur patience lorsque j'étais totalement absorbé dans mon travail.

Merci à mon équipier, grâce à qui je sais que la voile est, en compétition comme ailleurs, avant tout une question de confiance et d'amitié. Il m'a appris qu'aucun capteur ne peut rendre compte de ce qu'est véritablement un équipage, sur l'eau comme à terre.

Merci à mes deux coéquipières pour leur présence dans les turbulentes premières heures de tests du projet, et, surtout, d'avoir su les relever de vos rires. Vous m'avez montré que c'est lorsqu'aucun problème ne vient à bout d'un sourire que l'on est prêt·e à prendre la mer.

Merci à mes autres coéquipier·ères et aux membres du club. J'ai navigué avec vous, et sans vous la voile ne signifierait pas la même chose pour moi.

Merci pour finir à vous toustes que je n'ai pas pu nommer, mais sans qui ce travail ne serait pas le même.



Fig. 37. – Photographie prise lors du SailGP 2025 à Genève.

13 BIBLIOGRAPHIE

13.1 Ouvrages

- HESSE Hermann, *Demian. Die Geschichte von Emil Sinclairs Jugend*, Berlin, Suhrkamp, 2012.

13.2 Entretiens et sources orales

- Philosophe spécialiste des questions relatives à l'IA, discussion du 09.06.2026.
- Entraîneur de l'équipe de compétition du Yacht Club de Genève, *Cours théoriques, briefing et debriefing*, connaissances orales non publiées.
- PSAROFAGHIS Arnaud, Compétiteur professionnel, discussion du 22.09.2026.

13.3 Systèmes existants

- « Wearables smartwatches », *Garmin*, s.d., https://www.garmin.com/fr-CH/c/wearables-smartwatches/ (consulté le 11.10.2026).
- « Get Orca », *Orca*, s.d., https://getorca.com/product-configuration/ (consulté le 11.10.2026).
- « Atlas 2 », *Vakaros*, s.d., https://www.vakaros.com/en-eu/collections/atlas-family/products/atlas2 (consulté le 11.10.2026).

13.4 Documentation technique et dépôts de code

- HERRADA Eva, « Adafruit Ultimate GPS », *Adafruit*, 20.08.2026 (dernière mise à jour), https://cdn-learn.adafruit.com/downloads/pdf/adafruit-ultimate-gps.pdf (consulté le 21.10.2026).
- « Adafruit Powerboost 1000C », *Adafruit*, 08.03.2024, https://learn.adafruit.com/adafruit-powerboost-1000c-load-share-usb-charge-boost (consulté le 21.10.2026).
- « Camera software », *Raspberry Pi*, s.d., https://www.raspberrypi.com/documentation/computers/camera_software.html (consulté le 21.10.2026).
- HERRADA Eva et LEBLANC Williams, « Running CircuitPython Code without CircuitPython », *Adafruit*, 05.07.2023 (dernière mise à jour majeure), https://chk.me/O2xfxk0 (consulté le 21.10.2026).
- HERRADA Eva et LEBLANC Williams, « Installing Blinka on Raspberry Pi », *Adafruit*, 05.07.2023 (dernière mise à jour majeure), https://chk.me/Y40lkS7 (consulté le 21.10.2026).
- « OnOff SHIM », *Pimoroni*, s.d., https://shop.pimoroni.com/en-ch/products/onoff-shim (consulté le 21.10.2026).
- HERRADA Eva *et al.*, « Adafruit_CircuitPython_GPS », *GitHub*, 09.09.2026 (dernier commit), https://github.com/adafruit/Adafruit_CircuitPython_GPS (consulté le 21.10.2026).

- HOWARD Philip *et al.*, « icm20948-python », *GitHub*, 28.06.2024 (dernier commit), https://github.com/pimoroni/icm20948-python (consulté le 21.10.2026).
- « User Manual », *Hongyuv*, 30.04.2024, https://www.hongyuv.com/download/WDC2E_Manual.pdf (consulté le 21.10.2026).
- « Calculateur de largeur de piste de circuit imprimé », *DigiKey*, s.d., https://www.digikey.be/fr/resources/conversion-calculators/conversion-calculator-pcb-trace-width (consulté le 21.10.2026).

13.5 Articles et pages web

- « Anémomètre » in *Wikipedia*, s.d., https://chk.me/XaOHDIC (consulté le 21.10.2026).
- AVONTURE Christophe, « Linux - Using a progression bar in your script », *avonture.be*, 07.09.2024, https://www.avonture.be/blog/bash-progression-bar/ (consulté le 21.10.2026).
- EYGAY Erwan, « JSON ou SQL : Quel format choisir pour gérer vos données ? », *DataBird*, 19.03.2026 (dernière mise à jour), https://www.data-bird.co/blog/json-ou-sql (consulté le 21.10.2026).
- FAULKNER Andrew et BLANC Florian, « Avantages et faiblesses du protocole SSH », *Hakin9*, n°2, 2008, https://chk.me/qm9L8t2 (consulté le 21.10.2026).
- « Arnaud Psarofaghis », *Alinghi*, s.d., https://www.alinghi.com/crew/33_ArnaudPsarofaghis# (consulté le 21.10.2026).
- « Arnaud Psarofaghis », *SailGP*, s.d., https://sailgp.com/teams/switzerland/arnaud-psarofaghis/ (consulté le 21.10.2026).

13.6 Résultats de régates

- « Classement de la Belle Étoile 2026 », *Manage2Sail*, 25.08.2026, https://chk.me/tlAsHST (consulté le 21.10.2026).
- « Classement du Criterium Longtze YCG 2025 », *Manage2Sail*, 04.05.2025, https://www.manage2sail.com/fr/event/Longtze2025 (consulté le 21.10.2026).

14 ANNEXES

14.1 Photos du prototype



Fig. 38. – Montage de l'anémomètre sur le pont du Longtze à l'aide d'une ventouse. Sans titre, 22.10.2026, collection personnelle, avec l'accord du photographe.

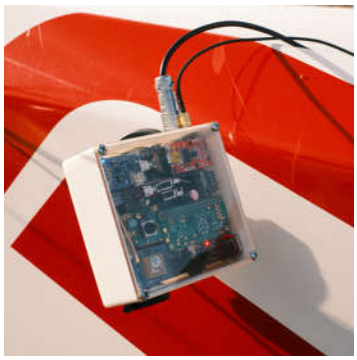


Fig. 39. – Abraxas sur l'étrave d'un Viper F16.

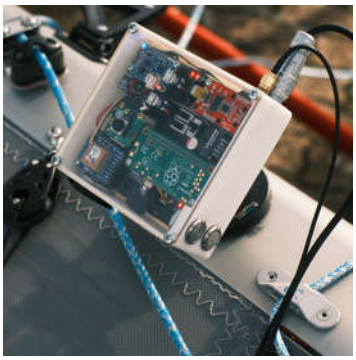


Fig. 40. – Abraxas sur la poutre d'un Nacra F16.



Fig. 41. – Abraxas sur le mât d'un Longtze.

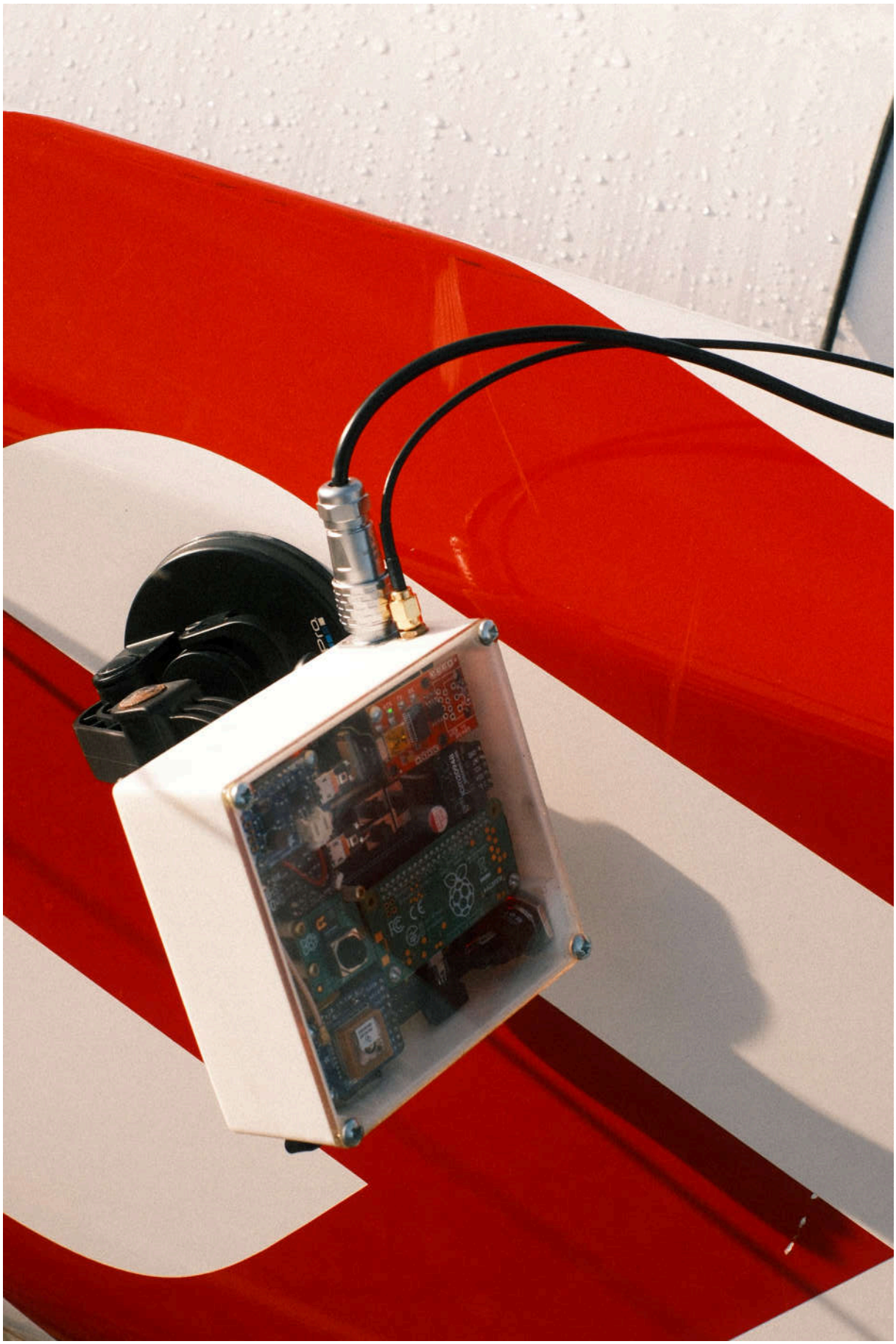


Fig. 42. – Abraxas sur l’étrave d’un Viper F16.



Fig. 43. – Abraxas sur l’étrave d’un Viper F16.

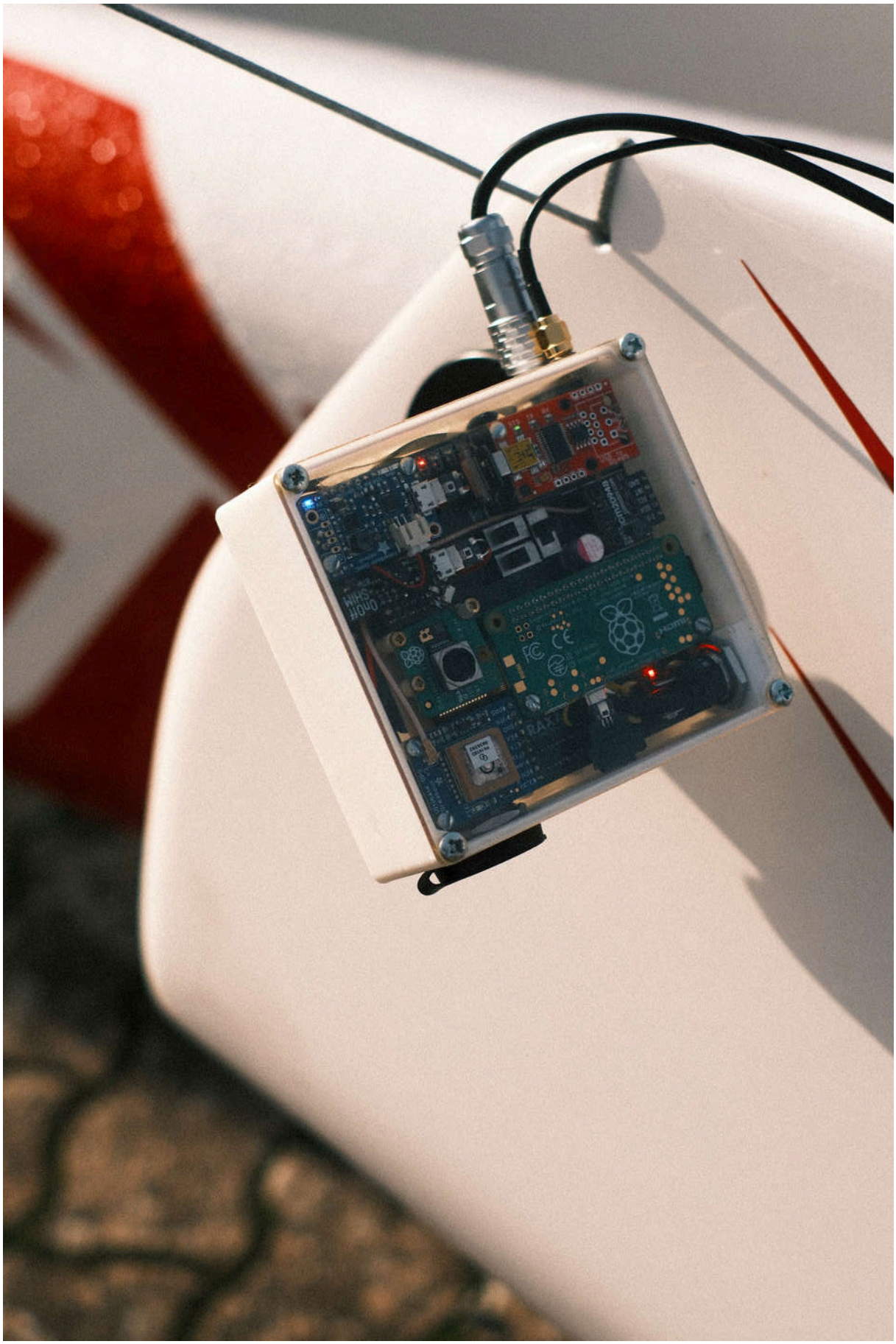


Fig. 44. – Abraxas sur l’étrave d’un Viper F16.

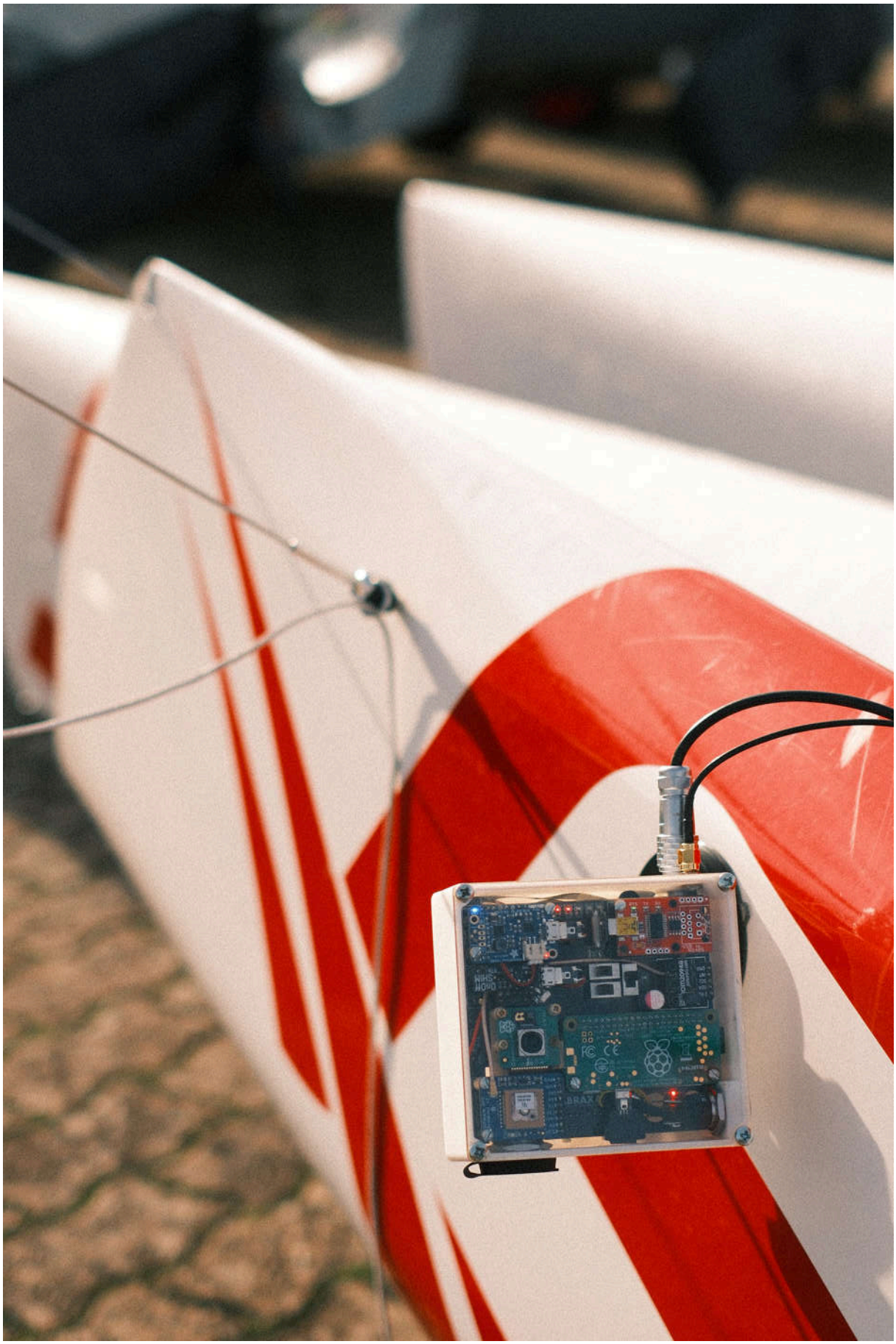


Fig. 45. – Abraxas sur l’étrave d’un Viper F16.



Fig. 46. – Anémomètre sur l’étrave d’un Viper F16.



Fig. 47. – Anémomètre sur la poutre d’un Nacra F16.

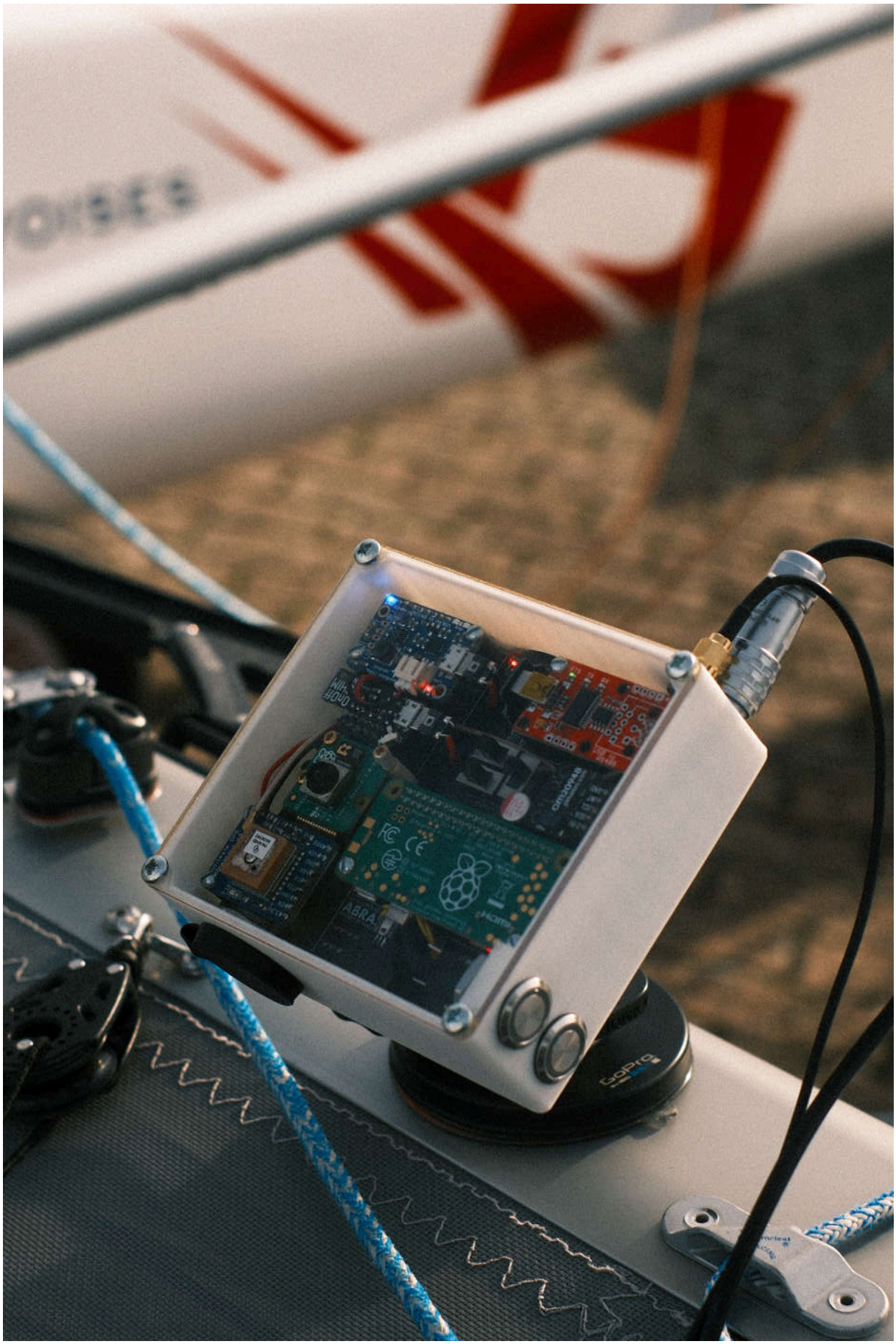


Fig. 48. – Abraxas sur la poutre d’un Nacra F16.

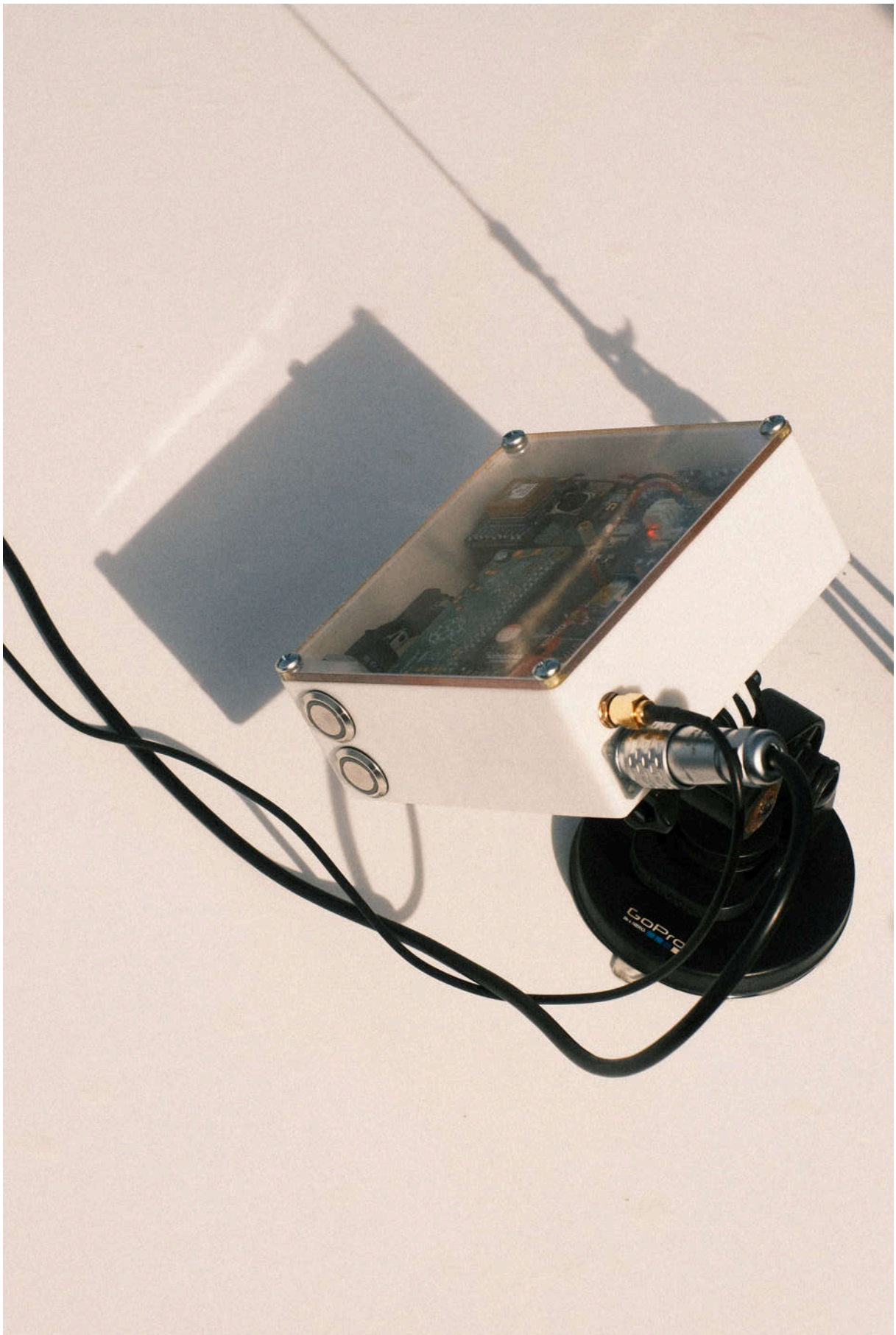


Fig. 49. – Abraxas sur la plage avant d’un Skeeta.



Fig. 50. – Anémomètre sur la plage avant d’un Skeeta.



Fig. 51. – Anémomètre sur la plage avant d’un Longtze.



Fig. 52. – Anémomètre sur la plage avant d’un Longtze.



Fig. 53. – Anémomètre sur l’étrave d’un Longtze.

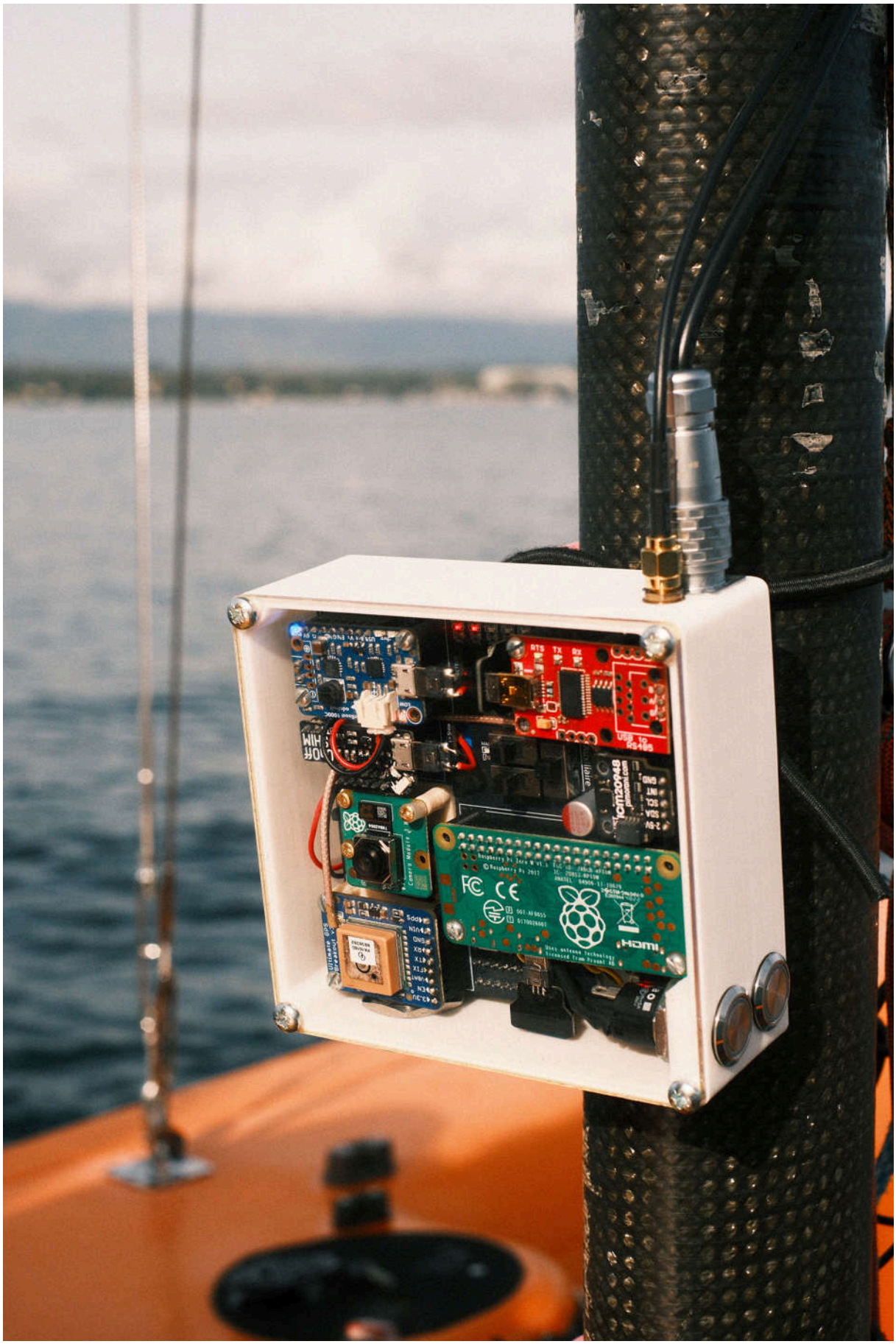


Fig. 54. – Abraxas sur le mât d’un Longtze.

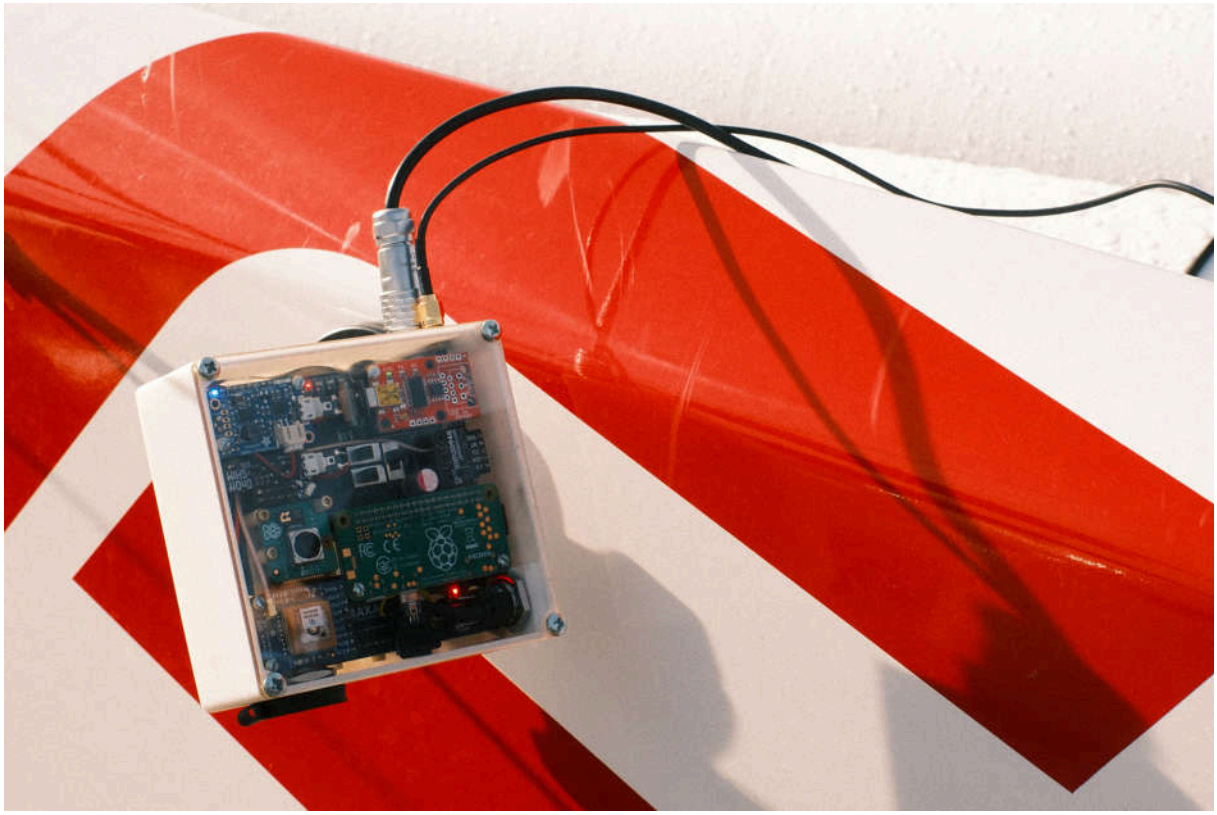


Fig. 55. – Abraxas sur l’étrave d’un Viper F16.

14.2 Accès au PDF du rapport et aux dépôts GitHub

PDF du rapport

Abraxas (codes)

Release d’Abraxas

Fichiers 3D

Release des fichiers 3D

Serveur

Release du serveur

PCB

Release du PCB

14.3 Accès au modèle CAD

Modèle OnShape

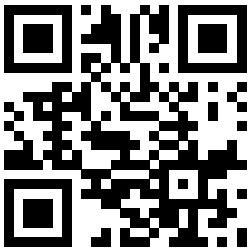
14.4 Accès au serveur

Serveur d’analyse

Import des données

Debrief de l’entraînement test

14.5 Accès au hotspot



Visualisation des données en direct



Export des données

14.6 Sites des solutions existantes



Garmin



Vakaros Atlas 2



Orca

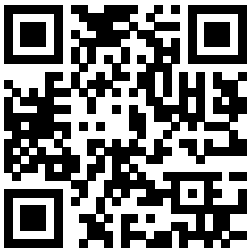
14.7 Matériel



IMU ICM20948 (Pimoroni)



Raspberry Pi Zero 2 WH



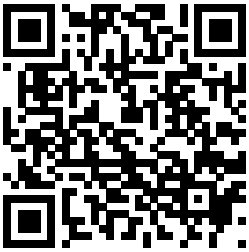
GPS MTK3339 (Adafruit)



Caméra Raspberry Pi V2



PowerBoost 1000C



Batterie LiPo 8000 mAh



Nappe CSI

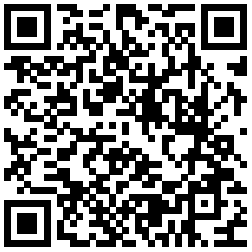


Anémomètre HY-WDC2E


Prise micro-USB encastrable


Prise USB-A encastrable


ON/OFF SHIM


USB/RS485 WaveShare


USB/RS485 SparkFun


Connecteur mâle


Connecteur femelle


Bouton IP67

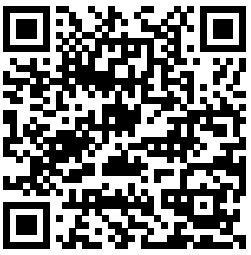

Hub USB Zero4U


Antenne GPS active


Connecteur d’antenne

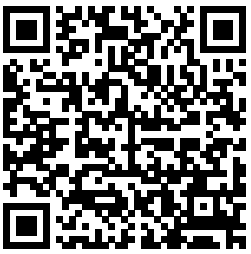

Câble USB coudé


Fiches micro-B


Convertisseur USB/RS485 SparkFun

14.8 Documentation technique


Adafruit Ultimate GPS (PDF)


Adafruit PowerBoost 1000C


Pimoroni OnOff SHIM


Manuel de l’anémomètre


Camera software (Raspberry Pi)


CircuitPython sans CircuitPython


Installer Blinka


Dépôt Adafruit CircuitPython GPS


Dépôt icm20948-python


Calculateur de largeur de piste (DigiKey)

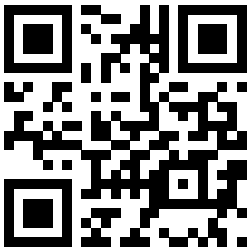
14.9 Articles et pages web


Avantages et faiblesses du protocole SSH


Anémomètre (Wikipédia)


JSON ou SQL (DataBird)


Barre de progression en bash


MyIP


Critérium Longtze 2025


Belle Étoile 2026


Page Alinghi


Page SailGP

14.10 Accès aux codes

L’accès au code se fait via les dépôts GitHub.

Imprimé chez Trajets le 22 octobre 2026

Trajets est une entreprise sociale active depuis 1979 dans l’intégration sociale et professionnelle de personnes vivant avec des difficultés psychiques